

國立成功大學
電機工程研究所
碩士論文

可應用於高能量效益多位元卷積神經網路的邊緣
裝置之 8T SRAM 記憶體內運算

**An 8T SRAM-based Compute-In-Memory for Highly
Energy-Efficient Multiple-Bit Convolutional Neural
Network Edge Devices**

研 究 生：林育緯

Student: Yu-Wei Lin

指導教授：邱瀝毅 博士

Advisor: Lih-Yih Chiou, Ph.D.

Department of Electrical Engineering
National Cheng Kung University
Tainan, Taiwan, R.O.C.
Thesis for Master of Science

July 2020

中華民國 一百零九 年 七 月

國立成功大學

碩士論文

可應用於高能量效益多位元卷積神經網路的邊緣裝置
之8T SRAM記憶體內運算

An 8T SRAM-based Compute-In-Memory for Highly
Energy-Efficient Multiple-Bit Convolutional Neural
Network Edge Devices

研究生：林育緯

本論文業經審查及口試合格特此證明

論文考試委員：

張順志

許明華

邱瀝毅

指導教授：

邱瀝毅

系(所)主管：

楊宏澤

中華民國 109 年 7 月 23 日

An 8T SRAM-based Compute-In-Memory for Highly
Energy-Efficient Multiple-Bit Convolutional Neural
Network Edge Devices

By

Yu-Wei Lin

A thesis submitted to the graduate division in partial
fulfillment of the requirement for the degree of
Master of Science

at

National Cheng Kung University
Tainan, Taiwan, Republic of China
July 2020

Approved by :

Ming Hua Shen

Soon-Jyh Chang

Lih-yih Chion

Advisor :

Lih-yih Chion

Chairman :

Ang-Jen Yang

可應用於高能量效益多位元卷積神經網路的邊緣裝置 之 8T SRAM 記憶體內運算

學生：林育緯

指導教授：邱瀝毅 博士

國立成功大學電機工程研究所 VLSI/CAD 組

摘要

近年來，人工智慧的高速發展與廣泛的應用，尤其以卷積神經網路為代表，像是影像辨識、物件辨識與語言翻譯等等，為了將人工智慧實現於邊緣裝置與物聯網裝置上，達到低能量消耗、低延遲與高安全性，因此發展出了各式的特殊應用積體電路，即人工智慧加速器。與傳統數位加速器相比，記憶體內運算為一種新興的 AI 加速器計算單元，將輸入資料與權重在記憶體電路中，直接進行乘法與累加的運算，將能避免傳統數位加速器過多資料搬移的現象，即記憶體牆之問題，因此大幅降低功率的消耗與計算的延遲時間。

本論文為採用 40 奈米製程設計，提出具有高能量效益與支援多位元乘加運算的 8T SRAM 記憶體內運算，利用權重位元切割累加技術，降低整體多位元計算時的量化誤差與輸出級對於單一高解析度資料轉換器的需求，避免過大的功率消耗；使用時間調變技術，降低數位類比轉換器的使用數量，從 $n \times n$ 個降低為一個，因此對於面積與功率消耗皆能獲得改善。與過去相關文獻相比，本論文提出的記憶體內運算，在整體電路的能量效益、操作速度與運算精準度均有較佳的比較性與競爭優勢。

關鍵字：人工智慧、卷積神經網路、記憶體內運算、權重位元切割累加、時間調變

An 8T SRAM-based Compute-In-Memory for Highly Energy-Efficient Multiple-Bit Convolutional Neural Network Edge Devices

Student: Yu-Wei Lin

Advisor: Lih-Yih Chiou, PhD

Department of Electrical Engineering
National Cheng Kung University,
Tainan, Taiwan, R.O.C

SUMMARY

In recent years, with fast growing amount of artificial intelligence (AI) applications, especially, convolutional neural networks (CNNs) have successfully implemented to image recognition, object recognition and language translation etc. To extend AI at the terminal side (referred to as edge) of internet of things (IoTs), it is required to achieve low energy consumption, low latency and high security. The application-specific integrated circuit (ASIC), i.e. AI accelerators, have been developed for edge devices. Compute-in-memory (CIM) is one of emerging computation paradigms. When compared with digital AI accelerators, CIM allows the operation of multiply-and-accumulate (MAC) operations inside memory. Therefore, CIM can help to mitigate data congestion between the processor and memory, i.e. memory wall issue. The computation paradigm promises less power consumption and latency than its counter part, von Neumann architectures.

In this thesis, we present a highly energy-efficient 8T SRAM-based CIM to support multiple-bit MAC operations, which use a weight bit-partition accumulation (WBPA) method to decrease quantization errors of multiple-bit MACs operation and reduce the output data converter overhead while maintaining precision. Moreover, for reducing the number of digital-to analog converter (DAC) from $n \times n$ to 1, the time modulation mechanism applies the proposed CIM to greatly improve area and power consumption. The whole circuit was fabricated as a test chip using 40-nm technology. When compared with the typical SRAM-based CIM, the proposed CIM is more competitive in energy-efficient, inference time and output precision.

Key words: Artificial intelligence (AI), convolutional neural networks (CNNs), compute-in-memory (CIM), weight bit-partitioned accumulation (WBPA), time modulation

INTRODUCTION

With wide-spreading applications, the artificial intelligence (AI) brings more convenience life in recent years. Among the artificial neural networks, convolutional neural networks (CNNs) have gaining popularity and are suitable for applications, such as object recognition, image classification, speech recognition, etc. However, the requirement for high accuracy of CNNs comes at the cost of huge amount of computation and storage. Therefore, it will be very challenging to lower power consumption of edge devices for internet of things (IoTs).

Digital AI accelerator have been developed to implement CNNs inference on edge device and worked to reduce power consumption. However, the major bottleneck for the digital AI accelerators lies in the memory access. Each multiply-and-accumulate (MAC) operations need to take data from the memory for calculation and write intermedium or final results back to the memory at the expense of high energy consumption and large delay times. Take AlexNet as an example, there are 724 million MAC operations and nearly 3 billion memory access in the worst case.

Compute-in-memory (CIM) is a novel computation paradigm. MACs operation can be processed inside memory macro. Therefore, it can reduce the amount of intermediate data between computation and memory. A processing elements (PE) consisting of CIM macros can perform MAC operations in blocks. Subsequently, the throughput can be increased significantly by activating multiple rows of CIM simultaneously. In short, CIM can not only cut down the cost of memory access time and energy consumption, but also increase throughput via parallelizing computation.

PROPOSED DESIGN

The proposed CIM employed 8T SRAM cells for multiple-bit weight and multiple-bit input feature map (IFMP) to perform MAC operations, the expression for the

convolution output can be derived as:

$$\vec{Y}_1 = \sum_{i=0}^n \vec{X}_i \times \vec{W}_i \quad (1)$$

, where $\vec{X}_i$ is IFMP, $\vec{W}_i$ is weight, $\vec{Y}_i$ is convolution output and n is filter size. A 3×3 filter with 5-bit weights, 4-bit IFMPs and 8-bit MACs is implemented the proposed CIM in this thesis. The whole structure is comprising a 54-bit weight block, a 4-bit pulse-width generator (PWG), a voltage-to-time converter (VTC) array, a 4-bit time-to-digital converter (TDC) array, and an 8-bit output combiner (OC).

The 8T SRAM-based CIM has two operation modes: the write mode and the computing mode for convolutional operations. In the write mode, the weight data are written in 8T SRAM cells and only one word-line (WL) is driven at a time. In the computing mode, multiple RWLs are activated synchronously by PWG and a driving-width pulse depends on multiple IFMP value, i.e. time modulation. The execution of multiple MACs operation is processed in weight block. Partial multiple MACs are converted into time signals START and STOP by the VTC. The time signal is quantized into partial digital output code by the TDC. Lastly, partial output codes are combined using the OC.

The 8T SRAM cells consist of a conventional 6T bit-cell for writing weight data and 2T for the compute-port. The bit-cell provides widely-use CMOS technology and prevents write disturb issue, i.e. the memory cells suffer data flipping, for noise margin improvement. The details for the MAC of binary-bit weight and multiple-bit IFMP operation can be divided three steps: first, the RWL is turned on by PWG and the pulse-width is modulated depending on the IFMP. Next, when the 8T SRAM is stored as a digital “1” or a digital “0”, representing the 1-b filter weight value is “1” or “0”, respectively, the output memory cell charge (Q_{MC}) is based on the pulse-width of RWL. Therefore, this proceed represents input-weight-product which ΔQ is generated by the

variable RWL pulse and a fixed memory cell current. Finally, all Q_{MC} of $n \times n$ 8T SRAM cells are summed at the RBLs sampling capacitor of the column-wise.

The proposed CIM macro utilize the weight bit-partitioned accumulation (WBPA) method to realize the multiple-bit weight operation. We adopt a digital computation and several sub-data converters to replace the single high-resolution data converter at high-precision operation. Thus, comparing with previous works, the implementation of WBPA method can avoid non-ideal effect of analog computation which like charge sharing, to reduce the quantization errors of the high-precision output and prevent the high-resolution data converter to use, i.e. the proposed method reduces the data converter overhead while maintaining sufficient output precision, to achieve low power consumption and high conversion rate.

EXPERIMENT RESULTS

We simulate and analyze the design by HSPICE using 40nm CMOS technology and MATLAB. The overall simulation INL is shown in Figure 1. The INL is $-0.58 \text{ LSB} \sim 0.91 \text{ LSB}$. The operation frequency varies from 100 MHz to 400 MHz, energy-efficient of the operation frequency variation is 23.5 TOPS/W to 26.3 TOPS/W and the energy-efficient of the corner variation is 19.4 TOPS/W to 25.4 TOPS/W. Figure 2 shows the energy breakdown of the proposed CIM.

CONCLUSION

In this thesis, we propose a highly energy-efficient 8T SRAM-based CIM with the WBPA methodology to reduce the quantization errors of the multiple-bit MACs operation. Additionally, the time modulation mechanism employed to avoid the number of digital-to analog converter from $n \times n$ to 1. When compared with previous related

works, the proposed design using 40nm CMOS technology features high energy efficiency, fast inference time and high-precision output and suits for low power CNN applications like IoTs.

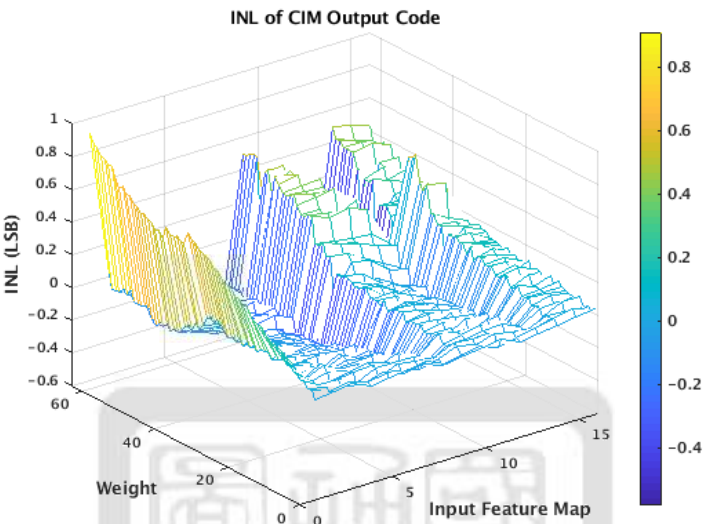


Figure 1. INL of proposed 8T SRAM-based CIM

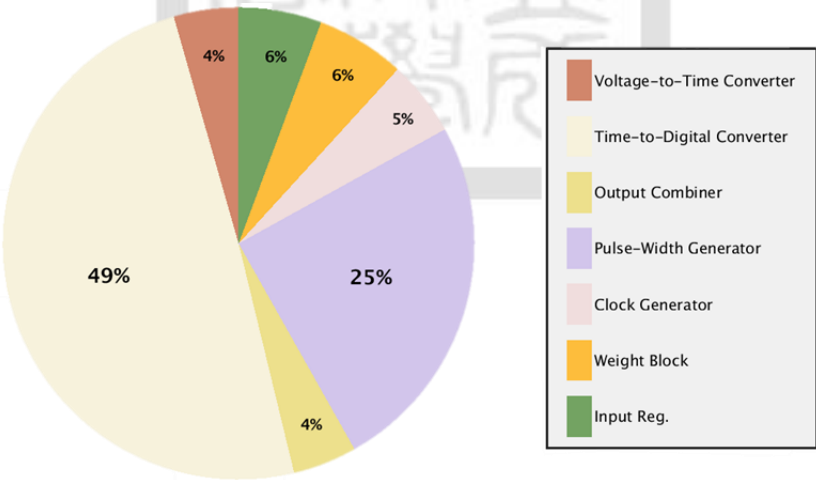


Figure 2. Energy breakdown of proposed 8T SRAM-based CIM

致謝

首先特別感謝指導教授 邱瀝毅老師，在我就讀研究所期間，不管是在修課或研究上，給予了我相當多寶貴的指引與建議，使我在邏輯思考、投影片制作、語言及研究等能力得以精進，讓碩士生涯能夠獲益匪淺。此外也感謝大學專題老師 丁信文老師，帶領我踏入 IC 設計這浩瀚的領域，探索無垠的知識。同時也要謝謝口試委員 張順志老師以及 許明華老師，在百忙之中予以論文建議與修改，使這篇碩士論文的內容可以更加完整詳盡。

另外由衷感謝 LPHP 實驗室的學長姐們：境昱、易溥、品修、宗翰、維軒、忠翰、哲彬、俊興、健棠、晉維、柏丞，在修課、研究、計畫撰寫及生活上給予我許多的建議及幫助，開拓了對研究上的視野。以及感謝我的同學們：憲譚、峻豪、昱宏、紀寬、唯宏，在遇上難題時得以互相討論學習及勉勵，同舟共濟度過三年的回憶長廊。而實驗室的學弟妹們：宗蕙、偉碩、志展、聖凱、錫峰、Tenzing、韻如、琬筠、心妤、中凱、晨哲、鎮中、子毅，謝謝你們在各方面的幫助，實驗室有你們的加入也增添了一分歡笑與活力。

最後誠摯感謝我摯愛的父母及妹妹，在我求學過程中，無怨無悔的無私付出及忍受我的嘮叨與抱怨，讓我能夠無後顧之憂完成碩士學業，在此與你們分享此刻的喜悅。最後的最後感謝所有在求學道路上幫助過我及關心的人，至上最深的感激！

林育緯 謹誌

中華民國一百零九年七月 府城台南

目錄

摘要.....	i
致謝.....	vi
目錄.....	vii
圖目錄.....	ix
表目錄.....	xiii
第 1 章 緒論.....	1
1.1 研究概觀.....	1
1.1.1 發展背景.....	1
1.1.2 記憶體內運算.....	5
1.1.2 相關名詞介紹.....	8
1.1.2.1 記憶體內運算名詞.....	8
1.1.2.2 資料轉換器名詞.....	9
1.1.3 記憶體內運算之分類.....	11
1.1.3.1 非揮發性記憶體內運算.....	12
1.1.3.2 揮發性記憶體內運算.....	15
1.2 研究動機.....	16
1.3 論文貢獻.....	19
1.4 論文架構.....	20
第 2 章 相關研究文獻	21
2.1 10T SRAM-based 記憶體內運算	21
2.2 Twin-8T SRAM-based 記憶體內運算.....	25
2.3 8T SRAM-based 記憶體內運算	30
2.4 相關文獻總結.....	32
第 3 章 具權重位元切割累加機制之 8T SRAM 記憶體內運算	35

3.1 設計概述.....	35
3.2 設計架構.....	36
3.3 權重計算區塊.....	37
3.4 脈衝寬度產生器.....	41
3.5 電壓時間轉換器.....	44
3.6 時間數位轉換器.....	47
3.7 權重位元切割累加機制.....	49
第 4 章 實驗結果與分析	54
4.1 電路操作模擬.....	54
4.1.1 權重計算區塊操作模擬及線性分析.....	54
4.1.2 取樣電容陣列.....	56
4.1.3 8T SRAM 運算元之靜態雜訊邊際分析.....	57
4.1.4 脈波寬度產生器操作模擬及穩定性分析.....	60
4.1.5 電壓時間轉換器操作模擬及穩定性分析.....	62
4.1.6 時間數位轉換器操作模擬及效能分析.....	64
4.2 記憶體內運算參數模擬.....	67
4.2.1 非線性分析.....	67
4.2.2 能量效益分析.....	68
4.2.3 電路能量佔比.....	70
4.3 晶片佈局.....	71
第 5 章 結論與未來研究方向	72
5.1 結論.....	72
5.2 未來工作.....	74
參考文獻.....	76

圖目錄

圖 1-1	DNN 演算法之演進[6].....	2
圖 1-2	不同處理器之運算特性差異	3
圖 1-3	不同記憶體階層與運算單位，資料所需能量消耗[13].....	4
圖 1-4	數位加速器與 CIM 之運算流程比較[17].....	5
圖 1-5	數位加速器與 CIM 之能量比較[18].....	6
圖 1-6	CIM 與 NN 演算法之比較圖	7
圖 1-7	資料轉換器之 DNL 示意圖	10
圖 1-8	資料轉換器之 INL 示意圖	10
圖 1-9	CIM 概略分類.....	12
圖 1-10	記憶體單元之操作速度與電壓比較[35].....	13
圖 1-11	nvCIM 與 NN 演算法之對應圖	14
圖 1-12	新興記憶體之阻值比較[36].....	15
圖 1-13	傳統 6T SRAM 單元電路	15
圖 1-14	AIoT 市場預測趨勢[6].....	17
圖 1-15	CIM 精準度發展趨勢[17].....	18
圖 1-16	SRAM-based CIM 電流式累加曲線[34]	19
圖 1-17	電流式累加時，有無校正機制比較曲線[33].....	19
圖 2-1	BNNs 對應於 CIM 電路之計算概念[23]	22
圖 2-2	10T SRAM-based 記憶體內運算[23]	22
圖 2-3	GBL DAC 電路架構與操作波型[23]	23
圖 2-4	10T SRAM array 電路架構與操作波型[23].....	24
圖 2-5	Charge Sharing ADC 電路架構與波型[23]	24
圖 2-6	T8T SRAM-based CIM 整體架構[20]	25
圖 2-7	Twin-8T SRAM cell [20].....	27

圖 2-8 C2WM 機制電路[20].....	28
圖 2-9 全域參考電壓產生器電路與波型[20].....	29
圖 2-10 8T SRAM-based CIM 整體架構[22].....	30
圖 2-11 標準 8T SRAM 架構圖	31
圖 2-12 以計數器轉換輸入之概念[22].....	31
圖 2-13 8T SRAM 陣列圖[22].....	32
圖 3-1 記憶體內運算架構圖	37
圖 3-2 Weight block 整體架構圖	38
圖 3-3 8T SRAM 計算單元	39
圖 3-4 當權重為 1 時，計算元操作情況	39
圖 3-5 當權重為 0 時，計算元操作情況	39
圖 3-6 當 weight block 操作於 ϕ_1 時之運算情況.....	40
圖 3-7 當 weight block 操作於 ϕ_2 時之運算情況.....	41
圖 3-8 時間脈波寬度產生器	42
圖 3-9 脈波寬度產生器操作波型	43
圖 3-10 有無時間調變機制之功率比較	44
圖 3-11 製程與偏差電壓曲線圖[39].....	45
圖 3-12 製程與偏差時間曲線圖[39].....	45
圖 3-13 傳統動態電壓時間轉換器[39].....	46
圖 3-14 具定電流校正之 VTC (a) 架構與 (b) 操作波型	46
圖 3-15 VTC 輸入電壓與輸出時間補償前後轉換曲線	47
圖 3-16 Vernier TDC (a) 架構與 (b) 波型.....	48
圖 3-17 數位與類比電路之 SNR 比較[45].....	49
圖 3-18 直式二進制乘法	50
圖 3-19 輸出結合器	51

圖 3-20 傳統 CIM 與本論文 CIM 之運算流程比較	52
圖 3-21 有無位元切割機制之 TDC 細胞數量比較	53
圖 4-1 Weight block 輸出累加模擬 (post-sim.)	55
圖 4-2 Weight block 輸入輸出轉換曲線 (post-sim.)	56
圖 4-3 MOM 單位電容	56
圖 4-4 取樣電容佈局陣列	57
圖 4-5 8T SRAM HSNM 分析模型與波型	58
圖 4-6 8T SRAM RSNM 分析模型與波型	58
圖 4-7 當操作電壓為 0.9V 時的 HSNM (pre-sim.)	59
圖 4-8 當操作電壓為 0.9V 時的 RSNM (pre-sim.)	59
圖 4-9 當 IFMP = 4b'0000 時, PWG 輸出變異情況 (post-sim.)	61
圖 4-10 當 IFMP = 4b'1111 時, PWG 輸出變異情況 (post-sim.)	61
圖 4-11 4-bit PWG 之 INL 模擬 (post-sim.)	62
圖 4-12 VTC 轉換過程模擬 (post-sim.)	62
圖 4-13 VTC 之輸入電壓與輸出時間轉換曲線模擬 (post-sim.)	63
圖 4-14 VTC 增益之 corner 變異模擬 (post-sim.)	64
圖 4-15 4-bit TDC 量化過程模擬 (post-sim.)	64
圖 4-16 4-bit TDC 動態效能模擬 (post-sim.)	65
圖 4-17 4-bit TDC FFT 模擬頻譜圖 (post-sim.)	65
圖 4-18 4-bit TDC 輸入輸出轉換曲線 (post-sim.)	66
圖 4-19 4-bit TDC 之 DNL 模擬 (post-sim.)	66
圖 4-20 4-bit TDC 之 INL 模擬 (post-sim.)	67
圖 4-21 CIM 輸出非線性曲線 (pre-sim.)	68
圖 4-22 操作頻率對能量效益之變化 (post-sim.)	69
圖 4-23 Corner 對能量效益之變化 (post-sim.)	70

圖 4-24 電路區塊之能量佔比 (post-sim.)	70
圖 4-25 記憶體內運算晶片佈局	71



表目錄

表 1-1	DNN 演算法之權重與 MAC 總數比較	4
表 2-1	2b×2b 累加增值表[20].....	26
表 2-2	EODC 機制下的輸入映射對照[20].....	27
表 2-3	相關文獻之比較	34
表 4-1	操作電壓與 RSNM 趨勢關係 (pre-sim.)	60
表 5-1	相關文獻比較	73



第1章 緒論

1.1 研究概觀

1.1.1 發展背景

近年來，人工智慧（Artificial Intelligence，AI）應用在不管是在學術領域或是日常生活，無所不在的出現在你我面前，尤其是機器學習（Machine Learning，ML）領域受到高度重視且被運用在許多領域中，其中又以類神經網路（Neural Network，NN）被成功的廣泛用在影像辨識、物件辨識或語言翻譯等應用中。其實早在 1940 年代 NN 早就被提出了，但直到了 1989 年，LeNet-5 演算法才成功的應用在辨認手寫數字上[1]，讓 NN 有更進一步的發展。發展緩慢的最主要原因有：中央處理器（Central Processing Unit，CPU）的運算速度與能力的不足、記憶體儲存容量小以及資料庫的短缺等等，讓空有模型理論的演算法難以驗證與應用。然而，隨著半導體科技蓬勃的發展，到了 2012 年，在 ImageNet 圖像辨識競賽中[2]，AlexNet 演算法憑藉著圖型處理器（Graphics Processing Unit，GPU）的高平行計算能力，一舉將錯誤率從以往的 25%的錯誤率，一口氣改善了將近 10%的錯誤率[3]，讓深度類神經網路（Deep NN，DNN）有了轉捩點的發展。到了 2015 年，由 Microsoft 所提出的 ResNet-50 演算法[4]，則首度將錯誤率降至低於人類的 5%。圖 1-1 為近年來 DNN 演算法的演進情況，可以觀察到近三年來，由於邊緣裝置（Edge Devices）的逐漸興起，Google 在 2017 年提出的 MobileNet 演算法[5]，降低整體運算量以實現於 edge devices 中，執行預處理（Pre-process）的工作判斷，減少不必要的資料傳輸，讓 DNN 的發展與應用更佳多樣化。

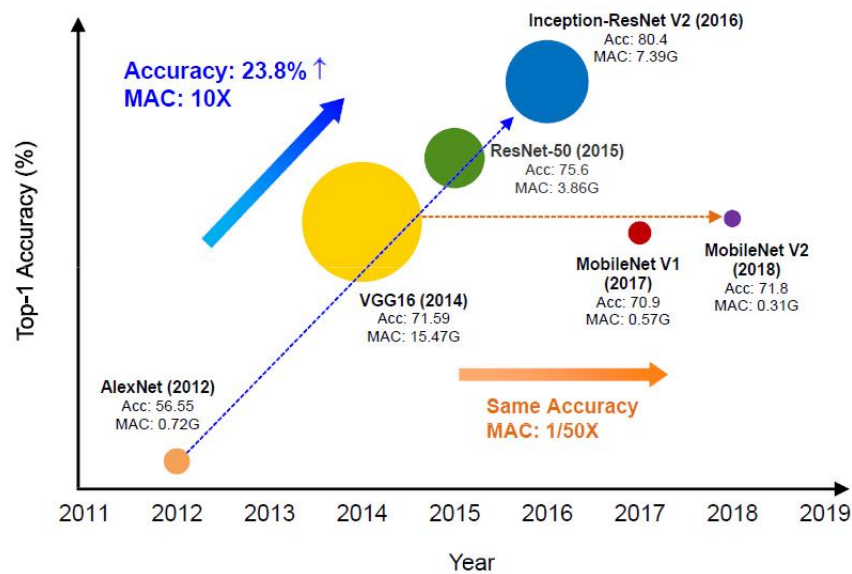


圖 1-1 DNN 演算法之演進[6]

隨著物聯網（Internet of things，IoTs）的高度普及蓬勃發展，傳輸資料日漸的多元與龐大，根據國際數據資訊（International Data Corporation，IDC）的預測，到了 2025 年將會有超過 416 億的裝置與 79.4 兆 bytes 傳輸資料量的產生[7]，屆時將容易造成網路塞車（Network traffic）與資訊安全性等相關問題[8]，因此減少 IoT 設備的資料傳輸，成了近幾年來 IoTs 的發展趨勢之一。為了降低 IoT 設備對於資料傳輸的需求，將 AI 導入 IoT 設備中，利用 IoT 中的感測器收集到的資料，直接進行 ML 的推論（Inference）運算，就能避免非必要的資料進行傳輸[9]，因此在 IoT 設備中，就能直接進行邊緣運算（Edge computing），實現即時性（Real-time）運算，以降低整體的延遲性（Latency）、功率消耗與增加安全性[10]。

為了在有限的資源中，實現 AI 在 IoT 設備上運行，功率消耗就是一個相當重要的指標。GPU 雖然有著高速平行矩陣運算能力，使整體運算的吞吐量（Throughput）有顯著的突破，但與 CPU 一樣有著共同的缺點，那就是大量的功率消耗。對於需要高速運算、演算開發或是訓練（Training）模型來說，似乎不是大問題；但對於需要低功率消耗的 IoT 設備來說，是非常不實際的。為了將 AI 實現於低功率消耗的 IoT 裝置，這幾年發展出了各式各樣的人工智慧處理器（AI

Processing Unit，APU)，即 AI 加速器，因為可編碼性相比於 GPU 是比較低的，因此讓 AI 在加速器上執行推論運算，解決了高耗電量與高延遲的問題。圖 1-2 為 CPU、GPU 與 APU 計算上的差異比較。

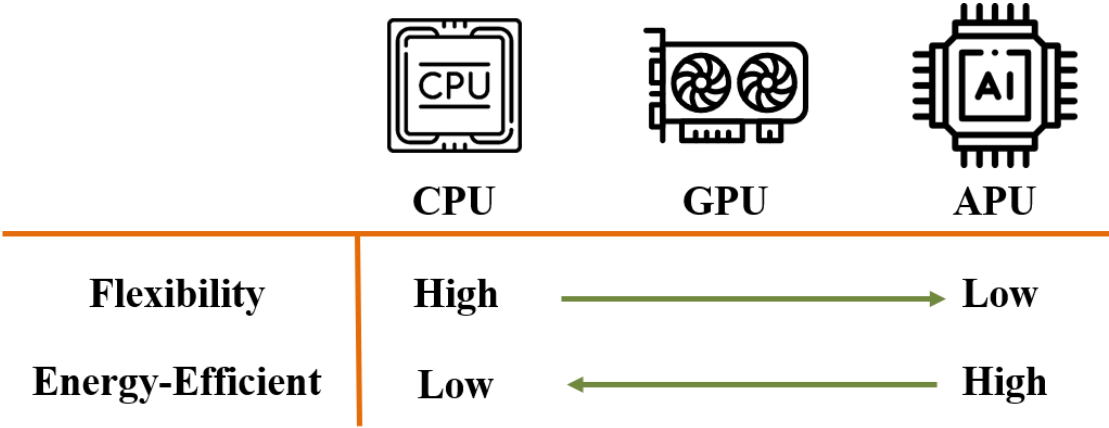


圖 1-2 不同處理器之運算特性差異

在傳統的數位加速器中，雖然有效提升了整體 DNN 運算時的能源效益 (Energy-efficient)[11][12]，但在每次執行乘加(Multiply and Accumulation，MAC)運算中，都需要跟記憶體搬取資料進行運算，資料運算完後，在透過記憶體改寫運算結果進行儲存，因而形成了記憶體牆 (Memory Wall) 的問題，即每次的邏輯運算都無法避免資料在記憶體與計算邏輯閘之間的搬移。圖 1-3 為資料在不同記憶體階層 (Memory hierarchy) 所花費的能量，可以觀察到，若資料需要從緩衝器 (Buffer) 搬移資料的話，能量消耗是暫存器 (Register File，RF) 的六倍；若是從動態存取記憶體 (Dynamic Random-Access Memory，DRAM) 搬移資料的話，能量消耗甚至是 RF 的 200 倍以上，因為資料需要透過匯流排 (Bus) 將資料從其他模組 (Module) 送入計算元中，不僅耗能也耗時。因此若能降低資料在不同記憶體階層之間搬移，則能有效減輕 memory wall 的現象，即能省掉大量的能量消耗與搬移時間，達到低功率且高效能的運算效果。

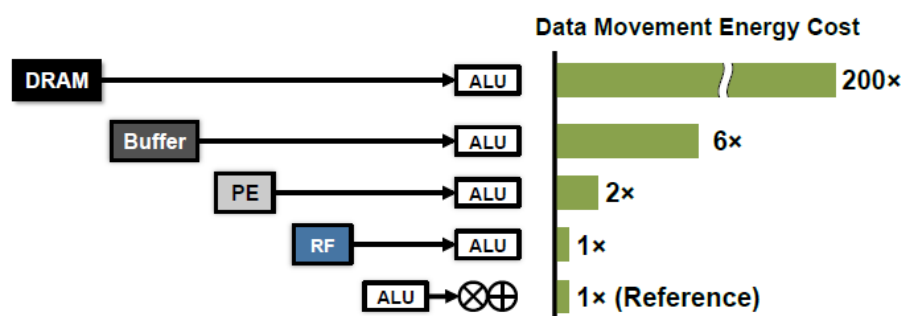


圖 1-3 不同記憶體階層與運算單位，資料所需能量消耗[13]

表 1-1 為這幾年來較熱門 DNN 演算法的權重 (Weight)、捲積運算層 (Convolutional Layer, CONV Layer)、全連結層 (Fully Connected Layer, FC Layer) 與 MAC 的數量。若以 AlexNet 演算法為例，整個演算法需要 7.24 億次的 MAC 需要被運算，在最差的情況下，則會有接近 30 億次的記憶體存取。因此從表 1-1 中可以觀察到當演算法越複雜時，雖然能增加判斷精確度，但 memory wall 的問題將趨於嚴峻，對於需要低功率消耗要求的 IoT 設備來說，也就越不利。為了解決記憶體讀取次數過多的問題，不少利用改變資料流 (Dataflow) 的方式，試圖增加資料在記憶體的利用率，達到重複使用的好處，以避免資料的搬移，如行固定法 (Row stationary) [13]、權重固定法 (Weight stationary) [14]或是輸出固定法 (Output stationary) [15]等等的方式。但改變運算資料流的缺點就是大多數的加速器僅支援固定的運算資料流，因此當資料結構改變時，反而會造成加速器的利用率下降，降低了讓整體的運算效率。

表 1-1 DNN 演算法之權重與 MAC 總數比較

Metrics	LeNet-5[1]	AlexNet[3]	VGG-16[16]	ResNet-50[4]
CONV Layers	2	5	16	49
FC Layers	2	3	3	1
Total Weights	431k	61M	138M	25.5M
Total MACs	2.3M	724M	15.5G	3.9G

1.1.2 記憶體內運算

為了減少資料在記憶體與邏輯運算的搬移，以符合 IoT 設備的輕量化與低功率消耗的需求，記憶體內運算 (Compute-In-Memory, CIM) 則是最有希望突破 memory wall 問題的一種新興 AI 加速器。CIM 電路是利用現有記憶體電路的天生矩陣架構，進行電路修改，使得 CIM 能與 GPU 一樣，讓 MAC 的運算達到高平行運算能力，達到電路的高頻寬 (Bandwidth) 運算。另外，由於輸入特徵資料 (Input Feature Map, IFMP) 與 weight 在計算時，是在記憶體內完成 MACs 的運算，過程中並不會進行資料搬移，因此能夠大幅降低 memory wall 之現象，解決傳統數位加速器中，必須經常將資料從記憶體與計算元中搬移的問題，圖 1-4 為馮紐曼 (Von Neumann) 傳統數位加速器與 CIM 運算流程的比較圖。圖 1-5 為馮紐曼架構下傳統數位加速器與非揮發性記憶體內運算 (Non-Volatile Based CIM, nvCIM) 的能量消耗比較圖，少去了大量資料在記憶體間搬移的 CIM，所消耗的能量是傳統數位加速器的 10 倍到 1000 倍以上的節省。因此 CIM 不僅解決了大量資料的被搬移需求，降低了整體電路的能量消耗，也提高了整體的 throughput，以符合 IoT 裝置對於低功率消耗的要求。

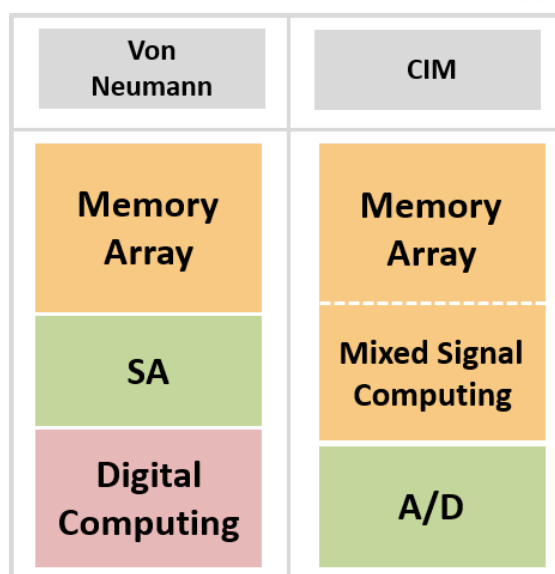


圖 1-4 數位加速器與 CIM 之運算流程比較[17]

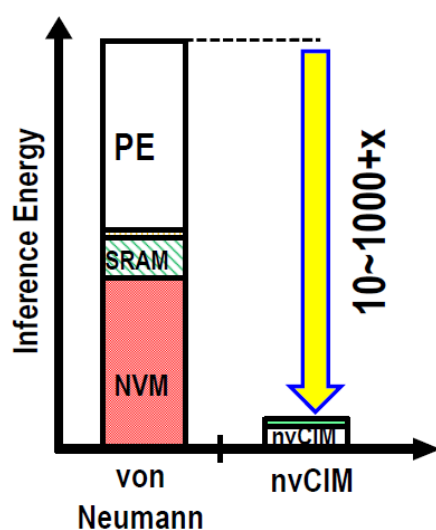


圖 1-5 數位加速器與 CIM 之能量比較[18]

圖 1-6 為 CIM 的電路架構與 NN 網路的對應圖，黑色的 $\bar{X}_l$ 為 IFMP，綠色的 $\bar{W}_l$ 為 weight，橘色的 $\bar{Y}_l$ 為輸出特徵資料（Output Feature Map，OFMP），為了要實現捲積運算如式 1-1，weight 在大部分記憶體單元（Memory Cell，MC）中的擺放規則如下：縱軸為特徵擷取（Filter）的大小，橫軸則為 weight 的位元（Bit）數，每塊記憶體區塊（Bank）的數量則為維度（Channel）的數量。整體 CIM 的運算過程如下：首先多位元的 IFMP 會透過字元線數位類比轉換器（Word-Line Digital-to-Analog Converter，WLDAC），將輸入資料從數位軸（Digital Domain）轉換到類比軸（Analog Domain），如電壓[19][20]、電流或是時間[21][22]，送入字元線（Word-Line，WL）提供 MC 執行乘法運算。再來經過 MC 所計算完的類比乘法值會透過位元線（Bit-Line，BL）進行累加，完成一次多位元 IFMP 與單位元 weight 的 MAC 運算。最後不同權重位元 weight 所計算的 MAC 會透過類比的手段，如電荷共享法（Charge Sharing）[20][22][23]或是電流鏡法（Current Mirror）[24]，將其值給予相對的權重值，合併送入讀取電路（Readout Circuit），把類比值轉換成數位值，完成一次 CIM 整體運算。

$$\vec{Y}_i = \sum_{i=0}^n \vec{X}_i \times \vec{W}_i \quad \text{式 1-1}$$

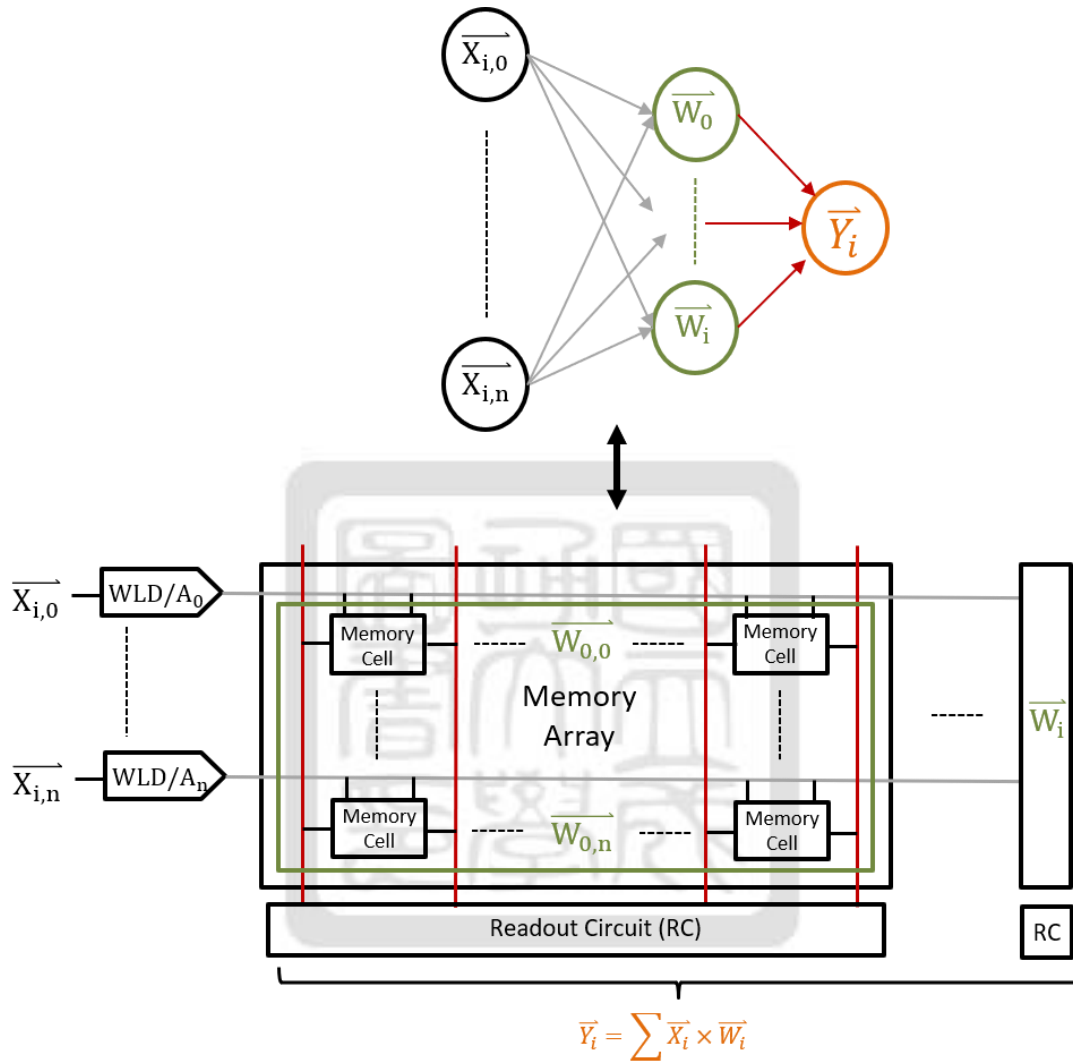


圖 1-6 CIM 與 NN 演算法之比較圖

現階段，記憶體內運算已成功應用於 inference 的系統當中[25][26]，效能的比較上，energy-efficient 更是傳統的數位加速器的三倍以上。除了 inference 之外，記憶體運算也被實現於 training 方面的應用[27][28]或是邏輯運算[29]，雖然效能上無法跟 inference 端比較，但卻也給了記憶體內運算有更多的發展潛力與應用。

1.1.2 相關名詞介紹

由於記憶體內運算在效能評估上，會有屬於自己的規格與名詞，因此會在本小節進行做介紹。此外，因為記憶體內運算的輸入級與輸出級皆為資料轉換器，因此也將一併介紹相關的規格與名詞，以便後續的效能評估。

1.1.2.1 記憶體內運算名詞

精準度 (Precision)：此規格為輸入資料、輸出資料與權重資料的類比值轉為多準確的數位值，單位為位元 (Bits)，越高表示 MAC 的計算值越精準。此規格主要由資料轉換器所決定，因此越高的精準度則會讓整體記憶體內運算的運行速度變慢與功耗變大。

吞吐量 (Throughput)：此規格為記憶體內運算在一次的計算時，能夠產出幾筆 MACs 的運算結果，單位為每秒運算次數 (Operations Per Second, OPS)。因此 filter 的大小、電路運行的時脈 (Clock) 與整個 CIM 能平行運算幾個 filter，都會影響整體電路的 throughput，如式 1-2 所示。式 1-2 中的 2 為乘法與加法運算；#filter 為 filter 的個數，因此越多 MC 則能切割成越多的 filter 與支援越大的 filter 大小，但整體記憶體內運算的面積與功耗則會變大。

$$\text{Throughput} = 2 \times \text{Filter size} \times \text{Clock rate} \times \text{\#filter} \quad \text{式 1-2}$$

能量效益 (Energy-Efficient)：此規格為每瓦的平均功率消耗下，能夠計算出幾筆 MACs 的運算，單位為 OPS/W。越高表示整體的計算效益越佳，即越快的 throughput 或越低的平均功率消耗，如式 1-3 所示。

$$\text{Energy Efficient} = \frac{\text{Throughput}}{\text{Power}} \quad \text{式 1-3}$$

推論時間 (Inference Time)：此代表完成一次 MAC 的計算需要花多少的時

間，單位為秒（Second）。由於大多 CIM 電路應用於推論的判斷上，因此系統的操作時間才以此命名。

1.1.2.2 資料轉換器名詞

非線性度（Non-Linearity）：此規格為資料轉換器的靜態規格（Static Specifications），且排除了位移誤差（Offset Error）及增益誤差（Gain Error）後的數據，主要分為兩種：一種為積分非線性（Integral Nonlinearity，INL），另一種為差分非線性（Differential Nonlinearity，DNL）。

一般而言，一個理想 N 位元的資料轉換器，每個類比區間對應到的數位碼的寬度應該都會相同，寬度則取決於資料轉換器的解析度（Resolution）與量化（Quantization）電壓或時間的範圍，這個寬度則會被定義為一個最低有效位元（Least Significant Bit，LSB）。因此 DNL 則會被定義為實際資料轉換器所轉換的數位碼寬度與理想的數位碼寬的差異，如圖 1-7 所示。以時間數位轉換器（Time-to-Digital Converter，TDC）為例的話，DNL 則會如式 1-4 所示，單位為 LSB。其中 T_n 為數位輸出碼為 n 時，所對應到的輸出時間； T_{n+1} 為數位輸出碼為 $n + 1$ 時，所對應到的輸出時間。因此 DNL 的最小值將發生於無對應輸出碼時的情況，即為負一。

$$DNL = \frac{T_{n+1} - T_n}{LSB} - 1 \quad \text{式 1-4}$$

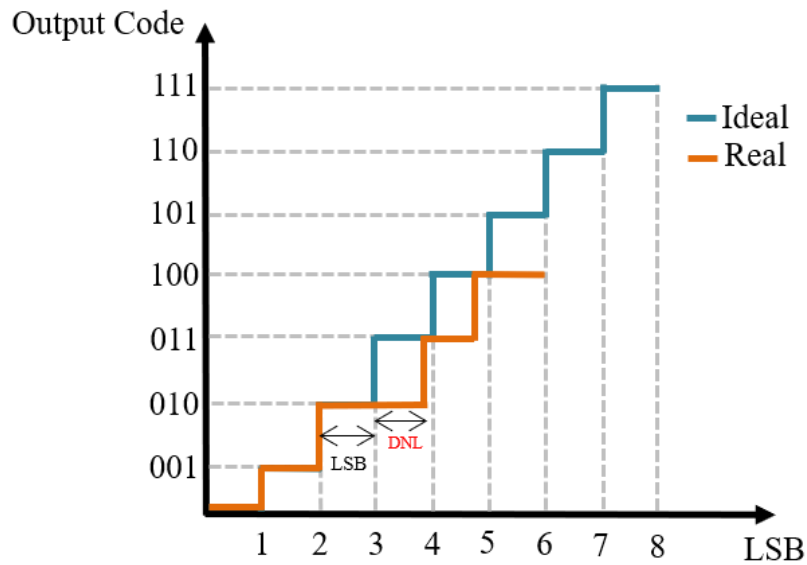


圖 1-7 資料轉換器之 DNL 示意圖

INL 則被定義為與理想參考線的差值，如圖 1-8 所示，理想參考線則是使用最佳近似線 (Best-fit Line) 所決定。此外，INL 通常等於 DNL 的累加，因此可如式 1-5 所示，單位為 LSB。

$$INL(n) = \sum_{k=1}^n DNL(k)$$

式 1-5

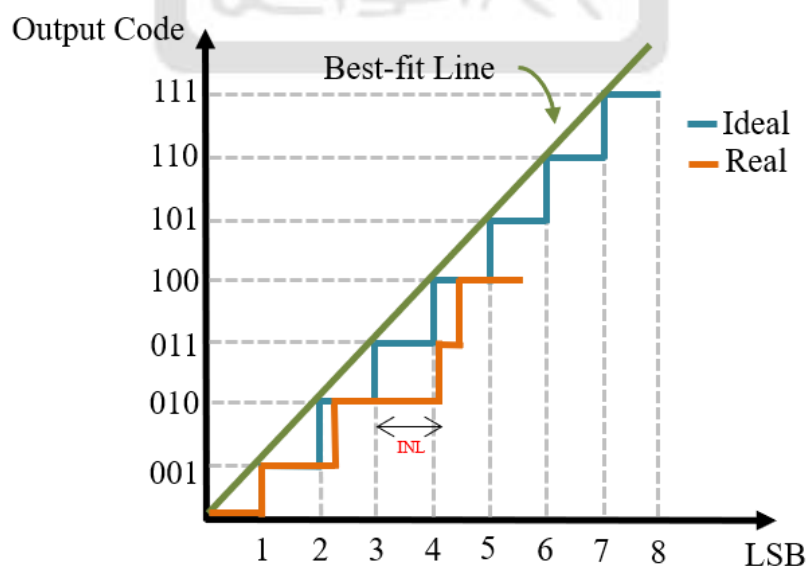


圖 1-8 資料轉換器之 INL 示意圖

信號雜訊失真比 (Signal-to-Noise and Distortion Ratio, SNDR)：此規格為資料轉換器的動態規格 (Dynamic Specifications)，且定義為訊號功率 (P_{signal}) 與雜訊功率 (P_{Noise}) 加諧波功率 ($P_{Distortion}$) 之比，如式 1-6 所示，單位為分貝 (dB)。因此當 SNDR 越大，表示電路對於抗雜訊 (Noise) 與諧波 (Distortion) 的能力較佳，輸出訊號不容意受到影響。

$$SNDR = \frac{P_{signal}}{P_{Noise} + P_{Distortion}} \quad \text{式 1-6}$$

有效位元 (Effective Number of Bits, ENOB)：指資料轉換器真正的輸出準確度，如式 1-7 所示，單位為 bits。因此當 ENOB 越接近原始資料轉換器的解析度，表示電路受 noise、distortion 或 non-linearity 越少。

$$ENOB = \frac{SNDR - 1.76}{6.02} \quad \text{式 1-7}$$

1.1.3 記憶體內運算之分類

記憶體內運算根據不同 MC 的儲存特性，可以大致上分為 nvCIM 與揮發性記憶體內運算 (Volatile Memory Based CIM, vCIM)，如圖 1-9 所示。nvCIM 主要又以新興記憶體 (Emerging Memory) 為大宗，常見的類型則有：相變化記憶體 (Phase Change Memory, PCM) [30]、可變電阻式記憶體 (Resistive Random-Access Memory, RRAM) [18][24] 或磁阻式隨機存取記憶體 (Magnetoresistive Random Access Memory, MRAM) [31][32]。VM based 的 CIM 則以靜態式隨機存取記憶體 (Static Random Access Memory, SRAM) 為主 [19]-[22][33][34]。本篇論文則會著墨於揮發性記憶體內運算上。

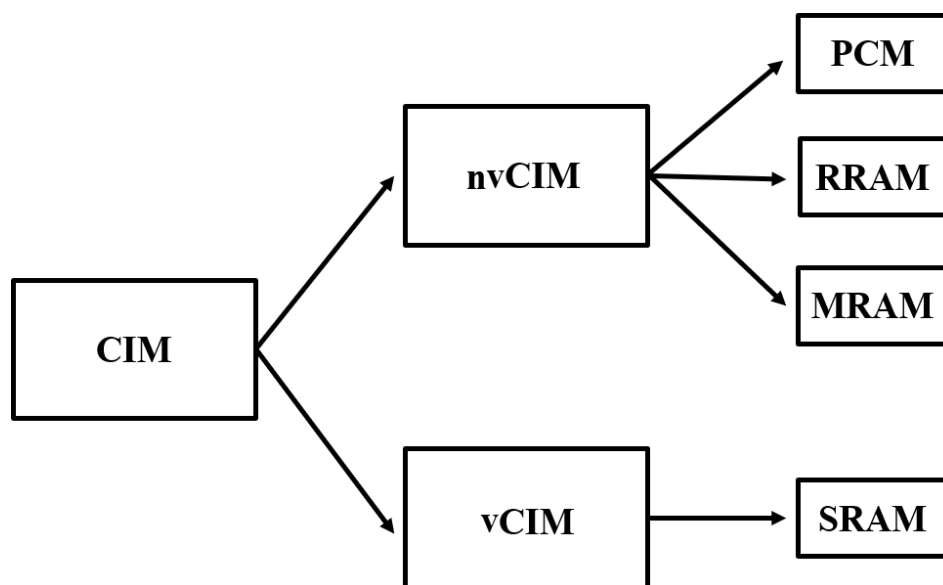


圖 1-9 CIM 概略分類

1.1.3.1 非揮發性記憶體內運算

由於 nvCIM 中儲存元件並不會隨著電源關閉 (Power-off)，導致資料被重置 (Reset)。因此當整體系統電源開啟 (Power-on) 時，就能夠避免資料需要重新從晶片外 (Off-chip) 的 DRAM 或是非揮發性記憶體 (Non-Volatile Memory, NVM)，經過 bus 送入 SRAM 重新讀取。與 nCIM 相比之下，省去了 power-on 後的資料搬移的功率消耗與提高整體系統的 latency。

如 1.1.3 小節所言，大部分的 nvCIM 是由 PCM、RRAM 或是 MRAM 為主要的 MC，雖然快閃記憶體 (Flash Memory) 的製程成熟度遠高於 emerging memory，但操作速度與操作電壓卻是遠大於 emerging memory[35]，如圖 1-10 所示，因此若要實現於需要低功耗的 IoT 設備中，flash memory 並不是一個好選擇。另外，大多的 emerging memory 是能相容於 CMOS 的後堆疊 (Stack) 製程中，因此相容性方面是優於 flash memory。

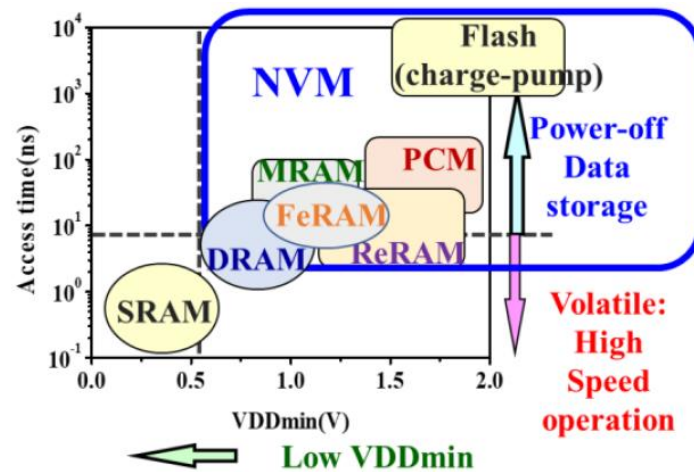


圖 1-10 記憶體單元之操作速度與電壓比較[35]

圖 1-11 為 nvCIM 的架構與 NN 演算法對應圖，黑色的 $\overline{X}_i$ 為 IFMP，綠色的 $\overline{W}_i$ 為 weight，橘色的 $\overline{Y}_i$ 為 OFMP。運算流程如下：首先 IFMP 會透過 WLDAC 將輸入資料從數位值轉換成電壓值，同時 weight 則會被存放在 NVM 中。由於 RRAM 或是 PCM 支援多層準位（Multilevel Cell，MLC）操作，因此只需要一個 MC 就能支援多位元權重運算。再來，不同 weight 的儲存，在 NVM 中則能類比成不同導通程度的電導，因此當電壓通過電導後，就能產生電流，如式 1-8，即類比乘法。最後 MC 所計算完的類比乘法值會透過 BL 進行累加，送入類比數位轉換器（Analog-to-Digital Converter，ADC），完成 MAC 計算。

$$I = G \times V \quad \text{式 1-8}$$

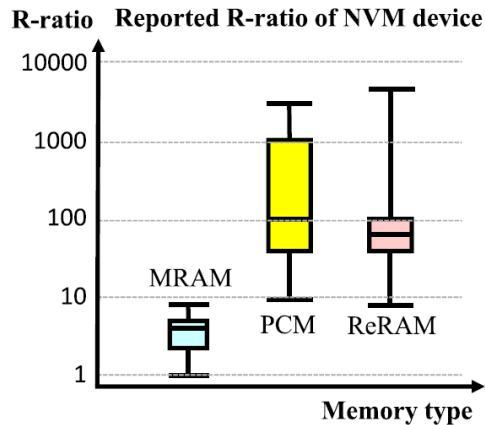


圖 1-12 新興記憶體之阻值比較[36]

1.1.3.2 揮發性記憶體內運算

vCIM 的計算元通常是以 SRAM 為主，其原因為 SRAM 的製程相容性最高，不需要額外的光罩，即能與邏輯製程一起製作。對比於也是揮發性記憶體的 DRAM 來說，除了在製程上並不能與邏輯製程相容以外，本身需要再充電 (Refresh) 機制，以預防電荷的流失，所以整體的操作速度與功率消耗都比 SRAM 還高上許多，因此 vCIM 大多則以 SRAM 為主要計算元。

SRAM 的儲存資料方式是透過栓鎖 (Latch)，將資料鎖定於正回授之中，因此 SRAM 的 Q 與 $\bar{Q}$ 的點電壓為邏輯“1”或邏輯“0”，如圖 1-13 為一般傳統的 6T SRAM 單元所示。在 MAC 的計算上，儲存 weight 的 SRAM 並不會像 NVM 一樣，有著導通阻抗容易受到製程飄移問題，而導致計算解析度的下降，因此一般來說，vCIM 計算的解析度是高於 nvCIM 的。

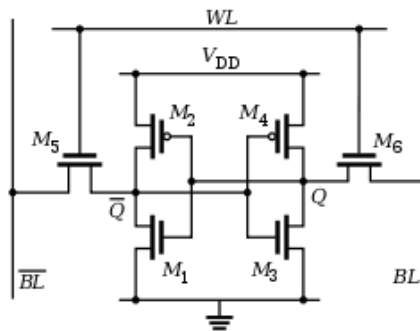


圖 1-13 傳統 6T SRAM 單元電路

vCIM 除了有高的運算解析度以外，SRAM 本身的操作速度與 NVM 相比，也是快上 10 倍左右[35]，因此有利於整體 CIM 的 throughput；另外，SRAM 的操作電壓可以透過動態電壓調節（Dynamic Voltage Scaling, DVS）或是資料保留（Data Retention）的方式，達到節省功率消耗的效果，因此對於需要低功率操作的 IoT 設備來說，是非常有利與合適的選擇。

vCIM 整體的計算流程如下：首先 IFMP 一樣會透過 WLDAC 將輸入資料從數位值轉換成電壓值，在相關文獻中，做法一共有兩種：將轉換完的電壓值灌入 WL，藉由 MOS 轉換成電流，累加至 BL 的電流式（Current-based）[19][20][33]或是將轉換完的電壓值皆輸入到 BL 的電荷式（Charge-based）[23][37]與 MC 進行運算。此外，因為 SRAM 只支援單層準位（Single-Level Cell, SLC）的儲存方式，因此在需要多位元 weight 的 NN 演算法上，面積則會成為挑戰。接下來，不同電的 WL 則會輸出不同的電流，即類比乘法運算。最後，MC 所計算完的類比乘法值會透過 BL 進行累加，送入 ADC 完成 MAC 計算。

然而，傳統 6T SRAM cell 在執行 MAC 運算時，容易發生寫入干擾（Wire Disturb）的現象[33]。其原因是在進行 MAC 時，是由 SRAM cell 內部 latch 的 Q 或是 $\bar{Q}$ 提供電荷給予 BL 進行電流的累加，因此當大量的 6T SRAM cell 同時進行放電時，將會造成 latch 發生翻轉，導致 SRAM 的誤寫。為了避免 write disturb 的發生，近年來的文獻使用了 8T[22]、Twin-8T[20]、10T[23] 的 SRAM cell 或是串列輸入結構（Serial Input Structure）[38]等方法，以避免 latch 本身節點的發生過充放電問題，以防止 write disturb 的發生。

1.2 研究動機

近幾年來，隨著智慧物聯網（AI in IoT, AIoT）逐漸的興起，也就是在 IoT 設備中，能夠自我辨識或是過濾資料的能力。圖 1-14 為聯發科技對於未來 AIoT 發展的市場調查與預測，在未來的 5 到 10 年內，AIoT 的相關產品將會達到 3500

億個裝置，產值將達 16 兆元且佔整個世界 GDP 的 14%以上[6]，因此降低整體 AIoT 設備的功率消耗，延長整體設備的工作時間，成了這幾年來，大多電路的發展趨勢。對於 CIM 相關文獻來說，為了提高 MAC 運算時的精準度，因而提高 IFMP、weight 與 OFMP 的精準度，然而高精準度的資料轉換器與記憶體數量卻也導致了整體 CIM 運行時的功率上升與操作速度的下降[35][36]。因此本研究將探討如何在提高精準度下，達到低功率消耗的效果，以實現記憶體內運算電路到 AIoT 設備中。

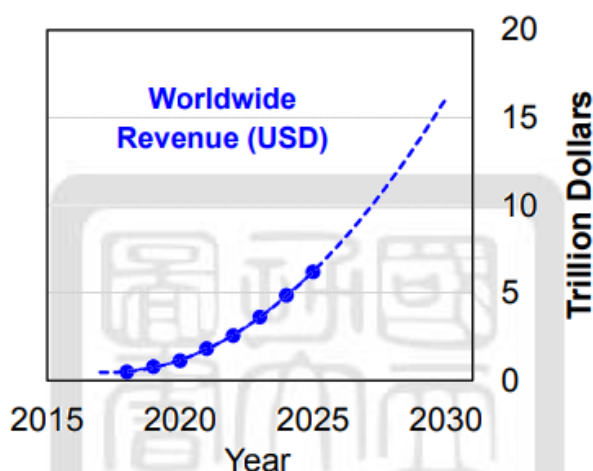


圖 1-14 AIoT 市場預測趨勢[6]

除了功率消耗以外，在資料的傳輸量也是逐年遞增[6][7]，由於傳統的數位加速器架構屬於 Von-Neumann 的系統架構，所以在資料傳輸之間皆有 memory wall 的問題發生，因此對於整體的功率消耗與 throughput 始終有著瓶頸存在。對於記憶體內運算而言，由於電路的操作行為並不需要經常性的將資料執行搬移的動作，因此能夠大量的降低資料搬移時的所消耗能量，降低系統對於 memory wall 問題的發生。此外，省下了資料搬移時的消耗時間，對於系統整體的 throughput 來說，也能夠間接的被提高，因而得到較高的 energy-efficient。因此將 CIM 的運算方式與傳統的數位加速器把 MAC 運算切成數個運算引擎 (Processing Engine, PE) 的平行運算方式做相比，記憶體內運算為一個能有效解決 memory wall 的運

算方式，對於高傳輸量的未來中，記憶體內運算電路則成了一個的最佳選擇。

對於近年來 deep neural network (DNN)演算法來說，隨著各式各樣的應用產生，使得演算法的複雜度趨於複雜化[35]，因此提高 CIM 電路對於 IFMP、weight 與 OFMP 的精準度以符合演算法所需的精確度，也就成為近三年 CIM 非常重要的發展趨勢[17]，如圖 1-15 所示。因此本篇論文將試圖提出具有高運算精度且具高能量效益的記憶體內運算之設計，以應用於更複雜的應用情境當中。

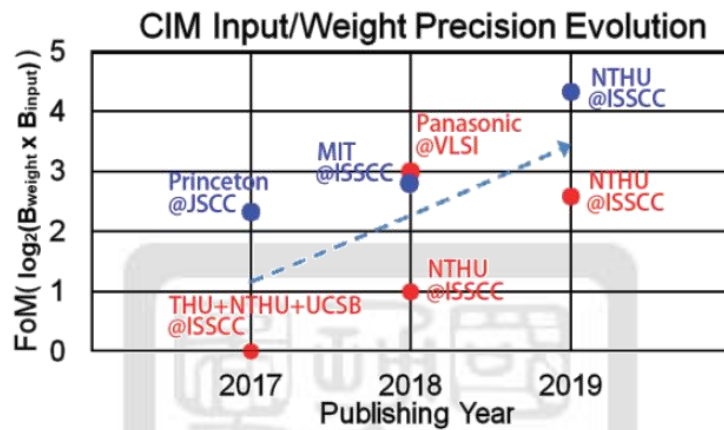


圖 1-15 CIM 精準度發展趨勢[17]

但輸入精準度上升時，對於利用電流式累加的 SRAM-based CIM 而言，因為 MOS 在實際操作上，電流與電壓並非完全成正比，因此在線性的操作範圍是有限的[34]，如圖 1-16 所示。為了要提高整體輸入精準度的操作範圍，在文獻[33]中，則是使用了誤差補償（Offset-Compensation）的方式，以彌補當輸入電壓比較小時，累加電流的非線性現象，如圖 1-17 所示。此方法的壞處在於需要額外的面積與功率消耗當作代價，對於需要低功率消耗需求的 IoT 設備而言是相當不利的，因此本篇論文將試圖提出利用最少的代價來完成較線性的的累加曲線，避免後級的需要額外的電路，增加系統的功率消耗。

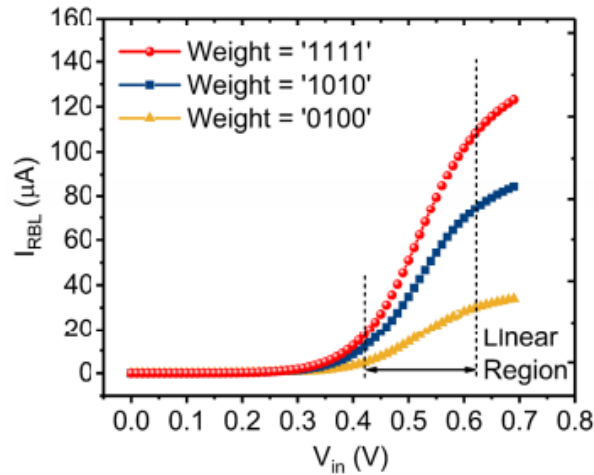


圖 1-16 SRAM-based CIM 電流式累加曲線[34]

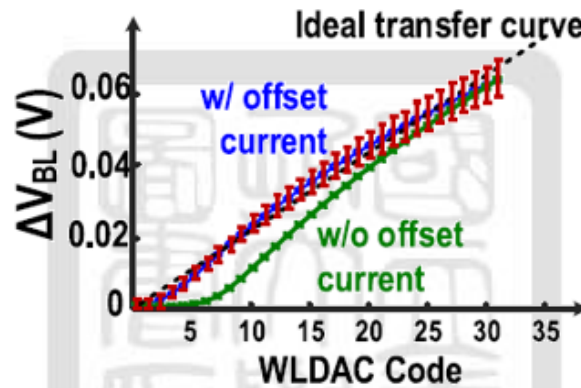


圖 1-17 電流式累加時，有無校正機制比較曲線[33]

1.3 論文貢獻

在本論文中，提出一個能應用於多位元卷積類神經網路 (Convolutional NN, CNN) 的邊緣裝置之高能量效益 8T SRAM 記憶體內運算，並使用權重位元切割累加 (Weight Bit-Partitioned Accumulation, WBPA) 機制，降低在多位元 MAC 運算下的量化誤差，也能避免當演算法需要高精準度輸出時，過高解析度的 ADC 造成功率與操作時間成正向上升之問題。此外，與相同解析度與架構下的資料轉器比較下，不管是在 cell 的使用量與操作頻率都能得到有效的提升與改善。

在輸入端方面，則是使用了時間調變 (Time Modulation) 的方式，提高 MACs 運算時的線性度，以避免後級需要使用校正電路，而導致整體功率消耗的上升。

此方法的另一個優點為避免先前文獻的記憶體內運算電路中，在每一條的 RWL 上，都需要搭配一個 DAC 進行資料的轉換，因此在消耗功率以及面積表現上，皆能夠有效的降低。

最後，隨著製程的微縮 (Scaling)，數位電路的延遲時間 (Delay time) 與偏移時間 (Offset time) 隨著製程的進步而的縮小[39]，因此我們將輸出級的量化電路從原本的電壓軸量化改為時間軸量化，不僅能夠提高電路整體運作時的解析度，對於製程微縮的兼容性也較高。

1.4 論文架構

在本論文中，第 1 章節的主軸為介紹記憶體內運算的發展背景、相關名詞、計算元分類及操作概念、研究動機與貢獻；第 2 章節則會闡述關於近幾年以 SRAM 為計算元之記憶體內運算相關文獻之電路操作與運用的方法所遭遇到的困難及挑戰；第 3 章節則會針對本篇論文所提出之記憶體內運算進行設計概念的描述、實現的方法與內部電路的運行原理；第 4 章節將會實驗本論文所提的記憶體內運算，並對每個電路區塊與整體系統進行分析與模擬；第 5 章節為本論文的結論與未來研究與改善方向。

第2章 相關研究文獻

本章節將會針對揮發性記憶體為主的 SRAM 計算元的記憶體內運算之相關文獻進行探討，在 2.1 節介紹以 10T SRAM 為計算元的記憶體內運算；在 2.2 節中介紹以 Twin-8T SRAM 為計算元的記憶體內運算；在 2.3 節將會介紹以 8T SRAM 為計算元的記憶體內運算。最後在 2.4 節中，將會總結上述三篇相關文獻之優勢、劣勢以及當中能加以改善之方式。

2.1 10T SRAM-based 記憶體內運算

此篇文獻為 10T SRAM 為計算元的記憶體內運算[23]，主要應用於二值化權重捲積類神經網路（Binary-Weight CNNs, BNNs）[40]，如式 2-1 所示。其原因為在 BNNs 演算法之下，能夠有效的減少 weight 的資料數量，與 CNN 演算法相比之下，減少了約 32 倍的記憶體存取的能量消耗，因此使用 BNNs 演算法做為此篇文獻的運算基礎。

$$Y_k = a_k \sum_{i=1}^{R \times R \times C} \varpi_{k,i} \times X_i, \varpi_i \in (+1, -1) \quad \text{式 2-1}$$

為了將演算中的 MACs 實現於電路當中，圖 2-1 為 BNNs 演算法對應於 CIM 電路的計算概念圖，首先利用 DAC 將 IFMP 從數位訊號轉換成類比訊號，提供給 SRAM 做為 MAC 的計算數值，再來利用 SRAM 與前一步驟所轉換的 IFMP 執行 MAC，完成整體的運算，最後再透過 ADC 把計算完的類比訊號轉換回數位訊號，給予後級數位電路執行部分合（Partial Sum）的累加、池化（Pooling）與激活函數（Activation Function）等運算。完成整體 BNNs 演算法運算。

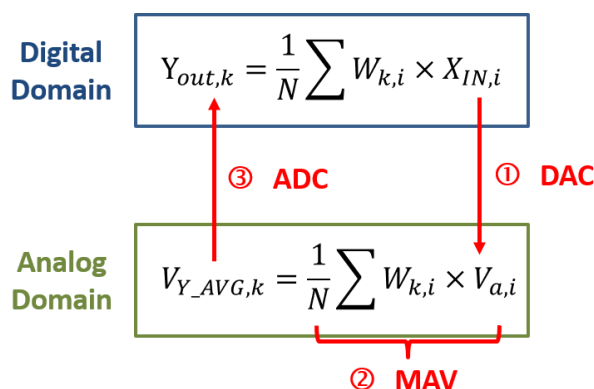


圖 2-1 BNNs 對應於 CIM 電路之計算概念[23]

圖 2-2 為 10T SRAM 為計算元的記憶體內運算之整體電路架構圖，是由 64 個 7 bits 的 DAC、16 個 10T SRAM 陣列與 16 個 7 bits 的 ADC 所組成。因此每一個 10T SRAM 陣列可以等校於演算法中的 channel 及 16 個 ADC 輸出則可以等校為不同 channel 的 partial sum。

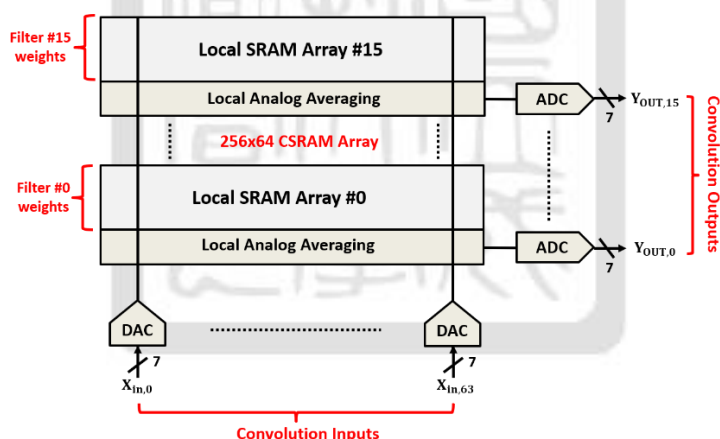


圖 2-2 10T SRAM-based 記憶體內運算[23]

首先當 IFMP 輸入至 CIM 後，DAC 會將不同位元的 IFMP 轉換為不同大小的電荷給予 10T SRAM array 進行運算。圖 2-3 為此篇文獻所使用的 DAC 架構，使用多工器選擇不同時間長短的脈波（Pulse），控制電流源的輸出電荷進行資料轉換，輸入端的訊號則由 IFMP 所供給，輸出端則會連結到全域讀取位元線（Global Read BL，GBL）當中，給予每一塊 10T SRAM array 使用。電路的運行一共有兩個步驟：首先 IFMP 會先經過數位時間轉換器（Digital-to-Time Converter，

DTC)，將不同數位碼轉換成時間長短不同相位 (Phase)，再來則是將不同長短的 pulse 送入電流源，控制輸出的電荷大小，完成一次 DAC 的轉換。在此篇文獻中，為了避免 DTC 中的 6-bit 多功器 (Multiplexer, MUX) 有過大面積消耗，因此將架構換成二階架構解決此問題。在電流源方面則是需要控制 M_{P1} 的操作區域，保持定電流的輸出線性度。

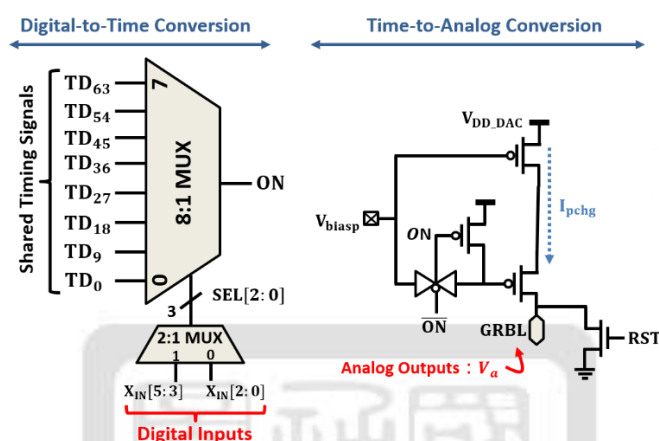


圖 2-3 GBL DAC 電路架構與操作波型[23]

接著，當輸入資料灌入 SRAM array 中的 GBL 後，則為 CIM 整體運算的核心，即 MACs 運算。圖 2-4 為 SRAM array 整體的電路架構圖，主要由 10T SRAM 進行乘法運算以及開關電路進行加法及平均的運算。在電路操作方面上，首先，開關 PCH_G 會被開啟，將 DAC 所換的數值輸入到訊號線 LBLT 與 LBLF 當中。當 SRAM 儲存邏輯“1”時，即 Q 為高電壓、 $\bar{Q}$ 為低電壓，此時連接 LBLT 的路徑會被開啟，將 DAC 所輸入的電荷放電至零，因此當我們將 LBLT 與 LBLF 兩條訊號線進行相減，將會得到負的輸入電壓，則可以類比為輸入電壓乘上 -1 的權重，完成一次乘法運算；反之，若是 SRAM 儲存邏輯“0”時，則可以類比成輸入電壓乘上 1 的權重。再來，開關 EN_p 與 EN_n 會由 IFMP 的最高有效位元 (Most Significant Bit, MSB) 所決定，所以當 MSB 為 1 時，即 IFMP 為負值，此時則會開啟 EN_p ；反之，若 MSB 為 0 時， EN_n 則會被開啟。因此每一個乘法值會依照 IFMP 的 MSB，在 V_{pAVG} 與 V_{nAVG} 訊號線當中，分別進行 charge sharing 的動作，完成各自正負

MACs 運算。最後在運算的過程中，屬於雙端的操作，因此在 noise 的抵抗能力是較其他單端架構還要佳的。

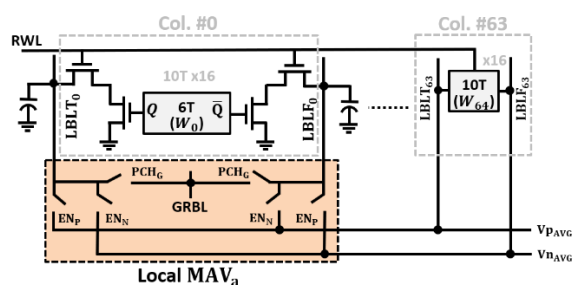


圖 2-4 10T SRAM array 電路架構與操作波型[23]

最後，計算完的正負 MACs 由 $V_{p_{AVG}}$ 與 $V_{n_{AVG}}$ 傳入 local ADC，完成單一 channel 的 partial sum。圖 2-5 為此文獻所使用之 ADC，為 charge sharing 型的 ADC，其架構好處為有較高的 energy-efficient 與面積的節省。其操作為：首先會先利用感測放大器（Sense Amplifier, SA）判斷 $V_{p_{AVG}}$ 與 $V_{n_{AVG}}$ 的大小，決定輸出 MSB 為多少，當 $V_{p_{AVG}} > V_{n_{AVG}}$ ， Y_{OUT} 則為正值，因此 MSB 為 0；若 $V_{p_{AVG}} < V_{n_{AVG}}$ ，MSB 則為 1。接著假設 MSB 為 0，此時較低電壓方（ $V_{n_{AVG}}$ ）與參考電壓將進行 charge sharing 的動作，再透過 SA 進行比較。若 $V_{n_{AVG}}$ 依舊小於 $V_{p_{AVG}}$ 的話，則會重複執行 charge sharing 與比較的動作，直到 $V_{n_{AVG}}$ 高於 $V_{p_{AVG}}$ ，完成量化的過程。最後邏輯電路內的計數器將會記錄執行了幾次 charge sharing，即為最後的輸出數值，完成整體 MACs 運算。

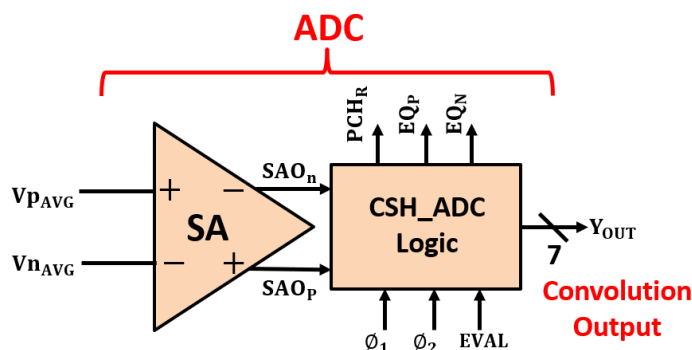


圖 2-5 Charge Sharing ADC 電路架構與波型[23]

2.2 Twin-8T SRAM-based 記憶體內運算

此篇文獻為 Twin-8T (T8T) SRAM 為計算元的記憶體內運算[20]，為第一篇應用於多重位元 CNN 演算法的記憶體內運算，以增加更複雜應用的判斷精確度，如式 1-1 所示。圖 2-6 為整體電路架構，其主要包含了：執行多位元 MACs 運算的 bank、控制 IFMP 運算量的奇偶雙通道 (Even-Odd Dual Channel, EODC) 機制與判別正負計算結果的二補數權重映射 (Two's Complement Weight Mapping, C2WM) 機制。

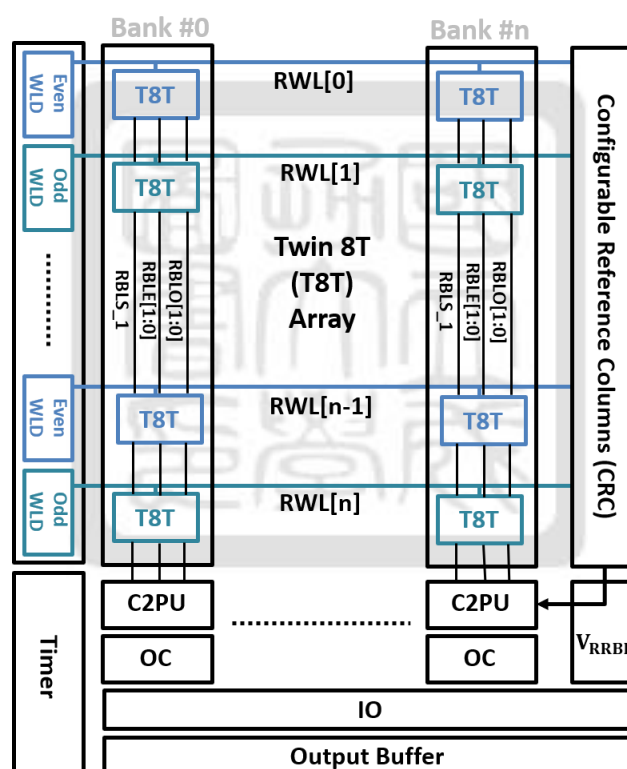


圖 2-6 T8T SRAM-based CIM 整體架構[20]

首先將介紹此篇文獻是如何實現多重位元 MACs 運算，圖 2-7 為單一 T8T SRAM 計算元，為了實現多位元權重的運算效果，使用了兩個 8T SRAM，將 MSB 讀取路徑電晶體 ML_1 與 ML_2 的大小，設計成 LSB 讀取路徑電晶體 MM_1 與 MM_2 的兩倍，即 MSB 讀取路徑的電流將是 LSB 的兩倍，以實現 2 bits 權重分別的概念。

因此在電路操作上，首先 IFMP 會透過 DAC 將數位值轉換為類比值，灌入讀取字元線（Read WL，RWL）中，此時讀取路徑的讀取電流則會隨著 RWL 的電壓大小，轉換成不同大小的輸出電流。因此當 SRAM 儲存邏輯“1”時，我們則可以類比成輸出電流乘上權重 1，完成乘法運算；反之，當 SRAM 儲存邏輯“0”時，則可以類比輸出電流乘上權重 0，表 2-1 為 2 bits 乘上 2 bits 時，所得到電流大小的整體增值表。最後，每一個乘法結果則會透過讀取位元線（Read BL，RBL）進行累加，完成 2 bits 乘 2 bits 的 MACs 運算。

表 2-1 2b×2b 累加增值表[20]

Input (2b)	RWL Voltage	Weight		RBL Current	Multiply
		M8T	L8T		
11	WLL3	1	1	$3 \times I_{MC}$	9
10	WLL2	1	1	$2 \times I_{MC}$	6
01	WLL1	1	1	$1 \times I_{MC}$	3
11	WLL3	1	0	$2 \times I_{MC}$	6
10	WLL2	1	0	$\frac{4}{3} \times I_{MC}$	4
01	WLL1	1	0	$\frac{2}{3} \times I_{MC}$	2
11	WLL3	0	1	I_{MC}	3
10	WLL2	0	1	$\frac{2}{3} \times I_{MC}$	2
01	WLL1	0	1	$\frac{1}{3} \times I_{MC}$	1
00	VSS	1/0	1/0	0	0

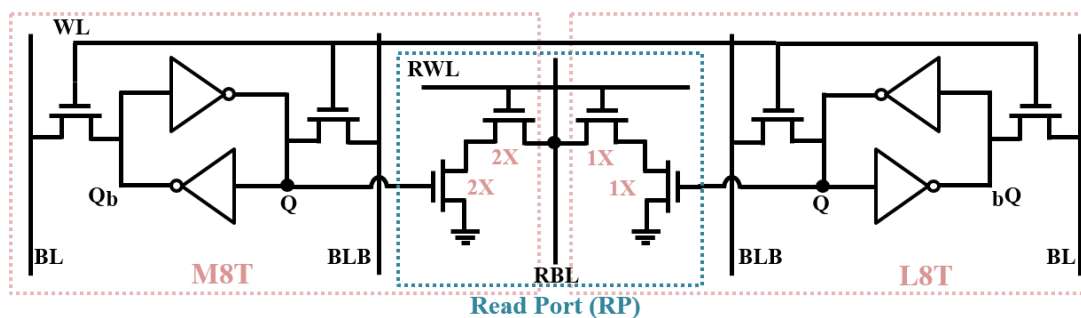


圖 2-7 Twin-8T SRAM cell [20]

由於實際應用情境下，對於硬體的精準度以及速度都有不同的需求，此篇文獻在輸入提供了 EODC 的機制以增加 CIM 對於應用的可用性。表 2-2 為 EODC 的機制下，輸入資料的擺放情況，一共有兩種模式：單通道模式 (Single Channel Mode, SC Mode) 及雙通道累加模式 (2-Channel-Accumulation Mode, 2CA Mode)。當輸入為 SC mode 時，只會開啟基數列或偶數列的 RWL，此時對於 RBL 訊號而言，操作訊號範圍較大，因此能有較高的計算精準度；當輸入為 2CA mode 時，全部的 RWL 將會一口氣的開啟做使用，此時輸入頻寬將是 SC mode 的兩倍，但 RBL 的操作訊號範圍將會變小，因此 2CA mode 適合需要較高 throughput 情況時使用。

表 2-2 EODC 機制下的輸入映射對照[20]

Operating Modes	Even channel	Odd channel
SC Mode	INE[3:0]	Bypass
	Bypass	INO[3:0]
2CA Mode	INE[3:0]	INO[3:0]

接著為了將不同 RBL 所計算出 2-bit IFMP 乘上 2-bit weight 的 MAC 合併成 4-bit IFMP 乘上 5-bit weight 的 MAC 送入 ADC 進行取樣，因此利 C2WM 機制來

完成不同位元權重下的合併。圖 2-8 為 C2WM 機制下的整體電路圖，由逐漸趨近式（Successive-Approximation, SAR）ADC 為基底下設計而成，因此與其他 ADC 架構比較下，energy-efficient 是較佳的。另外，RBLs 訊號線為有號（Signed）數運算，因此在開關的切法上，會與其他相反，達到減法的效果。在操作上，當每一條 RBL 完成 MAC 運算後，SC/2CA logic 區塊內的開關會依據不同的模式下做開啟，當為基數列 RWL 開啟下， S_{ODD} 將會開啟；偶數列 RWL 開啟下， S_{EVEN} 將會開啟；2CA 模式下， S_{SEO} 、 S_{ODD} 與 S_{EVEN} 皆會被開啟。當開關被開啟後，取樣電容將會去取樣 RBL 所計算的 MAC 運算結果，不同位元權重的 MAC 會透過取樣電容的大小，給予適當的權重還原大小。式 2-2 為 SC mode 下且經過 C2WM 機制後，所運算出 MAC 的結果；當為 2CA mode 下，因為 SC/2CA logic 的開關皆會被開啟，因此 RBLE 與 RBLO 所運算的訊號會進行 charge sharing 的動作，將兩個 MAC 值做平均，因此在 RBLs 取樣電容的大小，將會是 SC mode 下的一半，以避免位元權重錯誤，取樣電容的大則會由 SC/2CA MUX 所決定。

$$MAC = (-16) \times MAC_{RBLs} + (+4) \times MAC_{RBLE,O[1]} + MAC_{RBLE,O[0]} \quad \text{式 2-2}$$

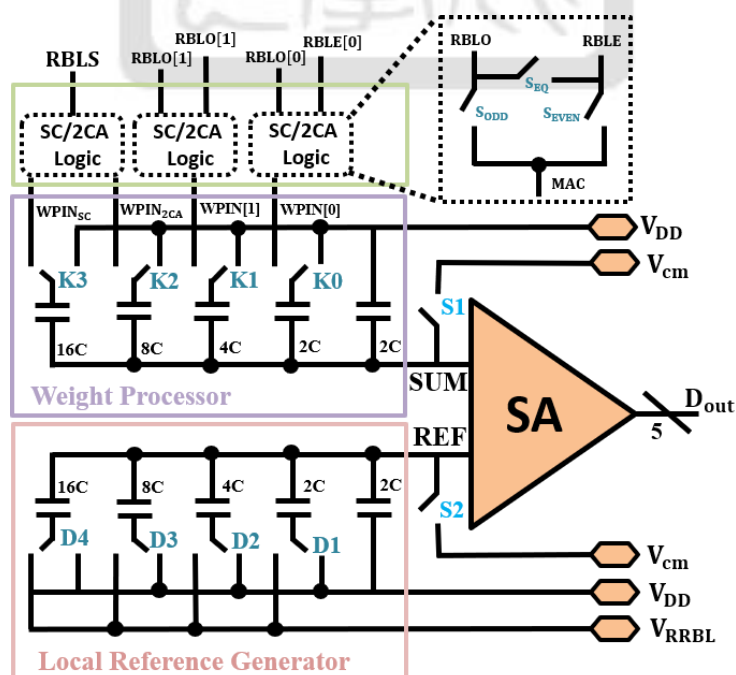


圖 2-8 C2WM 機制電路[20]

為了避免 ADC 在執行量化運算時，參考電壓受到製程的漂移而有誤差，因此在此篇文獻中利用全域參考電壓產生器（Global Reference Voltage Generator，GRVG）所產生的參考電壓，送入 SAR ADC 中進行量化。圖 2-9 為 GRVG 整體電路，由一條 RBL 與電壓緩衝器（Voltage Buffer）所組成，其中 RBL 中的 SRAM 所輸出的電壓大小，即為全域的參考電壓，因為此 RBL 的製程飄移將會與其他在執行 MAC 運算的 RBL 相似，因此將能有效降低製程影響。最後，參考電壓將會透過 voltage buffer，將值送入本地參考電壓產生器（Local Reference Generator，LRVG）中，進行 SAR 的電壓切換以量化整體 MAC 計算值。由於此 MAC 運算結果為[3:2] IFMP 乘上[4:0] weight，因此會先將此筆運算存至暫存器與位移器當中，在與下一個週期所運算出[1:0] IFMP 乘上[4:0] weight 的 MAC 進行加法運算，完成體 4-bit IFMP 乘上 5-bit weight 的 MAC 運算。

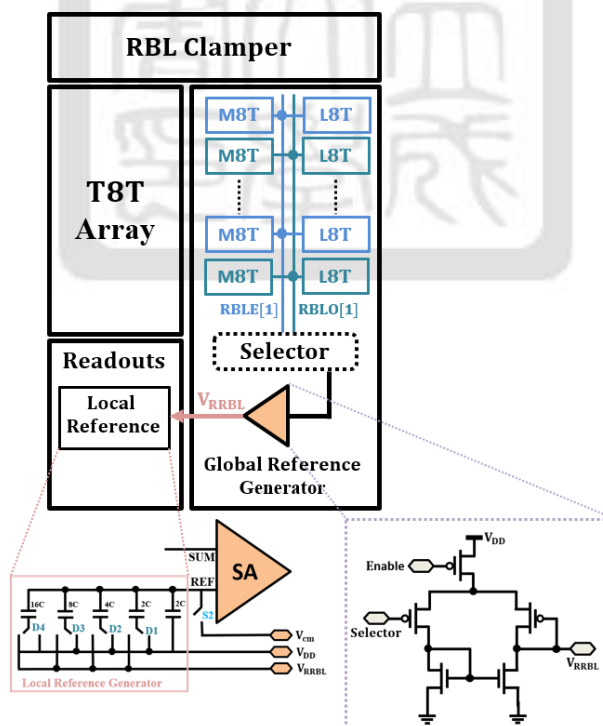


圖 2-9 全域參考電壓產生器電路與波型[20]

2.3 8T SRAM-based 記憶體內運算

此篇文獻為使用 8T SRAM 為計算元的記憶體內運算[22]，由於此 8T SRAM 使用記憶體產生器（Memory Compiler）所產生的，因此在面積的消耗方面，將比[20][23]還來的節省。圖 2-10 為整體電路架構，其中包含了：用來計算 MAC 運算的 8T SRAM array、將 IFMP 從數位值轉換成不同數量 pulse 的列計數器（Row Counter）、合併不同權重執行 Charge Sharing 的電容陣列與最後將計算完 MAC 值轉換成數位值的 ADC。

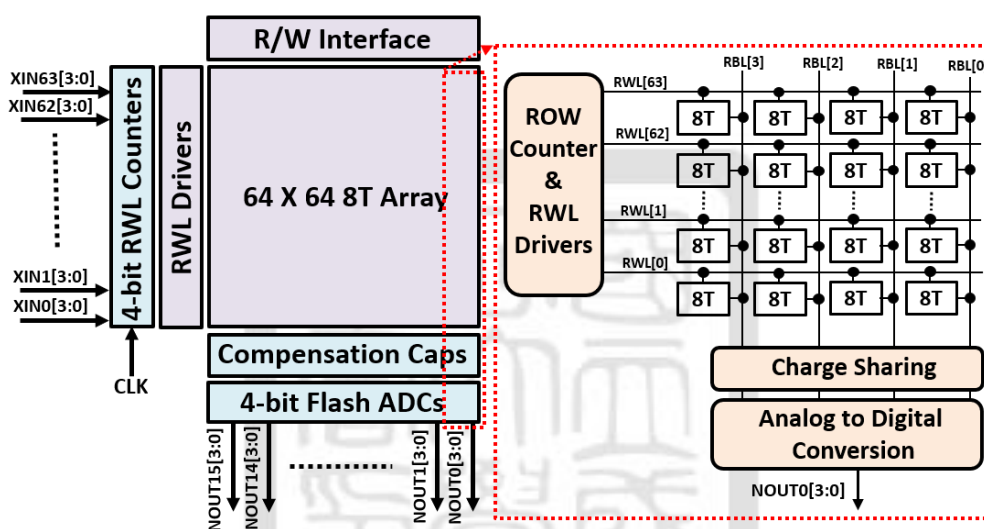


圖 2-10 8T SRAM-based CIM 整體架構[22]

首先說明在 8T SRAM cell 下，如何執行 MAC 運算，圖 2-11 為標準 8T SRAM cell，當每次 RWL 訊號的電壓皆相同時，讀取路徑的電流將也會相同，所以只需要改變 RWL 訊號開關時間的長短，即在讀取路徑中得到不同輸出電荷。因此當 SRAM 儲存邏輯“1”時，讀取路徑將會被開啟，此時我們就能夠類比成是輸出電荷乘上權重 1，完成一次乘法運算；反之，當 SRAM 儲存邏輯“0”時，為輸出電荷乘上 0 的權重，因此輸出為 0。最後我們只需要將每一個乘法結果累加至 RBL 中，即能完成 MAC 運算，如式 2-3 所示。

$$\Delta V_{RBL} = \frac{1}{C_{RBL}} \sum_{n=0}^j T_{RWLn} \times W_n \quad \text{式 2-3}$$

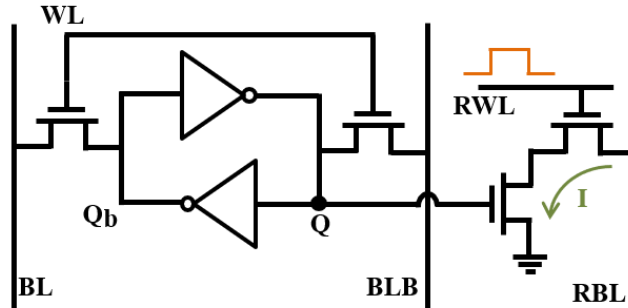


圖 2-11 標準 8T SRAM 架構圖

在本篇文獻中，為了給予 RWL 訊號不同長短的開關時間進行乘法運算，因此使用計數器（Counter）的方式將數位值轉換成類比值。圖 2-12 為 counter 的轉換概念，當輸入為 1 時，counter 將輸出一個單位脈波；當輸入 15 時，counter 將輸出 15 個單位脈波，此方法與直接將脈波進行調變相比，輸出的電荷較能保持線性度，因此有利於 MAC 計算上的精確度。

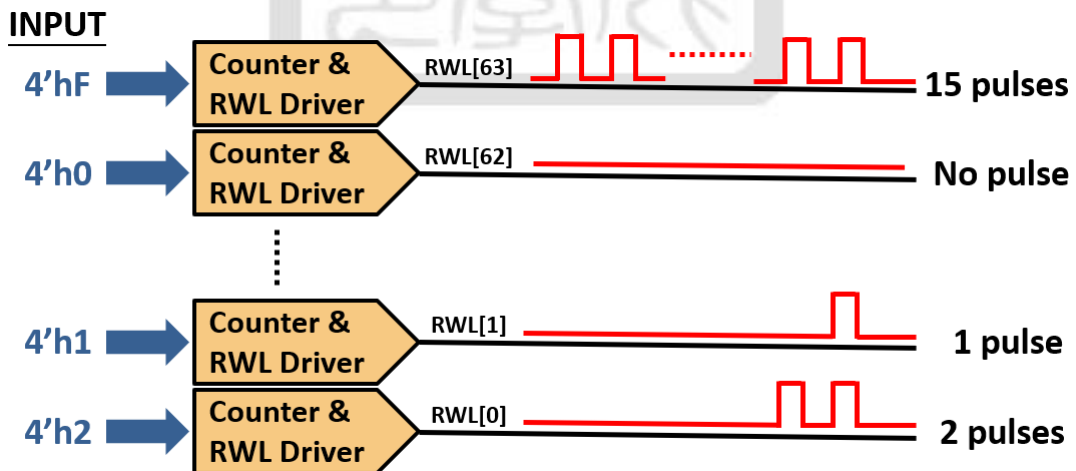


圖 2-12 以計數器轉換輸入之概念[22]

因此在整體的單一 filter 電路架構，如圖 2-13 所示，主要由 8T SRAM cell、開關電路與 ADC 所組成。電路操作如下：一開始會先將 RBL 進行預充電（Pre-

charging) 的動作，讓每一條 RBL 上的取樣電容都能夠充電到高電位；第二步為對 RBL 進行取樣之動作，即執行 MAC 運算。不同的 pulse 與 weight 將會造成 RBL 上有著不同的放電電荷，因此就能藉由放電的大小，量化單一 RBL 上的 MAC 輸出多寡，此時因為 RBL 之取樣電容皆相同，所以在計算上的位元權重也會相同；第三步則是進行 charge sharing 的動作，此時陣列中的 S_0 開關將會被關閉， S_1 將會被開啟，每個權重位元將依照 RBL 的取樣電容大小，給予適當位元權重分配，將每條 RBL 上所計算的 MAC 值累加乘一數值；最後一步則是將運算完的多位元 MAC 值送入 ADC 進量化，完成一次整體運算。

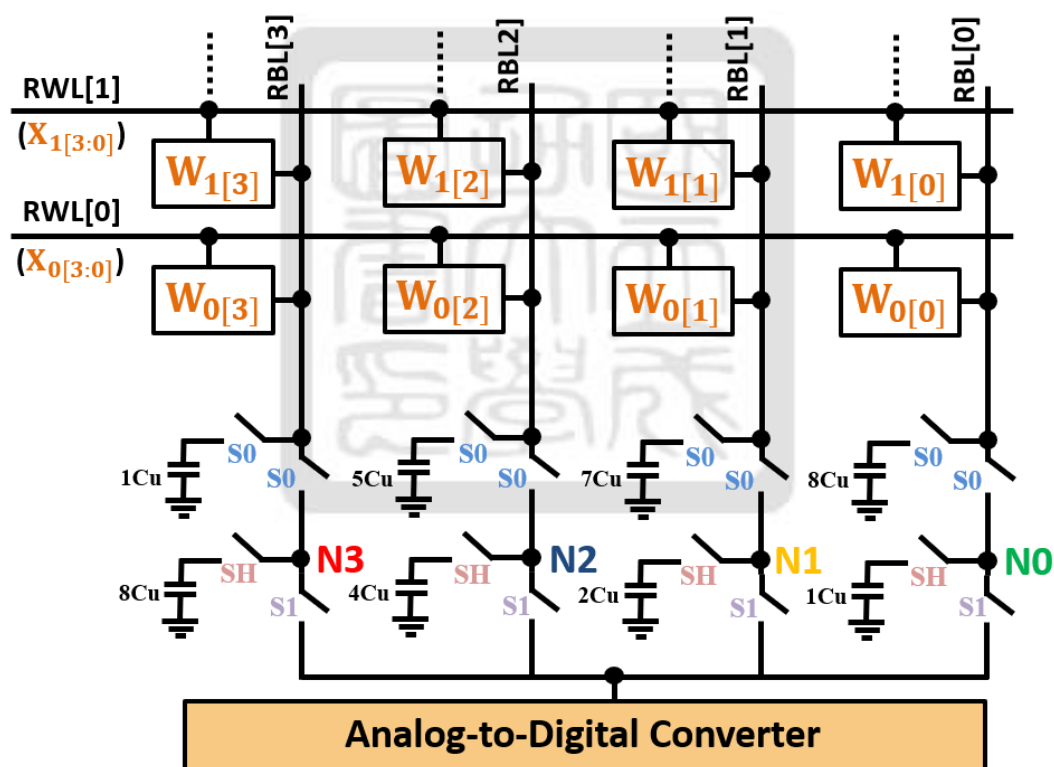


圖 2-13 8T SRAM 陣列圖[22]

2.4 相關文獻總結

表 2-3 為上述三篇相關文獻的整理，分別為使用不同計算元與計算方式的記憶體內運算，以下將會探討三篇相關文獻的優點與缺點。

在文獻[23]中，是利用 10T SRAM 來做為記憶體運算的計算元，其單一計算元為一雙端架構，雖然面積比標準 6T SRAM 還要大，但卻擁有較高抗雜訊的能力，因此有助於計算時的精確度。此外，在 MAC 的運算過程，屬於單純的電荷充放電，因此不需要過多的轉換，也增加了電路整體的穩定性。然而，在 ADC 方面，為了消除電路造成的 offset，系統需要兩個工作週期，才能將所需輸出轉換成數位碼，因此降低了整體的 throughput；在操作頻率上，為了讓輸入的脈波有較佳的穩定性，雖然有助於功率消耗的下降，但是也降低了整體的 throughput；在演算法上，雖然 BNN 能有效降低資料的搬移數量，但對於較複雜的應用環境下，略有挑戰。

在文獻[20]當中，為利用 T8T SRAM 來做為記憶體運算的計算元，此架構因為可適用於多重位元 CNN 的運算，且在輸入端增加了 EODC 機制，因此同時適用於需要較高精準度或是較高速的應用中使用。然而，在單一的計算元上，需要使用到共 16T 的記憶體，在面積上將造成負擔，不利於高密度設計。此外，由於 IFMP 是利用電晶體將輸入電壓轉換成輸出電流進行累加，由於電晶體本身線性操作範圍有限[34]，因此在單一工作週期下，只支援 2-bit 的輸入精準度，在 4-bit 情況下，需要花兩個工作週期才得以完成，降低了整體系統的 throughput。

最後，文獻[22]當中，使用了標準 8T SRAM 來做為記憶體運算的計算元，因此在面積消耗上只比標準 6T SRAM 多約 30%左右，此外在 MAC 計算上，是利用時間的大小，去取得電荷進行累加，因此有助於整體累加時的線性度。在量化電路上，與前兩篇相關文獻相比，並不是使用具有較高能量效益的 ADC 架構做使用，而是使用較高速架構的快閃式 (Flash) ADC，因此在 throughput 的表現上是較佳的。然而，在輸出的精準度上只支援 4-bit，因此對於較複雜的應用環境下，將是一大挑戰。

表 2-3 相關文獻之比較

Publisher	ISSCC'18[23]	ISSCC'19[20]	ISSCC'20[22]
Technology (nm)	65	55	7
SRAM Type	10T	T8T (16T)	8T
Computation	Binary—Weight Multiple—IFMP	Multiple—Weight Multiple—IFMP	Multiple—Weight Multiple—IFMP
Input Conversion	Charge	Voltage	Counter
Accumulation Type	Charge	Current	Charge
Input/Output/Weight Precision (bit)	6/6/1	4/7/5	4/4/4



第3章 具權重位元切割累加機制之 8T SRAM 記憶體內運算

本章節將會針對第 2 章中相關文獻的相關缺失，並提出相對應的改善方法。在 3.1 節中，將會講述在本記憶體內運算設計架構下，多位元 MAC 運算的概念；3.2 中則是在設計概念下，所實現的整體記憶體內運算電路架構；3.3 到 3.6 節則會介紹記憶體內運算所使用到的電路區塊；3.7 節則會介紹本記憶體內運算所使用的權重位元切割累加計算機制與實現。

3.1 設計概述

在先前的相關文獻當中，大多記憶體內運算的輸入端皆是將 IFMP 利用 DAC 改變 WL 的電壓大小，將其電壓利用 MOS 的導通強弱轉換成對應電流，在 BL 上進行累加[19][20][33][34]，即電流式的 CIM。但對於 MOS 元件而言，電壓與電流並非完全正比，導致了線性操作範圍有限[34]，輸入精準度難以有效提高，因此在後級則需要增加校正電路[33]或額外的代價，而提高系統的功率消耗。此外，這類型的轉換方式在每一條的 WL 皆需要搭配一個 DAC，將 IFMP 獨自進行轉換，因此在面積與功率消耗上皆成了一個大隱憂。在記憶體內運算的輸出端精準度方面，由於 ADC 的解析度將會直接影響到整體系統的運算速度與功率消耗，因此在過去的文獻中，為了實現高的 energy-efficient 與運行速度，無法將輸出精準度做有效的提高與改善，因此成了其中的瓶頸。

在本論文所提出的記憶體內運算當中，輸入端則將利用開關時間的長短去控制單一運算元的輸出電荷，即時間調變（Time Modulation）機制，接著將每一個運算元的輸出電荷將累加至 RBL 取樣電容上，完成一次的單位元 weight 乘上多位元 IFMP 的 MAC 運算，如式 3-1 所示。式 3-1 中的 $n \times n$ 為單一 filter 的大小；

I_{MC} 為單一運算元所產生的電流。由於每個運算元結構皆相同，因此每個電流大小會相同，且當訊號線上的負載電容相同時，位元權重也皆相同，即為 binary-weight 運算。最後，為了實現多位元 weight 乘上多位元 IFMP 的 MAC 運算，因此輸出端則會採用數位電路的方式，結合每一個由次 ADC (sub-ADC) 所量化完 MAC 運算的結果，藉此達到高輸出精準度，以避免單一高精準度 ADC 帶來的高耗能與低速的代價，即權重位元切割累加 (Weight Bit-Partitioned Accumulation, WBPA) 機制。

$$Q_j = \sum_{i=0}^{n \times n} I_{MC} \times T_{RWL_i}, j \in [MSB, LSB] \quad \text{式 3-1}$$

3.2 設計架構

圖 3-1 為本論文所提出的記憶體內運算的整體架構，其中包含了同步 IFMP 的 36-bit 輸入暫存陣列 (Input Register Array, IRA); 計算多位元 MAC 運算的權重計算區塊 (Weight Block)，其中由 54-bit 8T SRAM 運算元與開關電路組合而成; 實現時間調變機制的 4-bit 脈波寬度產生器 (Pulse-Width Generator, PWG) 與多工器陣列 (Multiplexer Array, Mux Array); 將 MAC 取樣電壓轉換為時間軸的六組電壓時間轉換器 (Voltage-to-Time Converter, VTC); 完成時間軸量化運算的六組 4-bit 時間數位轉換器 (Time-to-Digital Converter, TDC); 實現位元切割累加機制的輸出結合器 (Output Combiner, OC)。本架構將會被應用於 3×3 的 filter 中，其 IFMP 為 4-bit、weight 為 6-bit 與 OFMP 為 8-bit; 控制訊號方面則有執行 8T SRAM 寫入動作的 WEB 與 4-bit 的 ADDR; 進行 MAC 運算的 Compute 訊號; 控制架構時序的 CLK 訊號。

在電路操作上，一共有兩種模式：寫入模式與計算模式。當在寫入模式時，WEB 訊號為邏輯“1”，將 weight 資料寫入至 weight block 的 8T SRAM 當中，此時 WL 只有一條會被開啟，執行 8T SRAM 改寫之動作。當在計算模式時，Compute

訊號為邏輯“1”，將 IFMP 資料載入至 IRA 且輸出到 mux array 中，此時 RWL 將會被全部開啟做使用，執行 8T SRAM 計算之操作，以下小節將會詳細闡述計算模式下的電路操作情形。

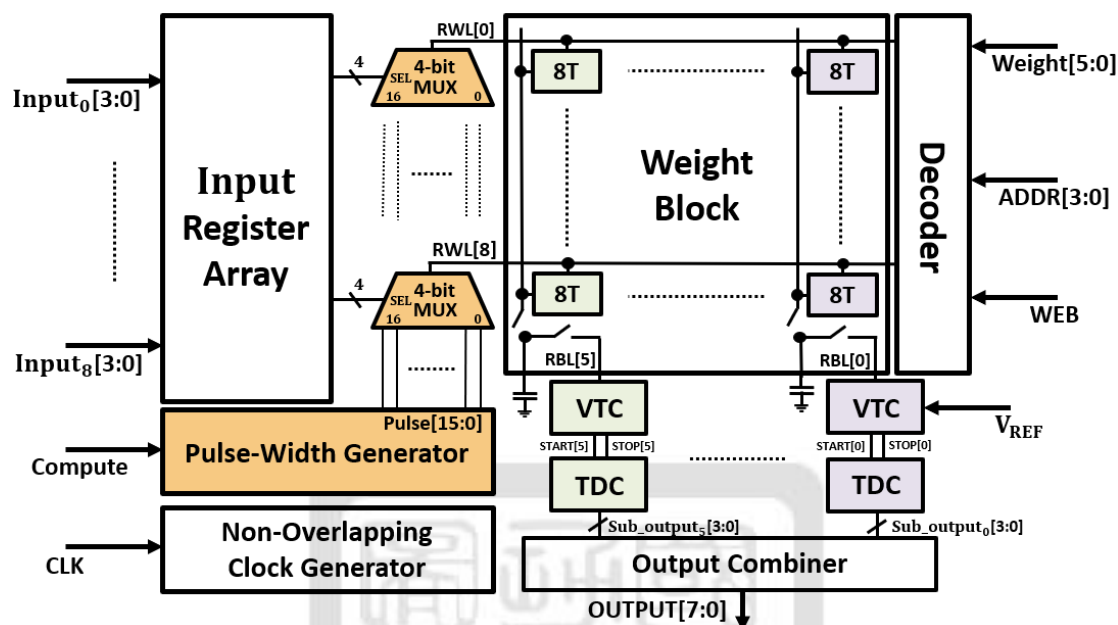


圖 3-1 記憶體內運算架構圖

3.3 權重計算區塊

為了實現 3.1 節中所敘述的單位元 weight 乘上多位元 IFMP 的 MAC 運算，圖 3-2 為本論文所提出的多位元 MAC 計算電路架構，其權重擺放方式為：縱軸為單一 filter 的大小，即 3×3 ；橫軸為 weight 的精準度，即 6-bit。整體電路區塊則包含：8T SRAM 做為該記憶體內運算的計算元，以避免在進行 MAC 運算時，發生 write distortion 的問題，以增加整體電路的穩健性 (Robustness)；在電路計算上，則是利用時間調變機制以取代 DAC 做使用，以實現式 3-1；最後為了將累加的電荷取樣至量化電路中執行量化運算，因此在 RBL 訊號線中將會加上取樣電容 (C_{RBL}) 與兩個非重疊 (Non-Overlap) 訊號控制的開關。其中 C_{RBL} 必須遠大於 8T SRAM 的寄生電容與 RBL 訊號線的寄生電容，以避免過多電荷的流失，導致計算精準度的下降；在取樣電容的實現上，則是使用金屬對金屬 (Metal-to-

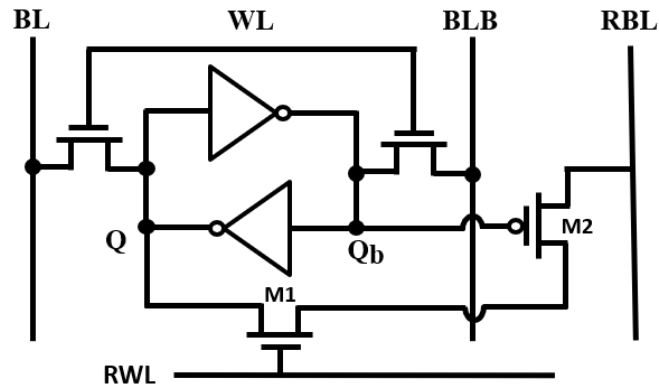


圖 3-3 8T SRAM 計算單元

在計算上，當 SRAM 儲存邏輯“1”時，即視為 weight 為 1 的情況，此時 M_1 將隨著 RWL 的開關時間長短，給予讀取路徑輸出轉換電荷，即為一乘法運算， M_2 則會因為 $\bar{Q}$ 訊號為邏輯“0”被開啟，將輸出電荷送入 RBL 進行累加，完成一次 MAC 運算，如圖 3-4 所示；當 SRAM 儲存邏輯“0”時，即視為 weight 為 0 的情況，此時 M_1 就算隨著 RWL 的訊號被開啟，但提供電荷的 Q 點為邏輯“0”，同時 M_2 則會因為 $\bar{Q}$ 訊號為邏輯“1”被關閉，因此能夠減少讀取路徑之漏電流 (Leakage) 累加到 RBL，以增加整體運算的精準度。

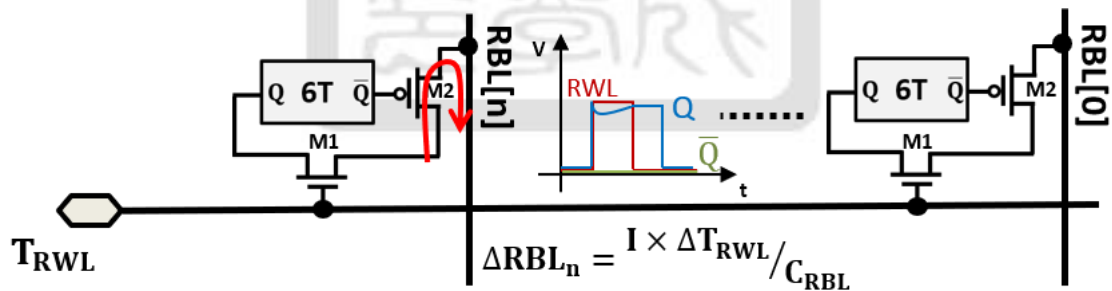


圖 3-4 當權重為 1 時，計算元操作情況

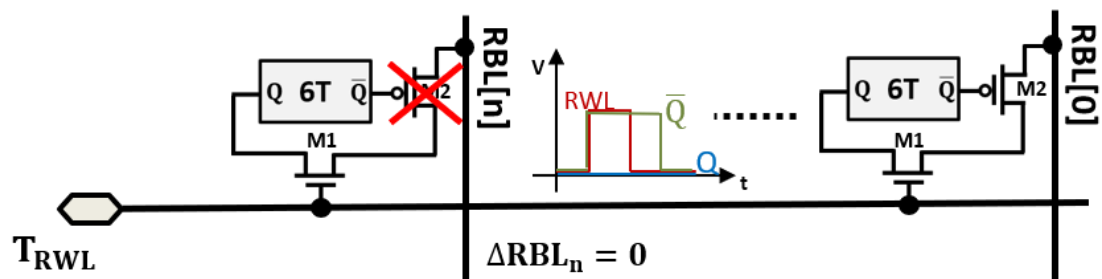


圖 3-5 當權重為 0 時，計算元操作情況

在 MACs 計算時序上，一共會有兩個操作階段：乘加計算階段 (MAC Phase, ϕ_1) 與量化階段 (Quantization Phase, ϕ_2)。因此當 ϕ_1 時，即為乘加計算階段，如圖 3-6 所示。連接 RBL 訊號線與 C_{RBL} 的開關將會被開啟，脈波調變後的不同開關時間長短的 RWL 訊號將會決定整體轉換的電荷量是多少，儲放在 SRAM 的 weight 則會決定是否輸出轉換電荷到 RBL 訊號上進行累加並取樣至 C_{RBL} 當中，在每一條 RBL 訊號線上的 C_{RBL} 大小皆相同，因此在計算的位元權重上皆相等，即完成單位元 weight 乘上多位元 IFMP 的 MAC 運算。

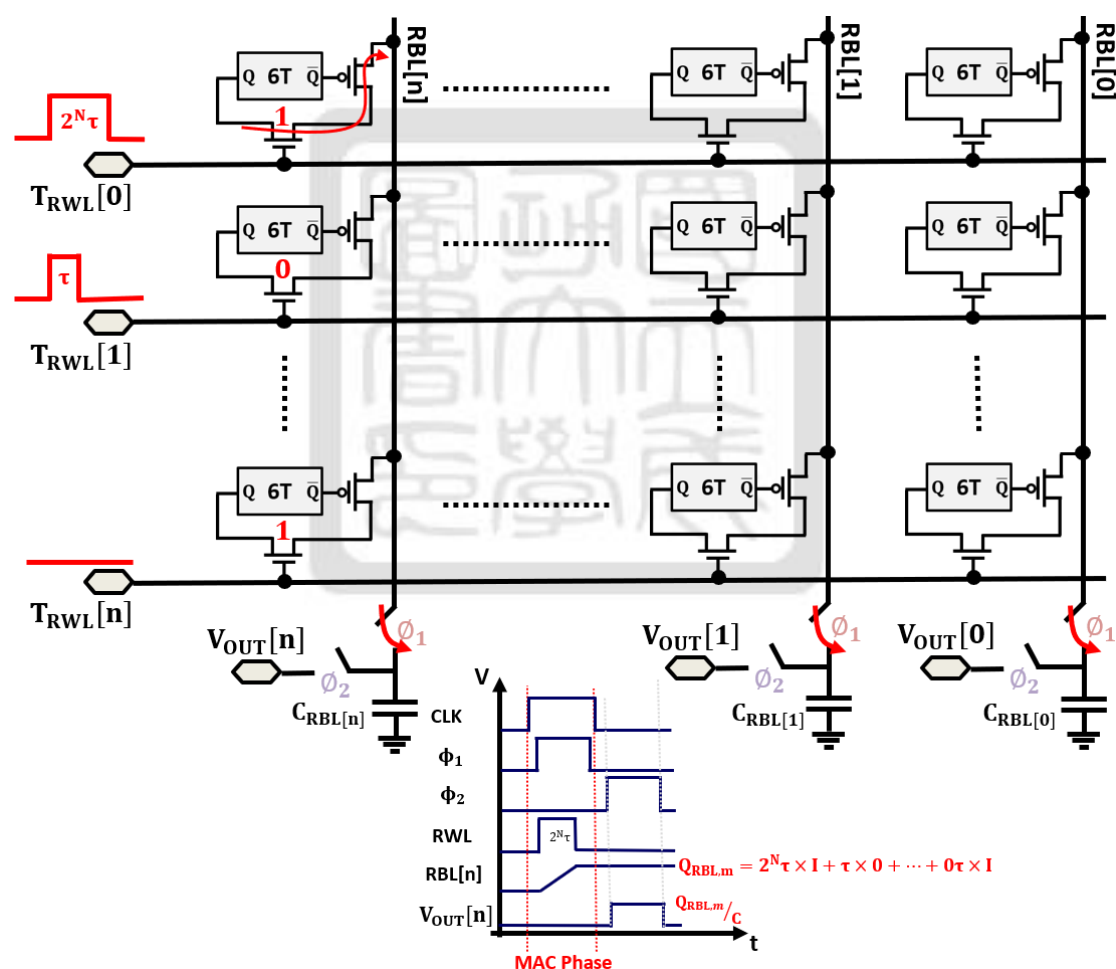


圖 3-6 當 weight block 操作於 ϕ_1 時之運算情況

當 ϕ_2 時，即為量化階段，如圖 3-7 所示。原先連接 RBL 訊號線與 C_{RBL} 的開關將會被斷開，連接 C_{RBL} 與下一級 VTC 電路的開關則會被開啟，將 C_{RBL} 所取樣

到的 MAC 電壓送入下一級電路，執行量化的運算；RBL 訊號線與提供 RWL 訊號的 PWG 將會在此階段被 reset 到電壓 0，以等待下一次 MAC 運算。

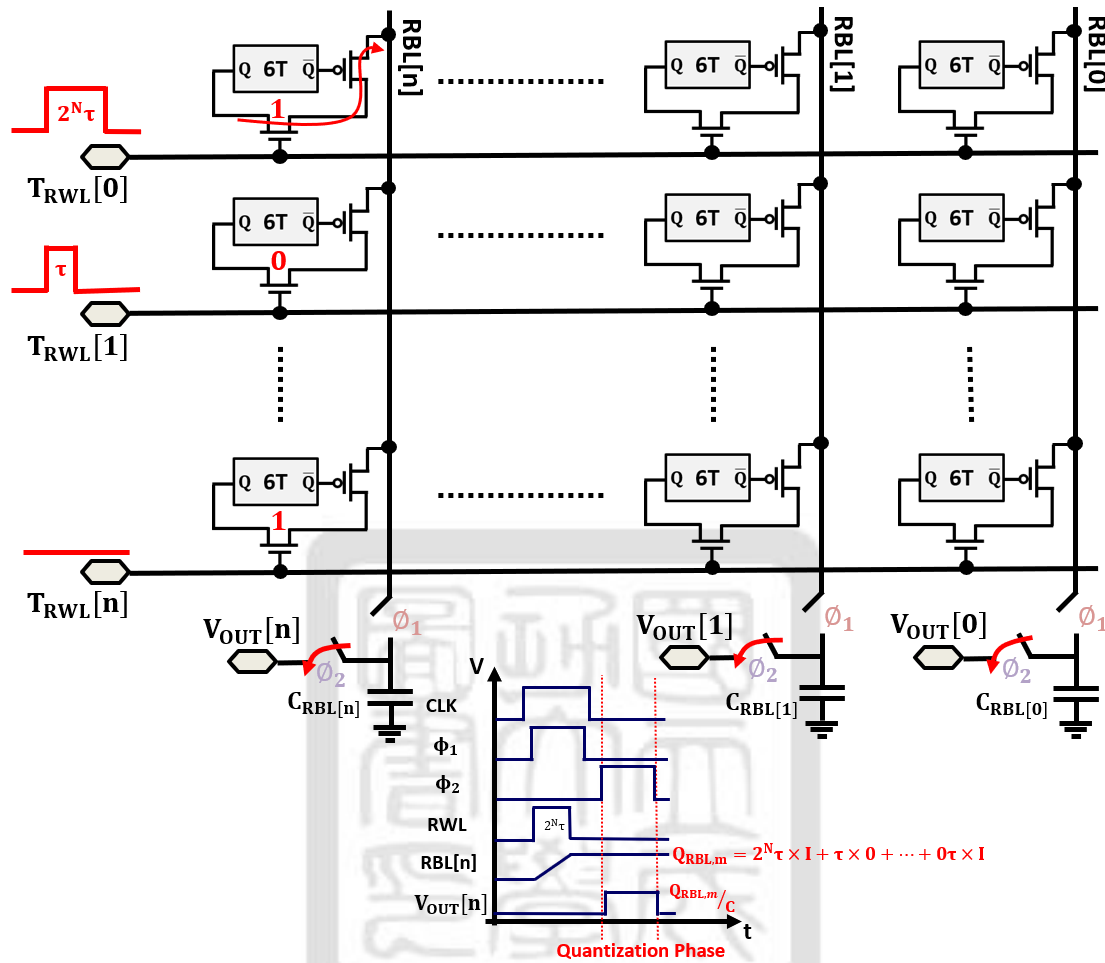


圖 3-7 當 weight block 操作於 ϕ_2 時之運算情況

3.4 脈衝寬度產生器

為了提供 RWL 有著不同時間長短的輸入 pulse，即時間脈波調變，以提供 weight block 執行 MAC 的計算，本論文使用 PWG 實現此功能，如圖 3-8 所示。其中包含了產生不同寬度 pulse 的 D 型正反器 (D-Flip-Flop, D-FF)、buffer 與及閘 (And Gate)；透過不同 IFMP 將不同寬度 pulse 傳入每條 RWL 中的 mux array。在機制實現上，只需要一組的 PWG 當作全域的輸入脈波，搭配 mux array 即可供給全部的 RWL 做使用，因此在面積與功率消耗上都能達到一定程度的節省。由於在設計單位 pulse 的寬度上約 100ps 左右，為了達到高速操作之目的，因此

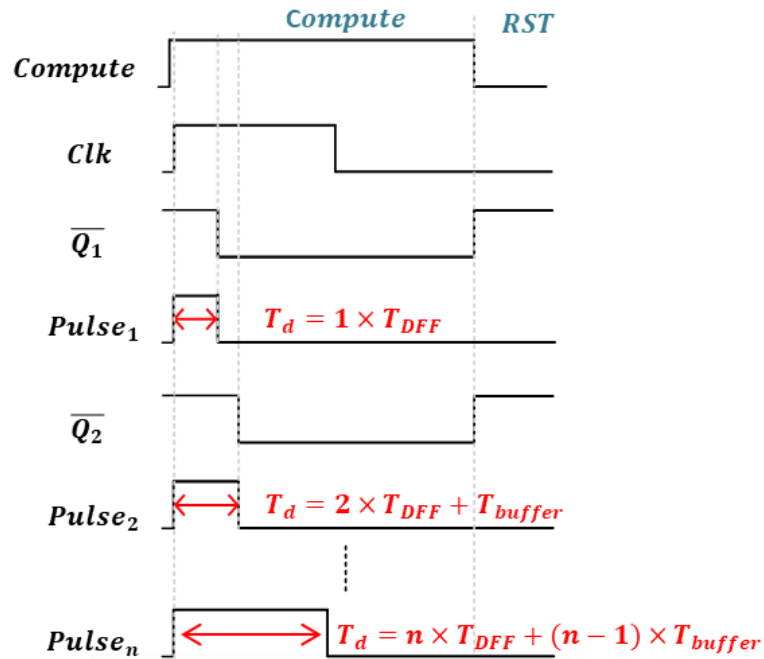


圖 3-9 脈波寬度產生器操作波型

圖 3-10 為有無時間調變機制下，DAC 與 PWG 的功率消耗情況。在實驗的對照組是選擇 4-bit 電荷重新分配式（Charge-Redistribution）DAC 的架構，在相同的模擬環境下做比較。由於將 IFMP 轉換成電壓的形式，無法只使用一個 DAC 供給全部的 RWL 在同一時間做使用，因此在本論文架構下需要 9 個做使用，其功率消耗則約 278 μ W；在使用時間調變機制下，單一 4-bit PWG 加上 Mux array 的功率消耗約為 207 μ W。因此在使用時間調變機制下，整體 CIM 架構可減少約 1.35 倍的功率消耗。若需要考慮到讀取路徑的線性範圍，則 DAC 中的電壓緩衝器規格與周邊電路將更複雜，以符合所需解析度，因此功率消耗的比較上，時間調變機制將能節省更多功率消耗。

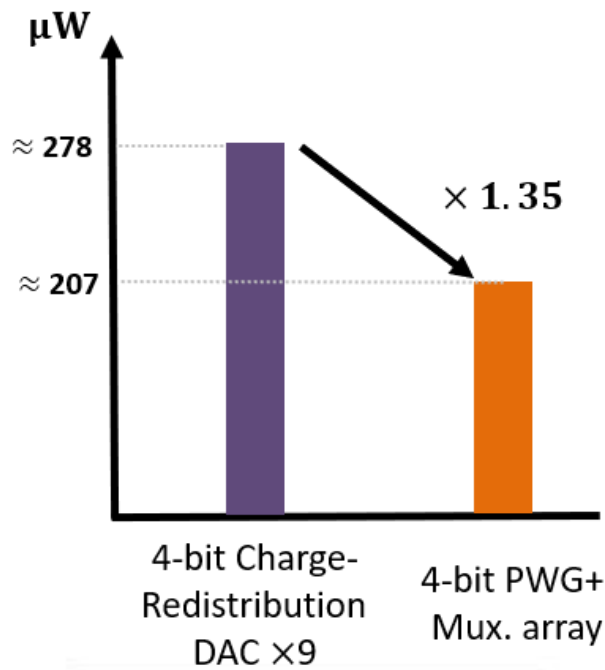


圖 3-10 有無時間調變機制之功率比較

3.5 電壓時間轉換器

隨著半導體製程的進步，數位電路的延遲時間隨著製程的進步而的縮短，大幅加速了整體電路的運算速度；然而，在半導體製程的微縮下，為了降低功率消耗，壓縮了電路的供應電壓，對於傳統的類比量化電路來說，大多數是使用電壓軸在進行資料的量化，導致取樣的精準度就必須上升，才能維持原先取樣電路的解析度[39]。除此之外，如圖 3-11 中當製程微縮時，整體的 offset 電壓並不會隨著製程進步而有太大的改善，因此需要更複雜的校正電路才有辦法控制 offset 的大小，不僅額外增加功率的消耗，也拖慢了整體的運作時間；但對於時間軸取樣而言，如圖 3-12 所示，當製程微縮時，整體的 offset 時間是隨著製程微縮而變小的。因此，將取樣電路設計在時間軸上進行取樣運算，相對於電壓軸上進行取樣是比較理想的，當製程微縮時，兼容性也較高。

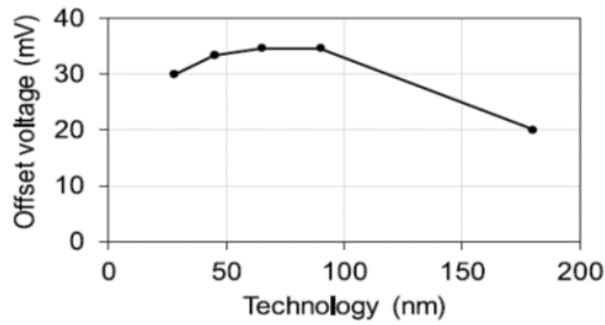


圖 3-11 製程與偏差電壓曲線圖[39]

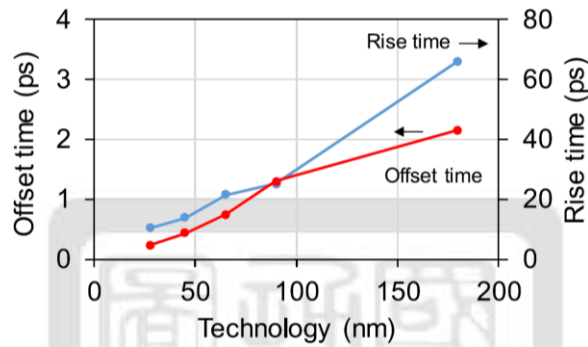


圖 3-12 製程與偏差時間曲線圖[39]

為了實現時間軸量化之目的，必須將 weight block 中，取樣電容所取樣到的電壓轉換成時間軸訊號，以便後級取樣電路執行取樣運算，因此在兩者之間，需要介面電路來完成此事，即電壓時間轉換器（Voltage-to-Time Converter，VTC）。在傳統的動態型 VTC 中，如圖 3-13 所示，當輸入電壓較小時，此時整體電路的運作行為一差動對，因此輸出電流與輸入電壓將成 gm 倍正比；當輸入電壓逐漸變大時，此時差動對失衡，整體電路不在屬於差動對運作行為。經過文獻[39]的輸出公式推導，如式 3-3 所示，當輸入電壓變大後，在輸出的三次方失真項將會被凸顯且放大，導致整體輸出的非線性。其中式 3-3 中的 V_{com} 為共模參考電壓； V_{TH} 為電晶體的閾值電壓（Threshold Voltage）； V_{in} 為 V_{inp} 與 V_{inn} 的差值。

$$T_{out} \approx \frac{2CVDD}{\beta_c(V_{com} - V_{TH})^3} V_{in} + \frac{CVDD}{\beta_c(V_{com} - V_{TH})^5} V_{in}^3 \quad \text{式 3-3}$$

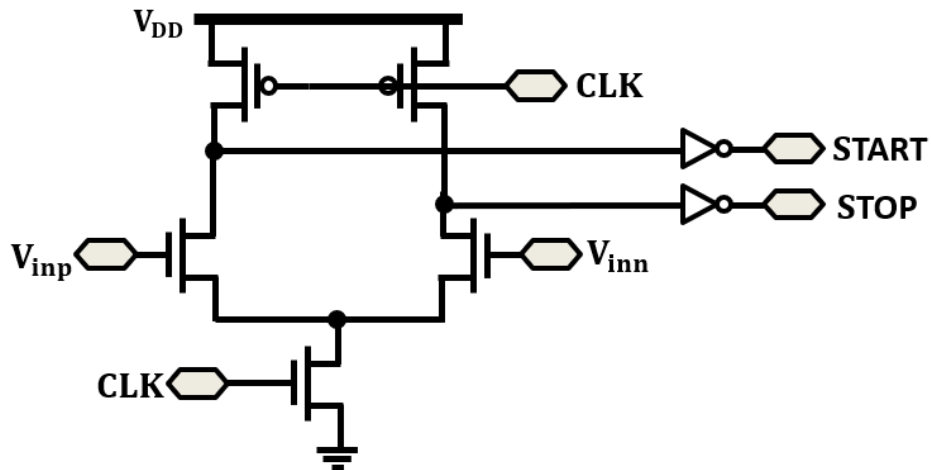


圖 3-13 傳統動態電壓時間轉換器[39]

為了避免當輸入過大時，輸出三次方失真影響整體 VTC 之線性度，導致量化電路無法正確的還原取樣電壓，因此本論文將利用文獻[39]中所提出的 VTC 為基底，做些許的電路修改，如圖 3-14 (a) 所示。透過由正參考電壓 (V_{ref+}) 控制 M5 與 M6 的電流大小，以補償當輸入電壓與負參考電壓 (V_{ref-}) 差距變大時的三次方失真效應。圖 3-15 為加入與未加入 M5 與 M6 電晶體時的輸入輸出轉換曲線，可以明顯的觀察出，當加入補償 M5 與 M6 電晶體後，能讓 VTC 的可操作線性區變大，因此 weight block 輸出訊號範圍的設計空間與彈性將會比傳統的 VTC 還要佳，對於後級取樣電路的時間解析度，將能設計的比較大，降低製程飄移的影響。

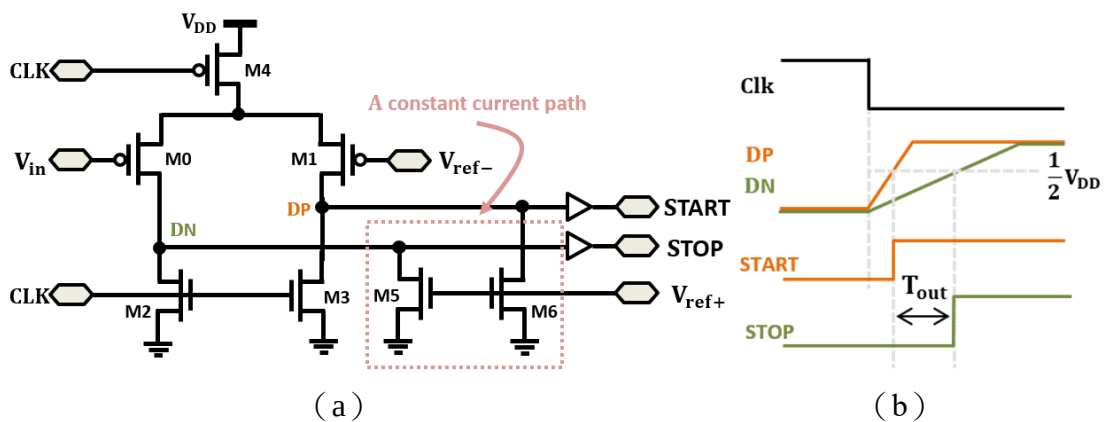


圖 3-14 具定電流校正之 VTC (a) 架構與 (b) 操作波型

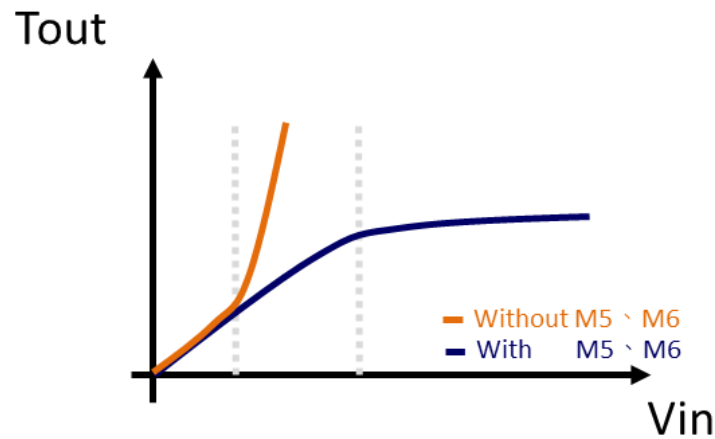


圖 3-15 VTC 輸入電壓與輸出時間補償前後轉換曲線

圖 3-14 (b) 為 VTC 整體電路的操作行為如下：首先當 CLK 訊號為邏輯“1”時，M2 與 M3 將會被開啟，此時 VTC 的輸出將被 reset 到邏輯“0”；接著當 CLK 訊號為邏輯“0”時，M2 與 M3 會被關閉，M4 將會被導通，藉此做為定電流源提供 M0 與 M1 使用，此時 M0 與 M1 將隨著輸入電壓的大小而產生不同導通強度的電流，藉由導通電流的大小對輸出電容充電，即能創造出兩個不同相位的輸出脈波，以提供後級取樣電路做使用，完成一次的電壓時間轉換。

3.6 時間數位轉換器

當 VTC 將取樣電壓轉換成兩個不同相位的輸出脈波後，時間數位轉換器 (Time-to-Digital Converter, TDC) 將會進行量化的運算，將兩個不同相位的輸出脈波量化為數位值。圖 3-16 (a) 為游標尺 TDC (Veriner TDC, VTDC) 電路架構[43]，由一快一慢的延遲線 (Buffer Line)、SA 與溫度計碼轉二位元編碼器 (Thermometer-to-Binary Encoder, T2B) 所組成的電路。其中第一條的延遲線的延遲時間為 t_{d1} ，第二條的延遲線的延遲時間為 t_{d2} ，在本論文中則是採用單端架構的 SA[44]做為量化器使用。VTDC 的優點在於 START 與 STOP 訊號節點負載是相似的，因此能夠避免延遲線型 TDC 在 STOP 訊號端負載過大的問題，導致

(a)

(b)

在效能上，VTDC 電路的最小解析度是由兩條延遲線的最小時間距離 (Time Interval) 所決定，如式 3-4 所示，因此在理論的解析度可以趨近於 0，即解析度可以突破至次閘延遲 (Sub-Gate Delay)，達到高解析度的輸出。但受於製程變異的影響，當 $t_{d2} > t_{d1}$ 時，將會造成量化上的錯誤，因此在實際上會將 t_{d1} 設計成 t_{d2} 的數倍大，避免製成變異造成的錯誤。另外，整體電路的轉換時間則由式 3-5 所示，可以發現到越精確的解析度會造成整體轉換速度的下降，因此在設計上必須由整體系統操作速度與輸出解析度去進行考量。

$$T_{\text{conversion}} = \frac{T_{\text{START}} - T_{\text{STOP}}}{t_{d1} - t_{d2}} \times t_{d1} \quad \text{式 3-5}$$

doi:10.6844/NCKU202002445

的 buffer 後，兩者訊號的差距將會縮減 $t_{d1} - t_{d2}$ 的距離。接著當 START 訊號尚未被 STOP 訊號追過時，由於 SA 的 CLK 被驅動時 D 點訊號皆為邏輯“1”，所以 SA 的輸出將會是邏輯“1”，因此可以藉由此資訊，得知 STOP 訊號需要經過幾個 SA 才能追上 START 訊號。當 STOP 訊號追過 START 訊號後，因為 SA 的 CLK 在 D 點為邏輯“1”前先被驅動，因此輸出 Q 點訊號為邏輯“0”，結束兩訊號追逐過程。最後 SA 的 Q 點訊號輸出將送入 T2B，完成訊號量化運算。

3.7 權重位元切割累加機制

在先前的相關文獻中，大多使用 charge sharing 的方式，在進入 ADC 前，將不同位元權重 MAC 的運算結果先進行累加的動作[20][22][23]，如式 2-2 所示。雖然此做法只需要一組的量化電路，就能完成 MAC 的量化運算，但當需要高精準輸出的 OFMP 時，就必須將量化電路的解析度提高。不幸的是，ADC 的輸出精準度與功率消耗及運行速度則是呈正向關係[20][45]。由圖 3-17 可以得知，當輸出精準度越高時，若只一味增加類比電路的輸出精密度，效益將不比數位電路，將導致整體記憶體內運算電路的 energy-efficient 下降。除了量化電路的解析度外，在執行 charge sharing 運算時，電容則會有佈局上的不匹配性（Mismatch）與製程上的變異問題，當 weight 的精準度提高後，RBL 訊號上的取樣電容將隨二指數次方增加，都將增加量化時誤差、操作速度的下降與面積的提高。

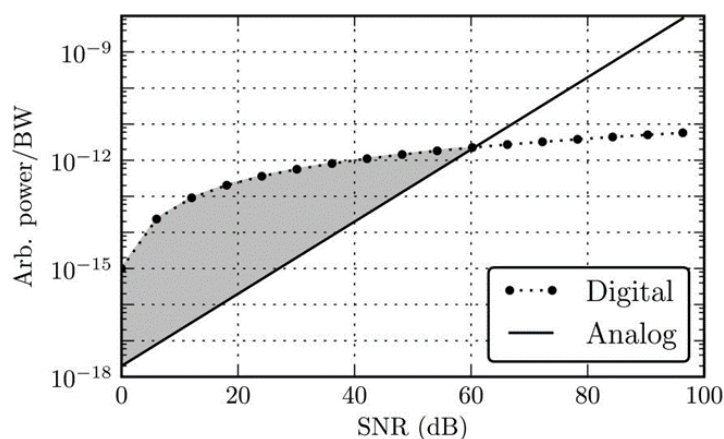


圖 3-17 數位與類比電路之 SNR 比較[45]

為了在高精準輸出下，也能維持低功率消耗與高速的操作，因此在本論文提出權重位元切割累加（Weight Bit-Partitioned Accumulation，WBPA）機制，即在每一 4-bit IFMP 乘上 1-bit weight 的 MAC 運算結果，先透過 3.6 節的 4-bit VTDC 將其值量化為次輸出（Sub-Output）4-bit 數位值，接著藉由二進制乘法運算規則，如圖 3-18 所示，再利用輸出結合器（Output Combiner，OC）還原 4-bit IFMP 乘上 6-bit weight 的多位元 MAC 完整運算，如圖 3-19 所示，其中電路包含了數位加法器及位移器。

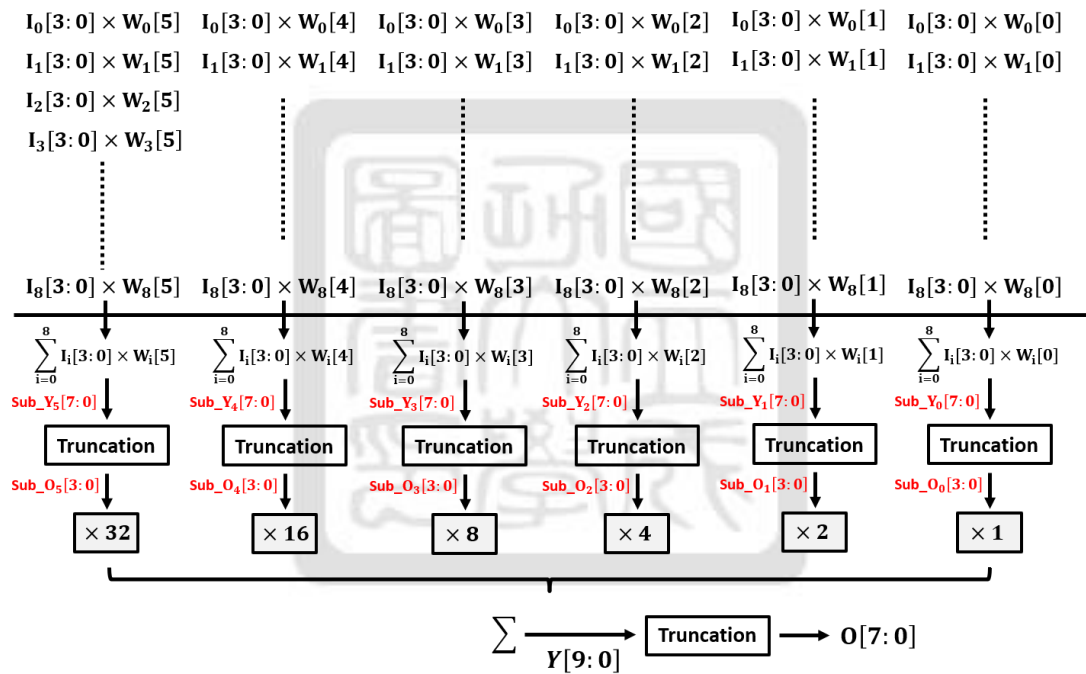


圖 3-18 直式二進制乘法

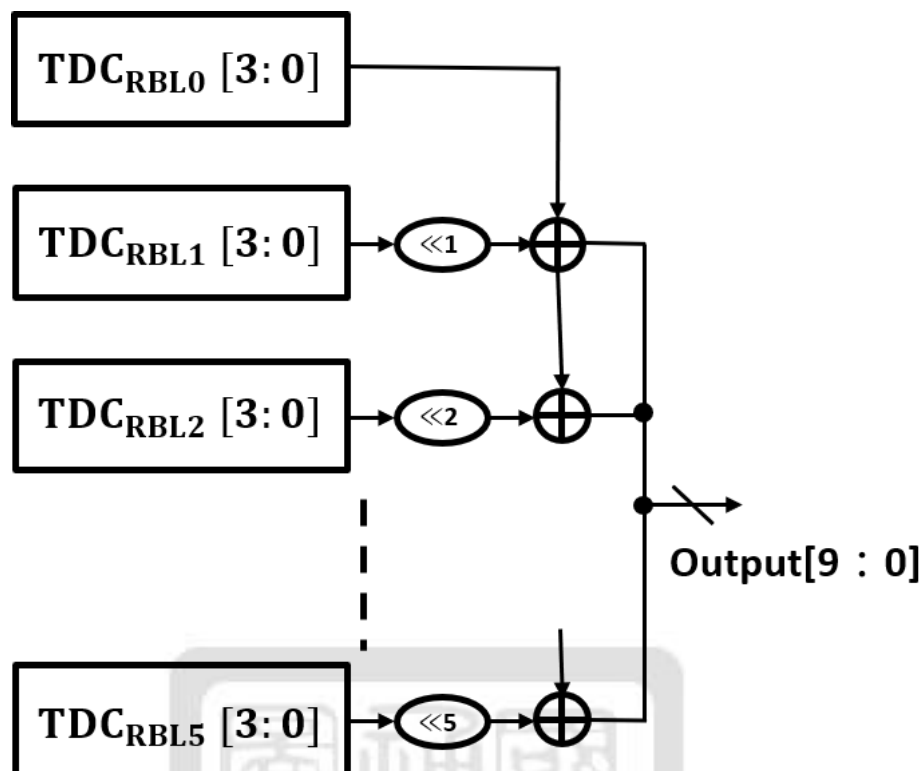


圖 3-19 輸出結合器

圖 3-20 為與先前相關文獻的記憶體內運算電路，在執行多位元 MAC 運算流程之比較，由於本論文執行位元權重合併計算時，操作於數位軸進行運算，而非與過去的文獻操作於類比軸進行 charge-sharing 的運算[20][22][23]，所以在計算上，本論文能夠少去一次執行類比運算的過程，因此對於 noise 的抵抗能力將能夠比過去文獻還要更佳；當 weight 的位元數提高時，也能避免受於電容陣列成二的指數次方增漲，而產生大量的面積消耗與受於製程的因素而產生電容間的 mismatch；最後，因為 WBPA 機制為利用每個 sub-output 所量化出的 MAC 運算值，合併成最終的輸出，因此當輸出精準度需要提高時，並不會受限於過去文獻中，需要設計單一高解析度 ADC 做為輸出，因此在操作速度與功率消耗將能得到大幅的改善，避免輸出精準度與 ADC 的解析度成直接且正向關係。

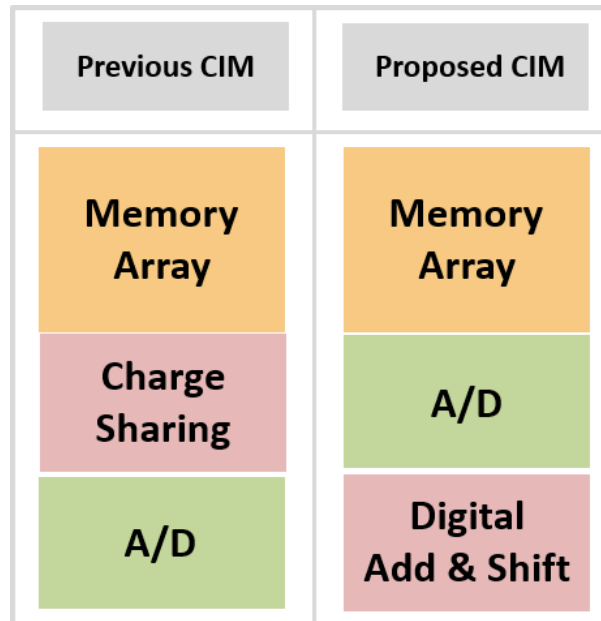


圖 3-20 傳統 CIM 與本論文 CIM 之運算流程比較

由於本機制的特點在於可以透過數位電路，將每個較低精準度的 sub-output 合併成一高精準度輸出的 OFMP。因此在本論文的架構下，當每個 sub-output 為相同 4-bit 解析度時，可透過 WBPA 機制得出最高 10-bit 的 OFMP 輸出，如式 3-6 所示。其中式 3-6 中 $Precision_{output}$ 為輸出精準度； $Precision_{weight}$ 為 weight 精準度； $Precision_{suboutput}$ 為次輸出精準度。因此當需要增加輸出精準度時，只需藉由增加 1-bit sub-output 與加法器的位元數，則輸出將能增加一倍 OFMP 的精準度，以避免過大的功率消耗與操作速度上的下降。

$$Precision_{output} = Precision_{weight} + Precision_{suboutput} \quad \text{式 3-6}$$

最後，為了驗證 WBPA 機制是否能有效的實現上述所描述的成效，圖 3-21 為單一個 8-bit VTDC 做為輸出量化器與使用 WBPA 機制下的 TDC cell（1 個 SA+2 個 buffer）數量比較，可以明顯得出，使用 WBPA 機制後的 TDC cell 數量將能減少約 2.67 倍左右，因此在面積與功率消耗都能得到有效的下降。此外，由於 8-bit VTDC 的 TDC cell 龐大，在佈局上容易造成不匹配的現象出現，將導致

整體線性度下降，對比於在 WBPA 機制下的佈局，只需要針對單一 sub TDC 做佈局匹配，因此能較容易得到高的輸出線性度與解析度的控制。在操作速度方面，在 8-bit VTDC 與本論文相同模擬環境下，最高的操作速度僅有約5MHz左右，與本論文所提出記憶體內運算架構的操作速度慢了 40 倍以上。因此在 WBPA 機制下，確實能同時有效增加輸出精準度、降低功率消耗與加快體運算速度。

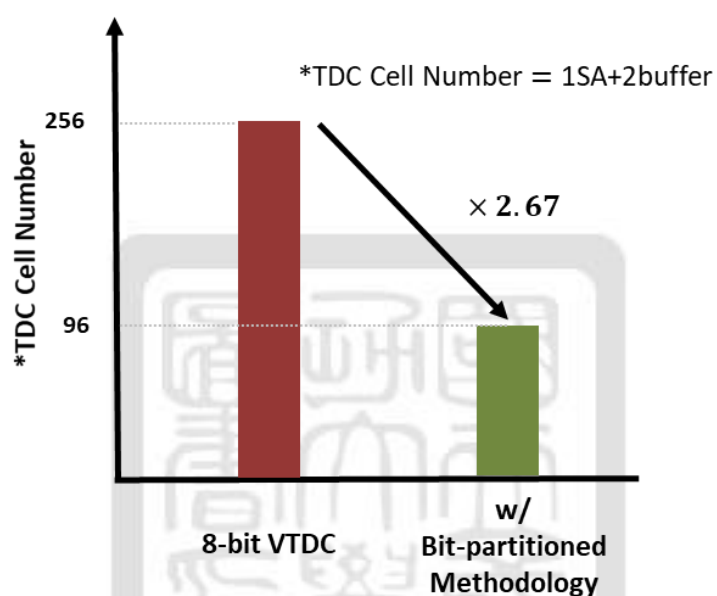


圖 3-21 有無位元切割機制之 TDC 細胞數量比較

第4章 實驗結果與分析

本篇論文所模擬環境是使用台積電 45nm CMOS LOGIC General Purpose Superb (40G) ELK Cu 1P10M 的製程模型下，且在 Hspice 環境進行 post-layout R+C+CC 之電路模擬。並包含 4 組的 4-bit IFMP、6-bit weight、8-bit OFMP 的高能量效益記憶體內運算及權重位元切割累加機制。晶片大小(含 IOPAD)為 1.18mm×1.18mm，並使用 CLCC 68-pin 做為封裝使用。

4.1 電路操作模擬

此小節將會針對第 3 章所提及之各電路區塊進行操作上的模擬、驗證電路區塊的行為是否正確及效能的分析，因此此小節皆為當 Compute 訊號升起時，即為電路的運算開始後之分析。

4.1.1 權重計算區塊操作模擬及線性分析

Weight block 最主要的分析內容為：接收 PWG 的 pulse 後，每個 8T SRAM 計算元是否有正確的提供輸出電荷給予 RBL 訊號線之中，如圖 4-1 所示。當 clk 與 compute 的訊號升起後，將會合成出 clk_compute 訊號，weight block 將會被啟動，8T SRAM 則會收到由 IFMP 所決定的 RWL 訊號的 pulse 時間寬度，開始執行電荷乘法的運算，接著 RBL 訊號則會隨著 RWL 訊號的 pulse 時間寬度進行電荷累加至取樣電容中。當 pulse 落下後，weight block 將不在對取樣電容進行充電之動作，即 SRAM 停止供應電荷給 RBL。最後，RBL 訊號會在 CLK 訊號的負半周與取樣電容斷開，因此紅色區域外的電壓雖然是不穩定的，但是此時與後級 VTC 並非相連，因此不影響操作。

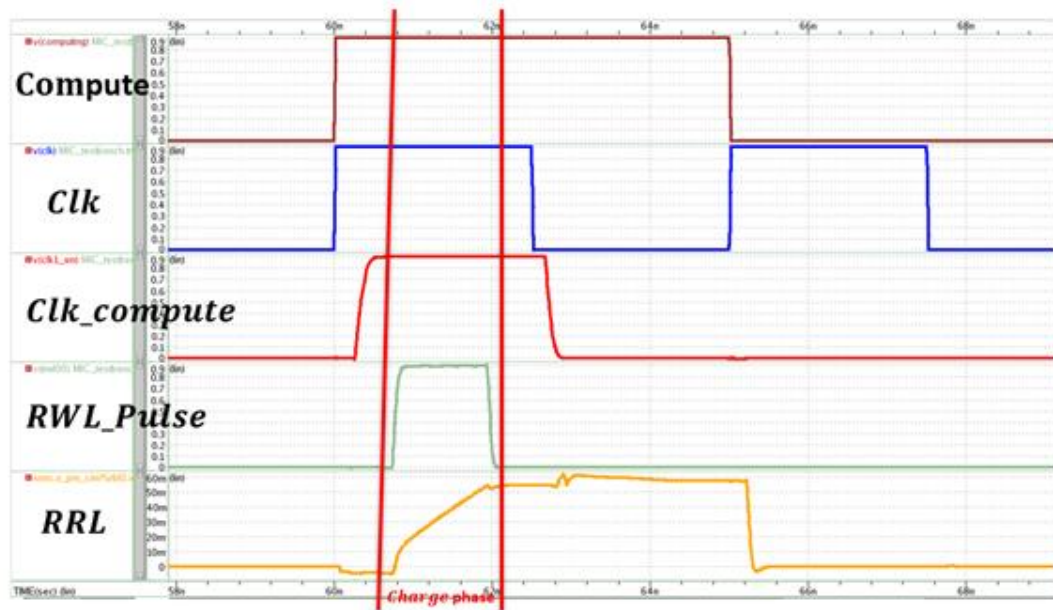


圖 4-1 Weight block 輸出累加模擬 (post-sim.)

接著為了觀察 weight block 是否為線性輸出，因此將輸入訊號改為 4-bit counter 灌入 PWG 電路中，類比斜坡訊號 (Ramp Function) 灌入 weight block 當中進行觀察，如圖 4-2 所示，為了方便觀察輸出是否線性，因此在右縱軸加入 INL 資訊。當輸入脈波寬度為 0τ 到 15τ 時，此時 INL 為正負 1.12 的 LSB；當輸入脈波寬度縮小為 2τ 到 15τ 時，INL 僅有正負 0.15LSB。有此實驗結果推論，當輸出訊號較小時，因為讀取路徑 MOS 的 V_{DS} 跨壓較大，導致了充電速度較輸出訊號大時還快，造成前端的非線性問題。最後，雖然整體 INL 稍大於文獻[38]，但排除前端非性區後，本論文所提出的累加放式之線性度則明顯優於文獻[38]，因此在需要高線性度的應用場景使用時，仍需簡單的校正電路做補償，如在 RBL 中增加模仿單元 (Dummy Cell)，以彌補前段充電速度較快之問題。

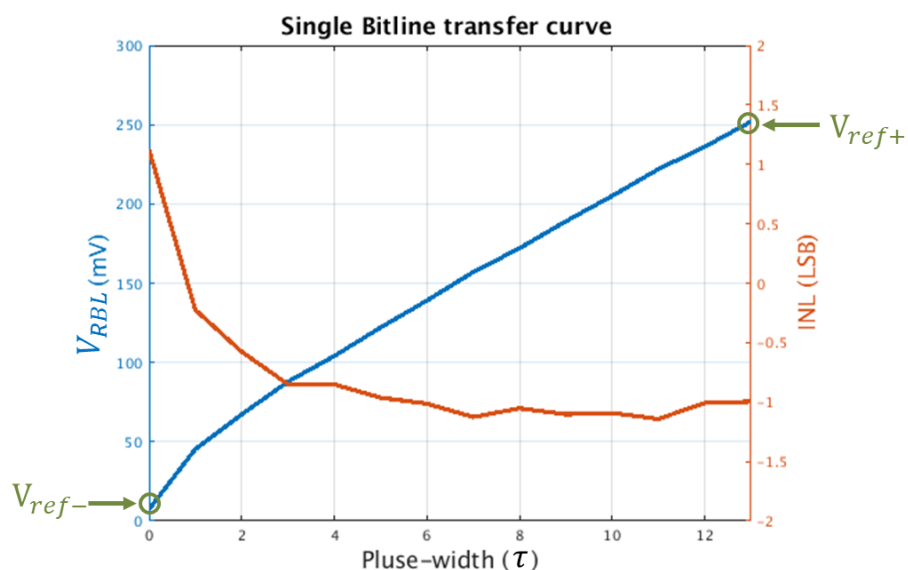


圖 4-2 Weight block 輸入輸出轉換曲線 (post-sim.)

4.1.2 取樣電容陣列

在 3.3 節中提到本論文所使用的取樣單位電容為 MOM 電容，如圖 4-3 所示，此取樣電容由 M3~M7 五層金屬所組成，中心處由 M4、M5 與 M6 組成，為單位電容的上板；外圍則由 M3~M7 組成，為單位電容的下板。此電容的好處在於下板將上板完全圍住，因此可以降低上板對外的寄生電容，在佈局上可以減少非線性現象的發生，此外，由於單位電容值不大，因此可以降低在 weight block 在取樣時的功率消耗。本論文的單位電容為 $1.2\mu\text{m} \times 1.2\mu\text{m}$ ，電容值約為 0.5 fF。

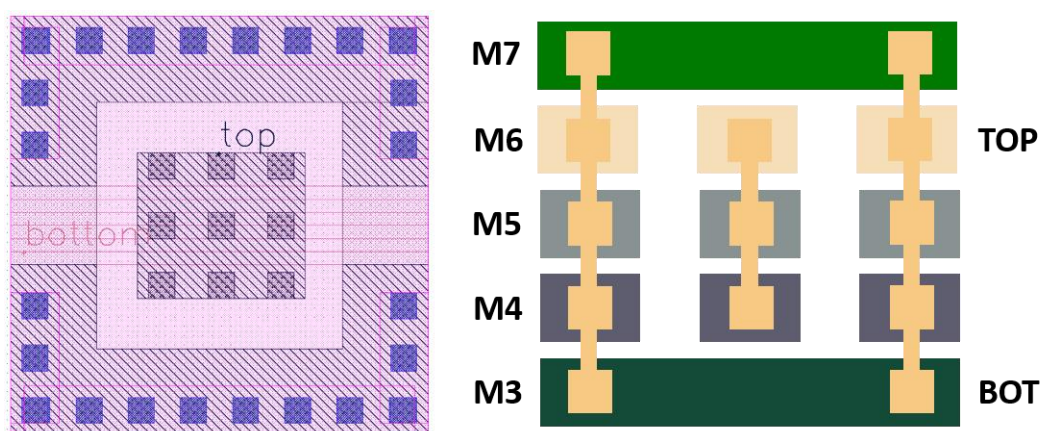


圖 4-3 MOM 單位電容

圖 4-4 為本論對於單位取樣電容的佈局擺法，由於 WBPA 機制的關係，MSB 的精準度將值影響 OFMP 精準度，為了降低製程的變異，減少每一條 RBL 的不匹配性，因此由 MSB 到 LSB 往外排列，在電容陣列外則加上模仿電容 (Dummy Capacitor) 與護圈 (Guard Ring) 降低周邊訊號造成的耦合 (Coupling)。每一條 RBL 訊號上的取樣電容值約為 13 fF，總電容陣列值約為 78 fF，總電容陣列大小則約為 $43.9\mu\text{m} \times 13.5\mu\text{m}$ ，配合 weight block 的佈局寬度做排列。



圖 4-4 取樣電容佈局陣列

4.1.3 8T SRAM 運算元之靜態雜訊邊際分析

此部分的模擬是為了決定 8T SRAM 的最低操作電壓，以降低整體電路的功率消耗；當 8T SRAM 在執行 MAC 運算時，是否能保持高可靠度，因此使用了靜態雜訊邊際 (Static Noise Margin, SNM) 分析以便觀察。SNM 主要由 SRAM 中的 latch 的轉態點決定，如圖 1-13 所示。因此將利用保持靜態雜訊容忍 (Hold SNM, HSNM) 的方式，如圖 4-5 所示，將 WL 訊號關閉，觀察 8T SRAM 的抗雜訊能力，確保在一定的操作電壓下，不因雜訊而造成 latch 的儲存資料被翻轉，造成資料儲存的錯誤。接著將會使用讀取靜態雜訊容忍 (Read SNM, RSNM) 的方式，如圖 4-6 所示，將 RWL 訊號開啟，觀察當 RBL 為 weight block 輸出最大電壓 250 mV 時，在我們選定的電壓下，是否會影響到 8T SRAM 的內部資料遭到誤寫。

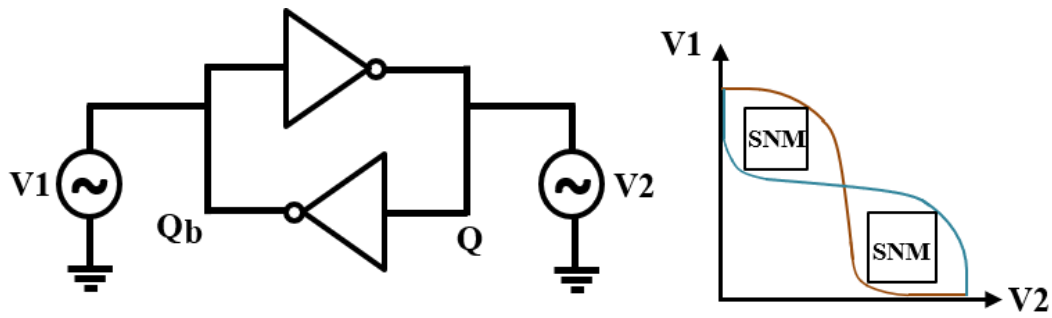


圖 4-5 8T SRAM HSNM 分析模型與波型

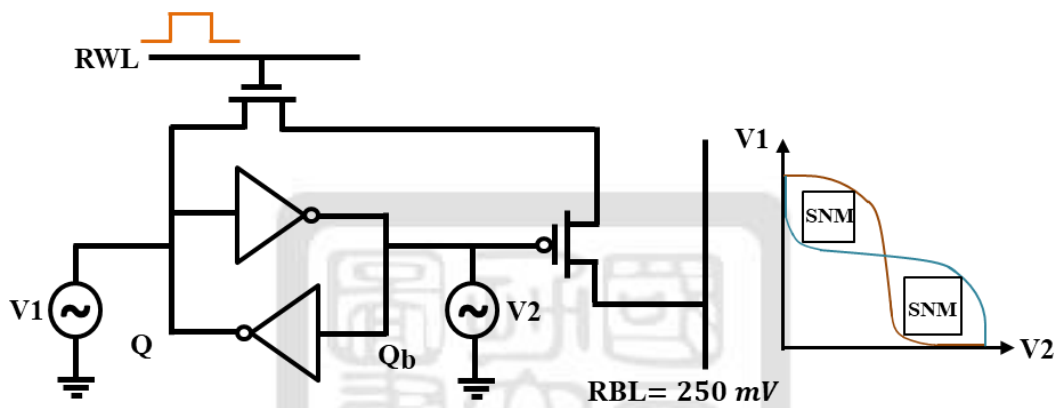


圖 4-6 8T SRAM RSNM 分析模型與波型

首先，當操作電壓為 0.9 V 且為 TT corner 時的 HSNM 模擬與 RSNM 模擬。首先圖 4-7 為 HSNM 的模擬結果，雜訊邊界約為 0.316V，接著為了觀察在製程變異下，是否會導致 8T SRAM 翻轉，因此在本地蒙地卡羅（Monte Carlo）的變異下，觀察整體變化情況。當執行 1000 次本地蒙地卡羅後，整體雜訊邊界標準差為 17 mV 的大小，在三倍標準差的值約為 51 mV，因此則有一定的信心水準表示在 0.9 V 的運作下，8T SRAM 所儲存的數值是正確無誤的；接著圖 4-8 為 RSNM 的模擬結果，雜訊邊界一樣約為 0.316 V，在 1000 次本地蒙地卡羅的變異下，約為 17 mV，因此可以證明 3.3 之敘述，此架構將與傳統 8T SRAM 架構一樣，不會遭遇 write distortion 的問題。

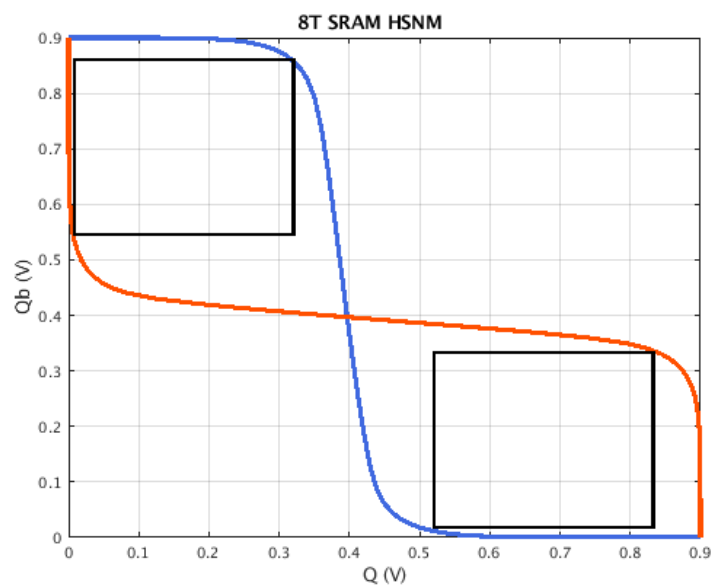


圖 4-7 當操作電壓為 0.9V 時的 HSNM (pre-sim.)

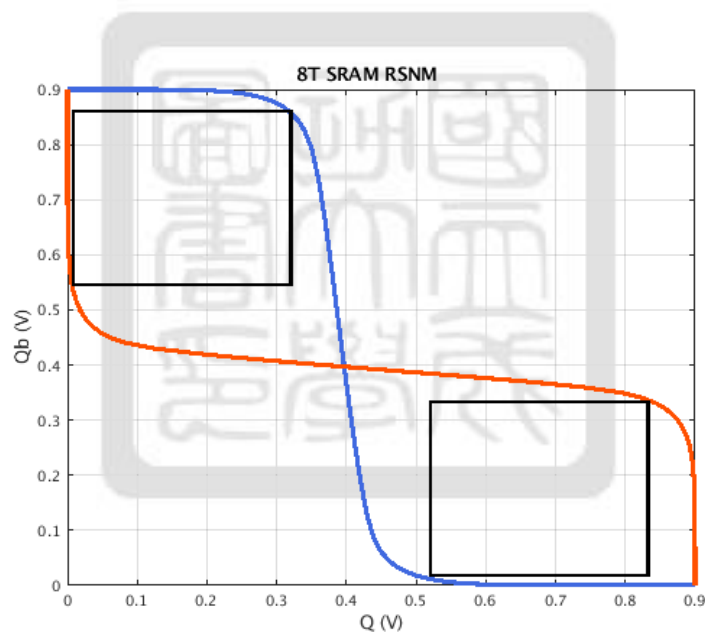


圖 4-8 當操作電壓為 0.9V 時的 RSNM (pre-sim.)

接著為了找尋合適的操作電壓，以避免額外功率消耗的產生，因此在實驗中將使用 RSMN 之分析。當為 TT corner 時，將變動操作電壓，觀察在 1000 次的本地蒙地卡羅當中，是否將有機會發生資料翻轉情況，表 4-1 為操作電壓由 0.5 V 至 0.35V 的變動下，RSNM 的寬度大小與其標準差的趨勢表，當操作電壓逐漸下降時，RSNM 的區域隨之變小，但標準差卻越來越大。為了確保在三倍標準差

的信心水準下，8T STAM 計算元能夠維持良好的操作穩定度，因此將選擇大於 0.4 V 以上的操作電壓，確保 RSNM 的大小以保證執行 MACs 運算時，不會發生 write distortion 現象。

表 4-1 操作電壓與 RSNM 趨勢關係 (pre-sim.)

Power Supply (V)	RSNM (V)	Standard Deviation (mV)
0.5	0.165	17.3
0.45	0.14	17.4
0.4	0.12	17.5
0.35	0.1	17.6

最後，本論文所提出的 8T SRAM cell 的面積為 $1.93\ \mu\text{m} \times 0.655\ \mu\text{m}$ ，與傳統架構 8T SRAM cell 相比之下，面積只增加約 0.7% 的消耗，單一 filter 陣列大小為 54-bit。

4.1.4 脈波寬度產生器操作模擬及穩定性分析

PWG 電路主分析的內容為，是否能正確產生 4-bit 解析度的脈衝，因此將會模擬在本地蒙地卡羅的變異下，是否會因為製程飄移而發生輸出脈波的重疊。圖 4-9 為當 IFMP 為 2b'0000 時，mux array 接受 PWG 的輸出訊號後，連接至 RWL 時的輸出變異情況，當執行 100 次本地蒙地卡羅後，輸出平均值為 85.1 ps，即為單位的脈波寬度，輸出標準差為 4.12 ps；則為當 IFMP 為 2b'1111 時，100 次本地蒙地卡羅後，輸出平均值為 1.19 ns，輸出標準差為 11.8p。從模擬結果可以得知，不管當 IFMP 為 2b'0000 或為 2b'1111 時，標準差僅數皮秒 (Picosecond)，而在最差三倍的標準差值約 30 ps，其遠小於單位脈波寬度，因此則有一定的信心水準表示在 4-bit 解析度下運作時，輸出精準度是正確無誤的。此部分模擬的最主要原因為觀察其變異趨勢，由於當 100 次蒙地卡羅後之結果標準差與單位脈

衡相去甚遠，因此將不提高模擬次數，以節省實驗時間。

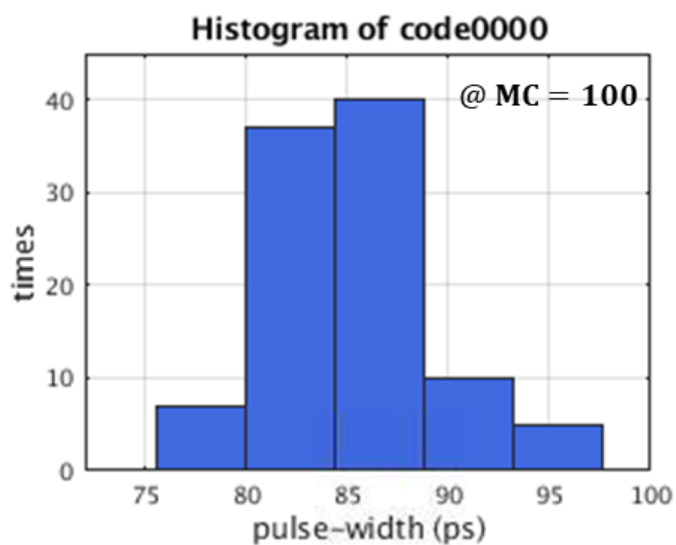


圖 4-9 當 IFMP = 4b'0000 時，PWG 輸出變異情況 (post-sim.)

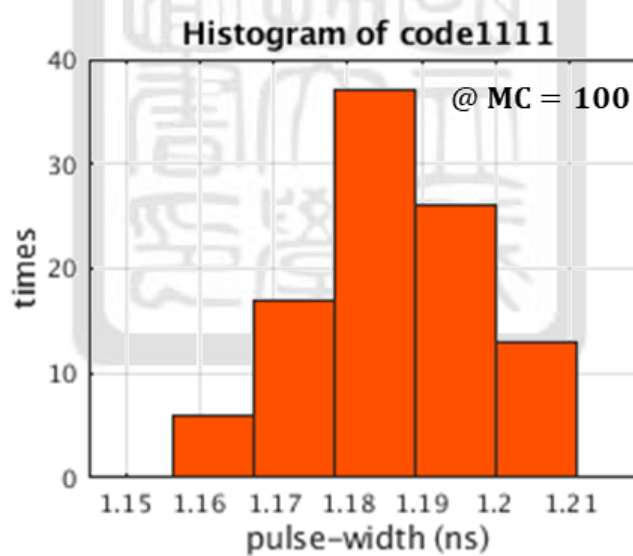


圖 4-10 當 IFMP = 4b'1111 時，PWG 輸出變異情況 (post-sim.)

最後為了觀察輸入資料經過 PWG 後仍維持線性轉換，避免在乘法運算時，有量化誤差過的問題，確保整體之線性乘度。圖 4-11 為 PWG 之 INL 非線性分析，其值為 $-0.563\text{LSB} \sim 0.661\text{LSB}$ 。

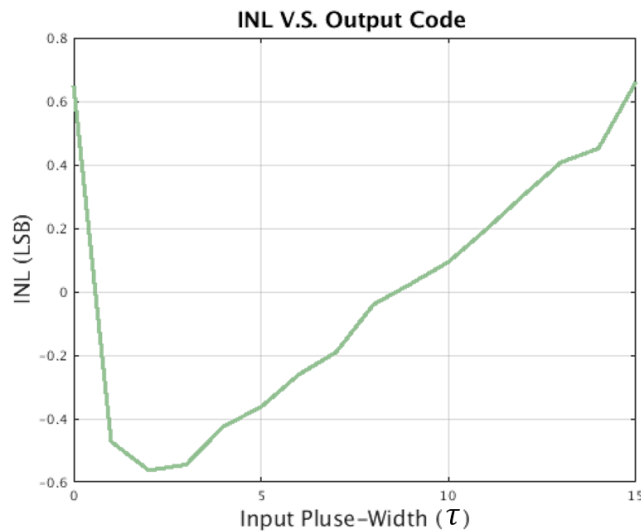


圖 4-11 4-bit PWG 之 INL 模擬 (post-sim.)

4.1.5 電壓時間轉換器操作模擬及穩定性分析

VTC 電路最主要觀察為，是否能將 weight block 所計算出來的 MAC 電荷轉換成時間軸輸出，圖 4-12 為 VTC 操作過程。當 CLK 訊號為負半周時與 compute 訊號為邏輯“1”時，系統將會合成出 clk_quantize 訊號驅動 VTC 需要開始工作與 weight block 所計算出來的 MAC 電荷會經由 ϕ_2 開關，將值傳入 VTC 的輸入端；當 compute 為邏輯“0”後，則進行 rst 的動作。圖 4-12 之模擬結果為，當 RBL 解出 58.2mV 時，VTC 將值轉換成 33 ps 的時間間距給下一級 TDC 做使用。

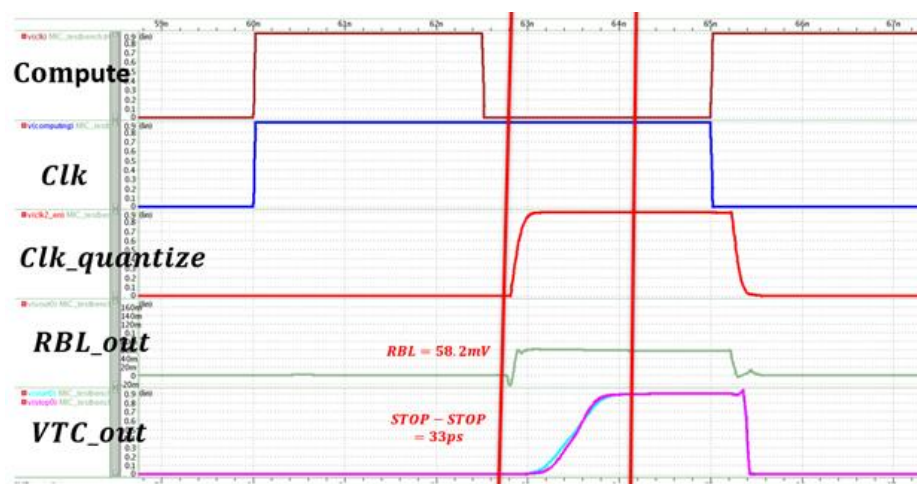


圖 4-12 VTC 轉換過程模擬 (post-sim.)

接著為了觀察 VTC 的輸入輸出曲線，是否如圖 3-15 所示，因此將單獨把此電路區塊拉出模擬觀察。圖 4-13 之橘色曲線為有增加 M5 與 M6 電晶體時，當輸入為 ramp function，輸出的時間差的變化情況，由於圖 4-2 的模擬結果得知，weight block 的最大電壓輸出最大約 0.25 V 左右，而 VTC 最大輸入範圍為約 0.5 V 左右，因此 VTC 在 weight block 的輸出範圍內將保證為線性輸出，且放大增益為 $1.42\mu\text{s}/\text{V}$ 。圖 4-13 之藍色曲線為無增加 M5 與 M6 電晶體時，輸入一樣為 ramp function，輸出的時間差的變化情況，明顯可以觀察到，因為沒了 M5 與 M6 在輸出提供定電流，因此輸入電壓不到 weight block 的最大電壓輸出 0.25V 時，輸出已經發散。

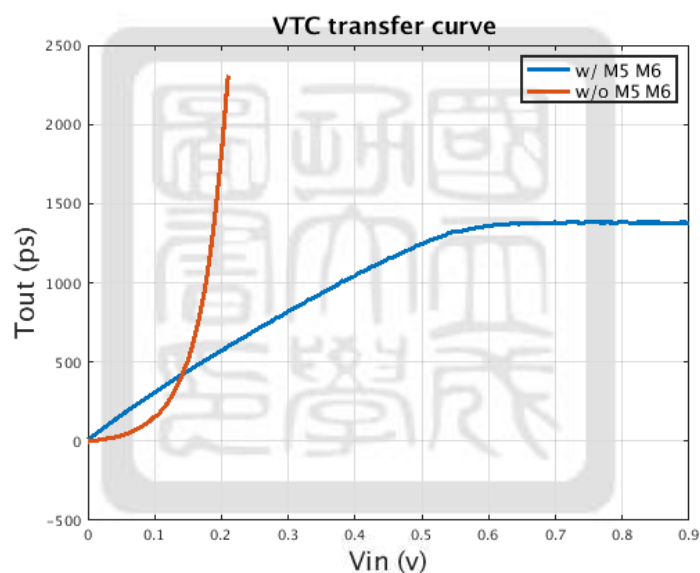


圖 4-13 VTC 之輸入電壓與輸出時間轉換曲線模擬 (post-sim.)

最後則是觀察 VTC 在不同的製程變異下，是否能維持一定的放大倍率，因此將進行 corner 的模擬測試觀察。圖 4-14 為各個 corner 的模擬下，VTC 放大增益變化情，若以 TT corner 為參考點，最大正變化則落於 SS corner 約 2.82%；最大負變化則落在 FF corner 的 -5.63% ，整體的變化率約 8.45%，與文獻[39]相比下，因為多了正負參考電壓的校正輔助，因此增了其電路穩定性，避免製程的影響，導致後級量化電路無法正常執行量化之運算。

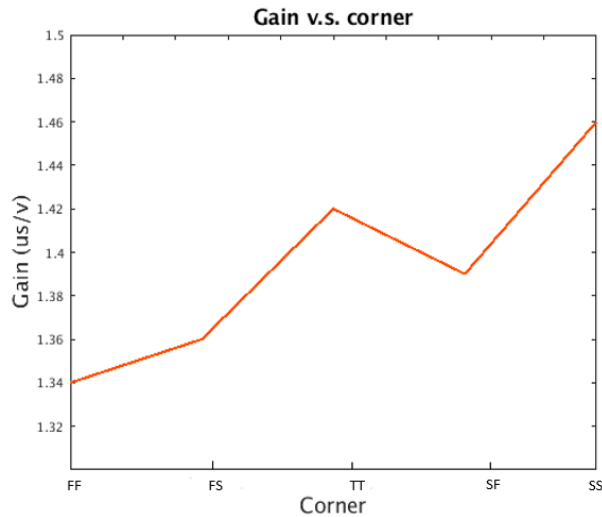


圖 4-14 VTC 增益之 corner 變異模擬 (post-sim.)

4.1.6 時間數位轉換器操作模擬及效能分析

TDC 最主要觀察是否能將上一級 VTC 的輸出脈波轉換成 4-bit 數位碼，送入 OC 電路，完成整體系統運算。圖 4-15 為當 CLK 訊號為負半周時且 compute 訊號為邏輯“1”時，系統將會合成出 clk_quantize 訊號驅動 TDC 開始工作。當 TDC 接收到 VTC 的 START 訊號後，就開始進行量化的運算，最後當 STOP 訊號追上 START 後，將會結束量化運算。圖 4-15 為 4-bit TDC 當 START 訊號減去 STOP 訊號的脈波寬度為 258.13 ps 時，TDC 的輸出將為 2b'1010。

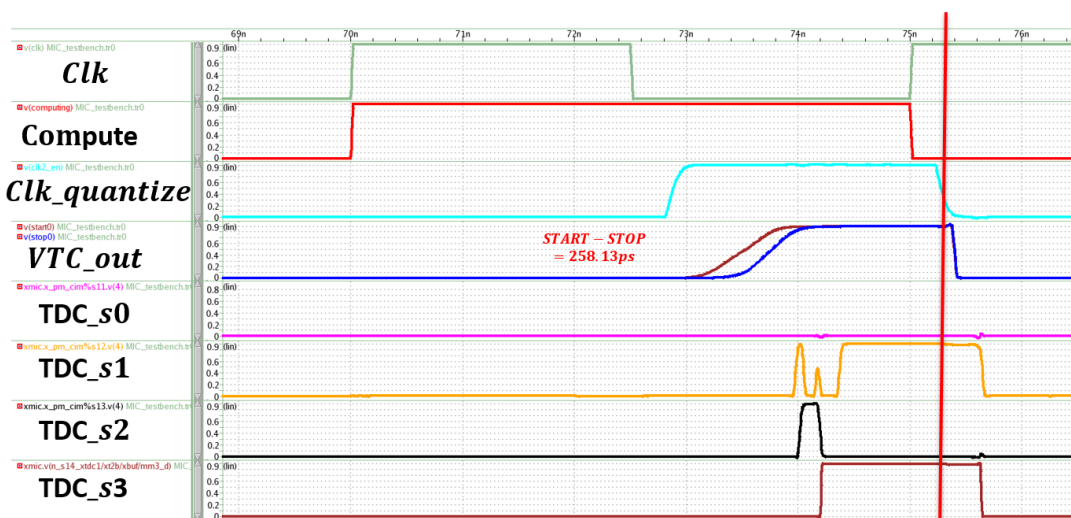


圖 4-15 4-bit TDC 量化過程模擬 (post-sim.)

接著為了量測 TDC 操作在 200 MHz 是否能夠輸出 4-bit 之解析度，因此將 TDC 與 VTC 單獨拉出做模擬。圖 4-17 為動態效能模擬配置，為了符合協調取樣（Coherent Sampling）的要求，因此在 VTC 的輸入將會灌入正弦波，其峰值為 weight block 的最大值輸出電壓 0.25 V，頻率則約為 24.22 MHz，採用 256 點取樣執行快速傅立葉轉換（Fast Fourier Transform，FFT）。圖 4-17 為最終 FFT 模擬結果之輸出頻譜圖，其 SNDR 為 24.49 dB，ENOB 則為 3.78bit。

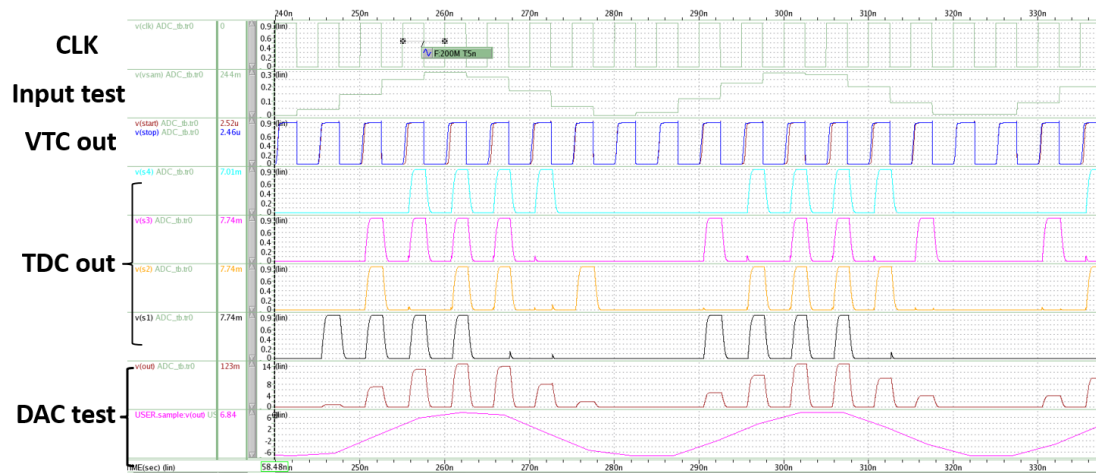


圖 4-16 4-bit TDC 動態效能模擬（post-sim.）

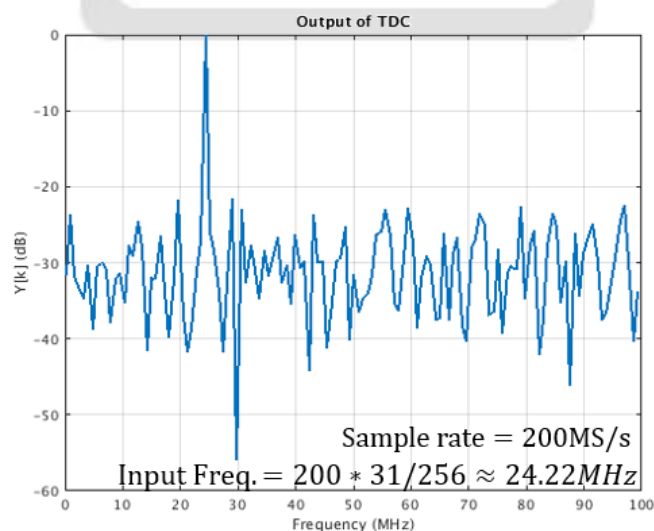


圖 4-17 4-bit TDC FFT 模擬頻譜圖（post-sim.）

最後為 TDC 線性度分析，輸入則由正弦波改為 ramp function，峰值一樣為 weight block 的最大值輸出電壓，其餘配置與 FFT 模擬相同。圖 4-18 為 4-bit TDC 之輸入輸出轉換曲線，圖 4-19 為輸出 4-bit TDC 之 DNL 結果，其值為 $-0.168\text{LSB} \sim 0.28\text{LSB}$ ，圖 4-20 為輸出 4-bit TDC 之 INL 結果，其值為 $-0.45\text{LSB} \sim 0.0837\text{LSB}$ 。

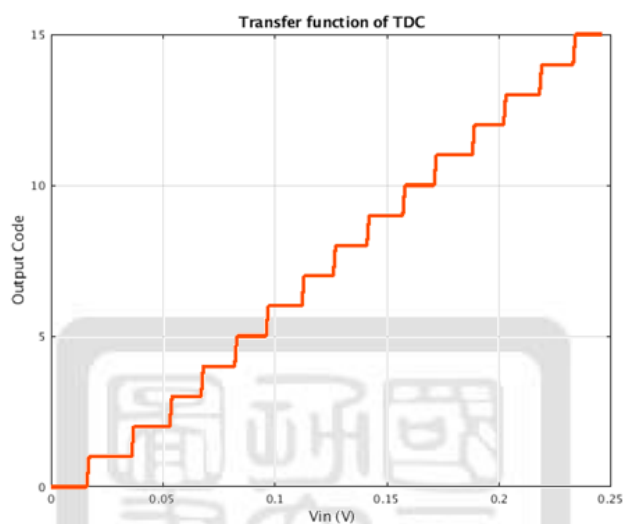


圖 4-18 4-bit TDC 輸入輸出轉換曲線 (post-sim.)

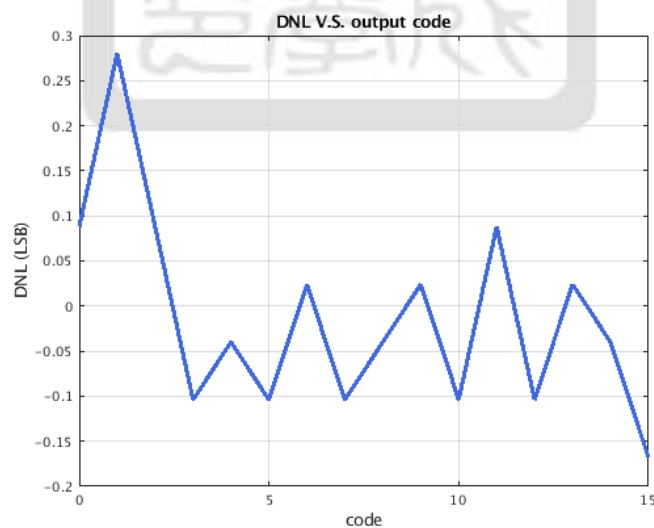


圖 4-19 4-bit TDC 之 DNL 模擬 (post-sim.)

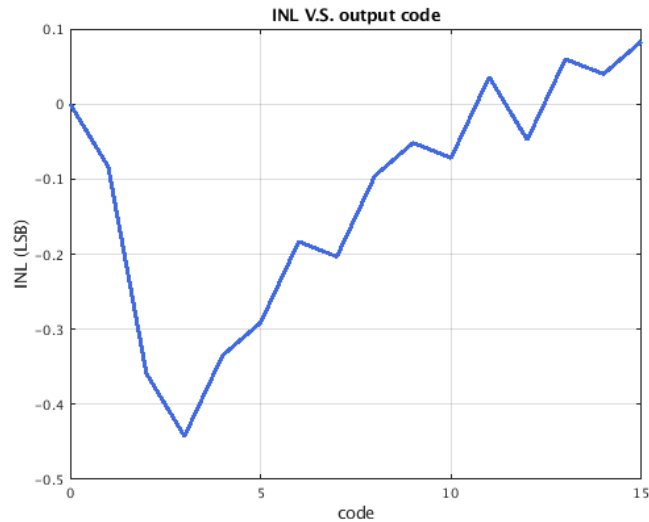


圖 4-20 4-bit TDC 之 INL 模擬 (post-sim.)

4.2 記憶體內運算參數模擬

在 4.1 小節中，最主要的實驗內容為 CIM 電路中，各區塊電路是否正常運作與達到所需規格。本小節的實驗為整體 CIM 的輸出，是否如預期的變化與規格的表現。

4.2.1 非線性分析

由 4.1.1 小節的模擬分析以發現到，當單一 RBL 計算累加較小時，容易產生非線性之問題。在整體的 INL 分析中，一樣可以觀察到，較大的非線性區同樣落在計算累加值較小時的情況，如圖 4-21 所示。圖 4-21 為整體 CIM 輸出之 INL 分析曲線，當電路操作於 200 MHz 與操作電壓為 0.9V 時，整體 INL 為 $-0.58 \text{ LSB} \sim 0.91 \text{ LSB}$ 。因此在前段累加上，在未來仍是一大改善空間，如增加 dummy cell，以補償前段 8T SRAM cell 充電速度較快之問題。

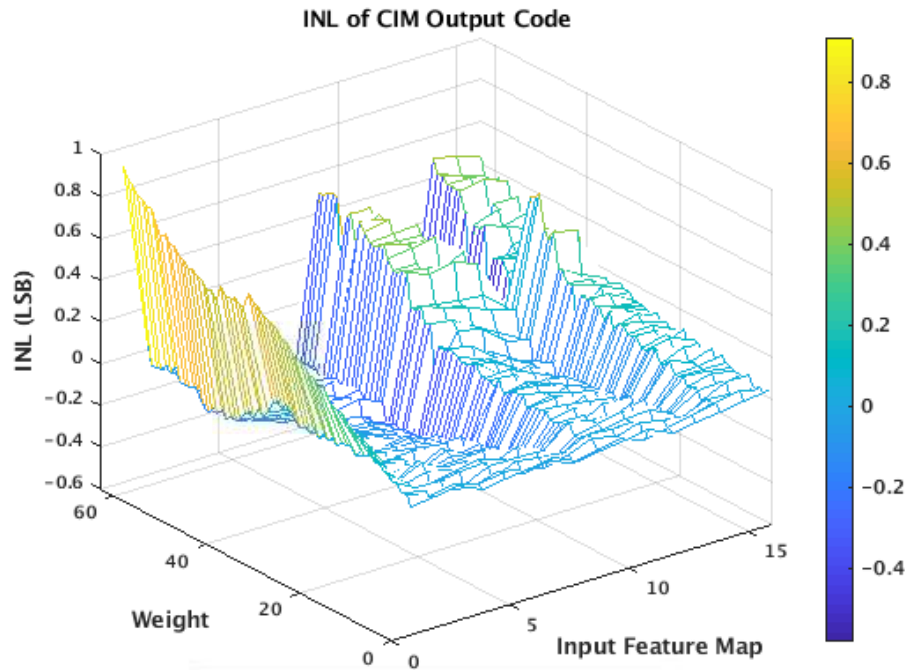


圖 4-21 CIM 輸出非線性曲線 (pre-sim.)

4.2.2 能量效益分析

能量效益之計算方式如式 1-3 所示，首先，模擬會針對當電路操作在 0.9V 的供應電壓且 TT corner 時，僅改變電路操作頻率下，能量效益變動情形。圖 4-22 為電路操作頻率與能量效益的變化情況，當操作在 100 MHz 時，整體的能量效益僅有 23.5 TOPS/W；當電路操作在 400 MHz 時，整體的能量效益則為 26.3 TOPS/W。當操作在低頻時，能量效益會下降的原因為 TDC 本身所使用到的 SA 屬於高速電路，當低頻時將會造成 leakage 增大，此外兩條的延遲線無 enable 機制，因此將增加待機時的能量消耗；在輸入方面，由於 PWG 對 weight block 的開啟時間是固定的，所以當電路操作於低頻時，僅會增加取樣電容保持電壓的時間，並無益於降低能量。因此整體導致在低頻操作時，功率消耗雖然較低，但對於能量效益來說，並無實質提升而下降。

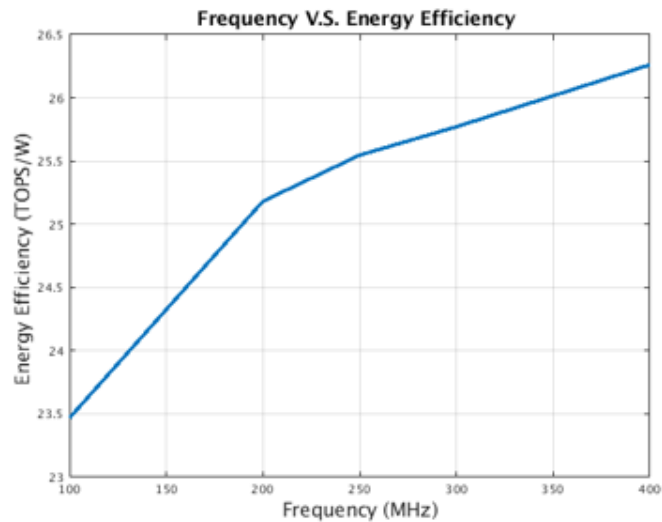


圖 4-22 操作頻率對能量效益之變化 (post-sim.)

接著則是當電路操作在 0.9V 的供應電壓且操作頻率為 200 MHz 時，僅改變電路操作的 corner 環境，能量效益變動情形。圖 4-23 為 corner 與能量效益的變化情況，當操作在 FF corner 時，整體的能量效益僅有 19.4 TOPS/W；當電路操作在其餘的 corner 時，若以 TT corner 為參考點，僅有不到6%的誤差。為了觀察在 FF corner 的能量效益下降的原因，因此改變了電路的操作頻率，觀察能量效益是否有所變化。當電路為 FF corner 且操作頻率為 333 MHz 時，能量效益提升到了 22.3 TOPS/W，因此推斷當電路操作於 FF corner 時，由於 PWG 的單位脈衝的變短，導致對 weight block 的開啟時間縮短，但當操作頻率不變時，只會增加取樣電容保持電壓的時間，造成能量效益的下降。

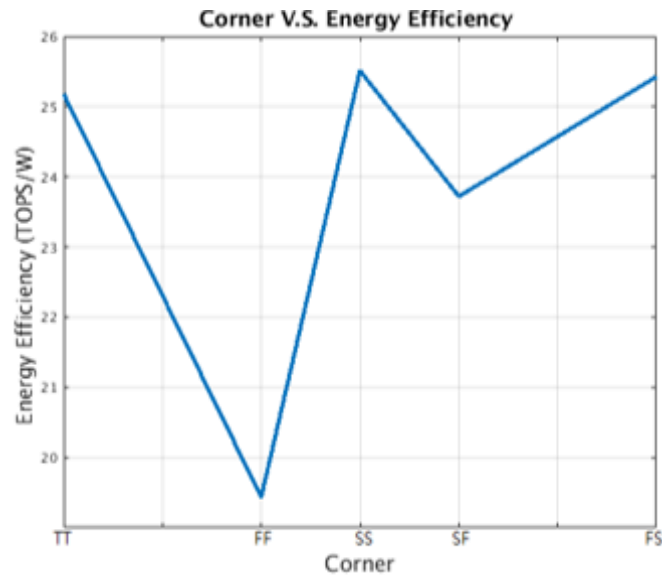


圖 4-23 Corner 對能量效益之變化 (post-sim.)

4.2.3 電路能量佔比

圖 4-24 為當電路操作在 0.9V 的供應電壓且操作頻率為 200 MHz 時，系統總功率消耗為 143 μ W，各個電路區塊的能量佔比圓餅圖。雖然在本論文供出了時間調變機制與權重位元累加機制，降低電路單元的使用量，以改善整體的功率消耗，但 PWG 的 25% 與 TDC 的 49% 仍佔了整體能量消耗的大宗，因此輸出級的資料轉器與 WBPA 機制在未來仍是需要改善的區域。

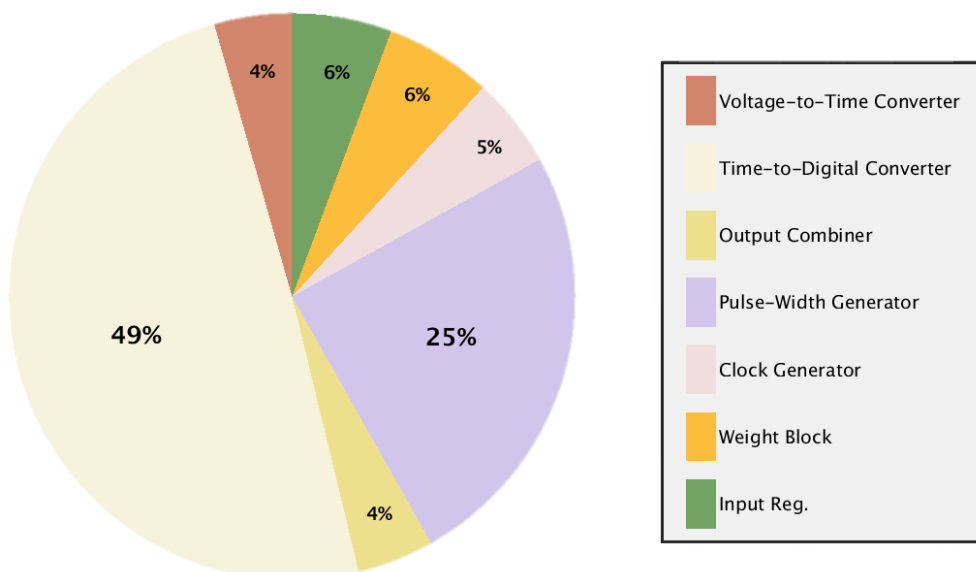


圖 4-24 電路區塊之能量佔比 (post-sim.)

4.3 晶片佈局

圖 4-25 為整體晶片佈局圖，此設計是採用 TN40G 製程，並使用 Encounter 及 Laker 佈局軟體完成，晶片大小為 1.18mmx1.18mm，並使用 CLCC 68-pin 做為封裝使用。在本次設計中，為了讓晶片能夠應用於後續 CNN 演算法當中，因此一共是放入了 4 個 tile 進行下線，由三條外部控制訊號做選擇。而 4.2.1 與 4.2.3 節中的模擬數據僅針對單一 tile 進行模擬。

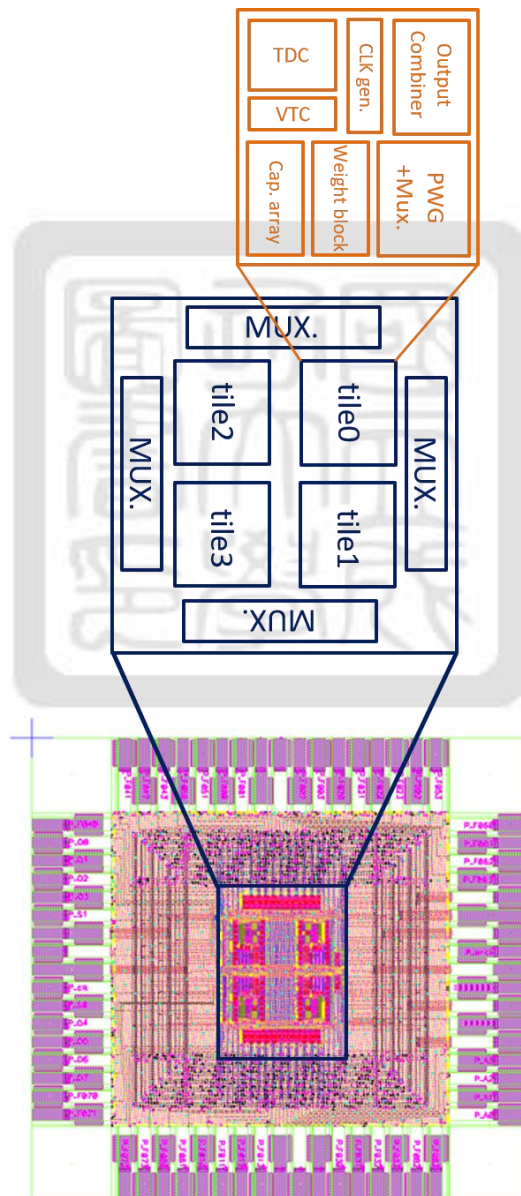


圖 4-25 記憶體內運算晶片佈局

第5章 結論與未來研究方向

5.1 結論

本論文所提出適用於 3×3 filter 的多位元權重及輸入資料計算之記憶體內運算具有高能量效益。透過時間調變機制，降低整體 DAC 的使用數量，由 $n \times n$ 降至一個 PWG 的使用，若與 9 個相同解析度之 4-bit charge-redistributon DACs 在相同模擬環境下比較，能節省約 1.35 倍的功率消耗，因此將有效的降低功率消耗與面積的使用；利用權重位元切割累加機制，降低整體輸出量化誤差，並與單一相同解析度之 8-bit VTDC 在相同模擬環境下比較，將能減少約 2.66 倍的 TDC cell 的使用，並且整體的操作速度將能加速 40 倍以上。表 5-1 為本論文實驗的結果與相關文獻[20] [22] [23]之效能比較，由於上述提出的機制，讓整體電路的操作速度、運算精準度與能量效益都有一定程度的優勢，因此在 $FOM_{precision}$ 與 $FOM_{performance}$ 兩項綜合指標，皆有一定的競爭程度。然而，由於本論文只實現了單一 3×3 filter 做為測試行為驗證，在單位位元消耗（Area per bit）上的表現是落後於相關文獻。其原因為在 PWG、IRA 與時脈產生器皆可以在不同 filter 上重複使用，因此在位元密度上升後，將能彌補此效能的差距。

表 5-1 相關文獻比較

	Proposed	ISSCC'18 [23]	ISSCC'19 [20]	ISSCC'20 [22]
Process	40 nm	65 nm	55 nm	7 nm
SRAM Type	8T	10T	T8T(16T)	8T
Computation	Multiple–Weight Multiple–IFMP	Binary–Weight Multiple–IFMP	Multiple–Weight Multiple–IFMP	Multiple–Weight Multiple–IFMP
SRAM Size	54-bit	16K-bit	64×60-bit	4K-bit
Macro Area Per Bit	85.21 $\mu\text{m}^2/\text{bit}$	4.09 $\mu\text{m}^2/\text{bit}$	9.64 $\mu\text{m}^2/\text{bit}$	0.78 $\mu\text{m}^2/\text{bit}$
Input Precision	4-bit	6-bit	4-bit	4-bit
Weight Precision	6-bit	1-bit	5-bit	4-bit
Output Precision	8-bit	6-bit	7-bit	4-bit
Inference Time	5 ns	150 ns	10.2 ns	4.5 ns
Throughput	3.6 GOPS	8 GOPS	21.2 GOPS	372.4 GOPS
Energy Efficiency	25.2 TOPS/W	40.3 TOPS/W	18.37 TOPS/W	262.3 TOPS/W
FOM for Precision*	4.59	2.59	4.32	4
FOM for Performance**	4.14	2.18	2.92	5.29

* $\text{FOM}_{\text{precision}} = \log_2(B_{\text{weight}} \times B_{\text{input}})$ (bit^2) [20]

** $\text{FOM}_{\text{performance}} = \log_{10}(\text{Energy Efficiency} \times \text{Throughput}/\text{SRAM size})$ ($\text{TOPS/W} \times \text{GOPS}/\text{KB}$) [23]

5.2 未來工作

本小節將針對本篇論文所提出的架構，在未來的工作可以針對以下幾點進行發展與改進。

■ 數據量測

對本論文所提出之記憶體內運算雛形晶片量測相關數據與紀錄分析。

■ 輸入精準度的提高

在文獻[46]的實驗中，當輸入精準度上升時，輸出的精準度隨之正比提高，因此可以得知，如果想要有效提高演算法的判斷精確度，將輸入精準度的提高比權種精準度提高還來得有效。然而當輸入精準度提高時，會導致幾個問題發生：首先脈波寬度產生器所需要產生的脈波變多，將造成電路執行 inference 的速度下降；接著在 RBL 訊號線上的 MACs 計算結果會變大，將造成讀取路徑中 MOS 的 V_{DS} 急遽縮小， I_{MC} 的電流不再保持常數，導致整體輸出的非線性；最後由於 RBL 訊號線上的 MACs 運算訊號範圍變小，因此更容易受到製程變異的影響，造成運算錯誤。因此在提高輸入精準度的同時，將必須考慮上述所提及之問題，否則可能導致整體設計的 OFMP 精準度下降。

■ 權重位元切割累加機制改良

雖然 WBPA 機制在記憶體內運算電路的效能，得到了大幅的改善，但輸出級的能量消耗仍佔據了整體電路達 49% 之多，如圖 4-24 所示，因此本機制尚有改善之空間。首先，可以透過調整每個 sub-TDC 的解析度大小，讓 MSB 的 TDC 擁有較高的解析度，而越靠近 LSB 的 TDC 則可以設計成較低解析度，以改善能量問題；除了客製化每個 sub-TDC 解析度外，接著也能先將未經過量化單位元 weight 與多位元 IFMP 的 MACs，先利用類比的手段，將每兩個或三個的單位元 weight 與多位元 IFMP 的 MACs 先行累加，如 charge sharing 等方法，再透過 sub-TDC 進行量化，減少 TDC 的使用，以優化整體能量消耗。

■ 可調式 filter 之設計

在實際演算法應用上，每一層 MACs 計算並不是統一的 filter 大小，最常見的有 1×1 、 3×3 與 5×5 的形式，因此為了能夠更靈活的應用在系統中與減少更多資料的搬移，可調式 filter 的 CIM 將成為不可缺少的功能之一。但可調式 filter 意味著 RBL 訊號上的取樣電容將隨之改變或是輸出級的資料轉換器之量化範圍的改變，因此如何進行電路的切換以保持 OFMP 的精準度，將成為此設計之挑戰。



參考文獻

- [1] Y. LeCun, L. Jackel, B. Boser, J. Denker, H. Graf, I. Guyon, *et al.*, “Handwritten Digit Recognition: Applications of Neural Network Chips and Automatic Learning,” *IEEE Communications Magazine*, vol. 27, no. November, pp. 41–46, 1989.
- [2] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, *et al.*, “ImageNet Large Scale Visual Recognition Challenge,” *International Journal of Computer Vision*, pp. 248–255, 2015.
- [3] A. Krizhevsky, *et al.*, “ImageNet Classification with Deep Convolutional Neural Networks,” *Proceedings of the 25th International Conference on Neural Information Processing Systems*, 2012, pp. 1097–1105.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [5] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, *et al.*, “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” 2017. [Online]. Available: <http://arxiv.org/abs/1704.04861>.
- [6] K. L. Loh, “Fertilizing AIoT from Roots to Leaves,” in *Proc. IEEE International Solid- State Circuits Conference - (ISSCC)*, 2020, pp. 15–21.
- [7] FRAMINGHAM, and Mass, “The Growth in Connected IoT Devices Is Expected to Generate 79.4ZB of Data in 2025, According to a New IDC ForecastTitle,” *IDC*, 2019. .
- [8] J. C. Talwana, and H. J. Hua, “Smart World of Internet of Things (IoT) and Its Security Concerns,” in *Proc. IEEE International Conference on Internet of*

- Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2016, pp. 240–245.
- [9] M. Gusev, and S. Dustdar, “Going back to the roots - The evolution of edge computing, an IoT perspective,” *IEEE Internet Computing*, vol. 22, no. 2, pp. 5–15, 2018.
 - [10] H. J. Yoo, “Intelligence on Silicon: From Deep-Neural-Network Accelerators to Brain Mimicking AI-SoCs,” in *Proc. IEEE International Solid-State Circuits Conference*, 2019, pp. 20–26.
 - [11] Y. H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
 - [12] B. Moons, R. Uytterhoeven, W. Dehaene, and M. Verhelst, “Envision: A 0.26-to-10TOPS/W subword-parallel dynamic-voltage-accuracy-frequency-scalable Convolutional Neural Network processor in 28nm FDSOI,” in *Proc. IEEE International Solid-State Circuits Conference*, 2017, vol. 60, pp. 246–247.
 - [13] Y. H. Chen, J. Emer, and V. Sze, “Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks,” in *Proc. 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.
 - [14] S. Park, K. Bong, D. Shin, J. Lee, S. Choi, and H. Yoo, “A 1.93TOPS/W Scalable Deep Learning /Inference Processor with Tetra-Parallel MIMD Architecture for Big-Data Applications,” in *Proc. 2015 IEEE International Solid-State Circuits Conference - (ISSCC) Digest of Technical Papers*, 2015, pp. 80–82.
 - [15] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, *et al.*, “ShiDianNao:

- Shifting vision processing closer to the sensor,” *Proceedings - International Symposium on Computer Architecture*, vol. 13-17-June, pp. 92–104, 2015.
- [16] K. Simonyan, and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. 3rd International Conference on Learning Representations, ICLR 2015*, 2015, pp. 1–14.
- [17] K. T. Tang, W. C. Wei, Z. W. Yeh, T. H. Hsu, Y. C. Chiu, C. X. Xue, *et al.*, “Considerations of Integrating Computing-In-Memory and Processing-In-Sensor into Convolutional Neural Network Accelerators for Low-Power Edge Devices,” in *Proc. IEEE Symposium on VLSI Circuits, Digest of Technical Papers*, 2019, vol. 2019-June, pp. T166–T167.
- [18] W. H. Chen, K. X. Li, W. Y. Lin, K. H. Hsu, P. Y. Li, C. H. Yang, *et al.*, “A 65nm 1Mb nonvolatile computing-in-memory ReRAM macro with sub-16ns multiply-and-accumulate for binary DNN AI edge processors,” in *Proc. IEEE International Solid-State Circuits Conference*, 2018, vol. 61, pp. 494–496.
- [19] W. S. Khwa, J. J. Chen, J. F. Li, X. Si, E. Y. Yang, X. Sun, *et al.*, “A 65nm 4Kb algorithm-dependent computing-in-memory SRAM unit-macro with 2.3ns and 55.8TOPS/W fully parallel product-sum operation for binary DNN edge processors,” in *Proc. IEEE International Solid-State Circuits Conference*, 2018, vol. 61, pp. 496–498.
- [20] X. Si, J. J. Chen, Y. N. Tu, W. H. Huang, J. H. Wang, Y. C. Chiu, *et al.*, “A Twin-8T SRAM Computation-In-Memory Macro for Multiple-Bit CNN-Based Machine Learning,” in *Proc. IEEE International Solid-State Circuits Conference*, 2019, pp. 396–398.
- [21] M. Kang, S. K. Gonugondla, S. Lim, and N. R. Shanbhag, “A 19.4-nJ/Decision, 364-K Decisions/s, In-Memory Random Forest Multi-Class Inference

- Accelerator,” in *Proc. IEEE Journal of Solid-State Circuits*, 2018, vol. 53, no. 7, pp. 2126–2135.
- [22] Q. Dong, M. E. Sinangil, B. Erbagci, D. Sun, W. Khwa, H. Liao, *et al.*, “A 351TOPS/W and 372.4GOPS Compute-in-Memory SRAM Macro in 7nm FinFET CMOS for Machine-Learning Applications,” in *Proc. IEEE International Solid-State Circuits Conference - (ISSCC)*, 2020, pp. 488–489.
- [23] A. Biswas, and A. P. Chandrakasan, “Conv-RAM: An Energy-Efficient SRAM with Embedded Convolution Computation for Low-Power CNN-Based Machine Learning Applications,” in *Proc. IEEE International Solid-State Circuits Conference*, 2018, vol. 61, pp. 488–490.
- [24] C. X. Xue, W. H. Chen, J. S. Liu, J. F. Li, W. Y. Lin, W. E. Lin, *et al.*, “A 1Mb Multibit ReRAM Computing-In-Memory Macro with 14.6ns Parallel MAC Computing Time for CNN Based AI Edge Processors,” in *Proc. IEEE International Solid-State Circuits Conference*, 2019, pp. 388–390.
- [25] R. Guo, Y. Liu, S. Zheng, S.-Y. Wu, P. Ouyang, W. S. Khwa, *et al.*, “A 5.1pJ/Neuron 127.3us/Inference RNN-based Speech Recognition Processor using 16 Computing-in-Memory SRAM Macros in 65nm CMOS,” in *Proc. 2019 Symposium on VLSI Circuits Digest of Technical Papers*, 2019, pp. 6351–6355.
- [26] J. Yue, Z. Yuan, X. Feng, Y. He, Z. Zhang, X. Si, *et al.*, “A 65nm Computing-in-Memory-Based CNN Processor with 2.9-to-35.8TOPS/W System Energy Efficiency Using Dynamic-Sparsity Performance-Scaling Architecture and Energy-Efficient Inter/Intra-Macro Data Reuse,” in *Proc. IEEE International Solid-State Circuits Conference*, 2020, pp. 234–236.
- [27] S. K. Gonugondla, M. Kang, and N. Shanbhag, “A 42pJ/Decision 3.12TOPS/W Robust In-Memory Machine Learning Classifier with On-Chip Training,” *IEEE*

- International Solid-State Circuits Conference*, vol. 61, pp. 490–492, 2018.
- [28] J. W. Su, X. Si, Y. C. Chou, T. W. Chang, W. H. Huang, Y. N. Tu, *et al.*, “A 28nm 64Kb Inference-Training Two-Way Transpose Multibit 6T SRAM Compute-in-Memory Macro for AI Edge Chips,” in *Proc. IEEE International Solid-State Circuits Conference*, 2020, pp. 240–242.
- [29] W. Chen, W. Lin, L. Lai, S. Li, C. Hsu, H. Lin, *et al.*, “A 16Mb Dual-Mode ReRAM Macro with Sub-14ns Computing-In-Memory and Memory Functions Enabled by Self-Write Termination Scheme,” in *Proc. IEEE International Electron Devices Meeting (IEDM)*, 2017, pp. 657–660.
- [30] I. Giannopoulos, A. Sebastian, M. LeGallo, V. P. Jonnalagadda, M. Sousa, M. N. Boon, *et al.*, “8-bit Precision In-Memory Multiplication with Projected Phase-Change Memory,” in *Proc. IEEE International Electron Devices Meeting (IEDM)*, 2018, pp. 27.7.1–27.7.4.
- [31] H. Farkhani, M. Tohidi, S. Farkhani, J. K. Madsen, and F. Moradi, “A Low-Power High-Speed Spintronics-Based Neuromorphic Computing System Using Real-Time Tracking Method,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 8, no. 3, pp. 627–638, 2018.
- [32] Y. Shi, S. Oh, Z. Huang, S. Member, X. Lu, S. H. Kang, *et al.*, “Performance Prospects of Deeply Scaled Spin-transfer Torque Magnetic Random-access Memory for In-memory Computing,” in *Proc. IEEE Electron Device Letters*, 2020, vol. 3106, no. c, pp. 1–4.
- [33] J. Zhang, Z. Wang, and N. Verma, “In-Memory Computation of a Machine-Learning Classifier in a Standard 6T SRAM Array,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 4, pp. 915–924, 2017.
- [34] A. Jaiswal, I. Chakraborty, A. Agrawal, and K. Roy, “8T SRAM Cell as a

- Multibit Dot-Product Engine for beyond Von Neumann Computing,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 11, pp. 2556–2567, 2019.
- [35] X. Si, H. Qian, M. F. Chang, C. X. Xue, J. W. Su, Z. Zhang, *et al.*, “Circuit Design Challenges in Computing-in-Memory for AI Edge Devices,” in *Proc. 2019 IEEE 13th International Conference on ASIC (ASICON)*, 2019, pp. 1–4.
- [36] J. M. Hung, X. Li, J. Wu, and M. F. Chang, “Challenges and Trends in Developing Nonvolatile Memory-Enabled Computing Chips for Intelligent Edge Devices,” *IEEE Transactions on Electron Devices*, vol. 67, no. 4, pp. 1444–1453, 2020.
- [37] Z. Jiang, S. Yin, M. Seok, and J. S. Seo, “XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks,” in *Proc. Digest of Technical Papers - Symposium on VLSI Technology*, 2018, pp. 173–174.
- [38] M. Kang, S. K. Gonugondla, A. Patil, and N. R. Shanbhag, “A Multi-Functional In-Memory Inference Processor Using a Standard 6T SRAM Array,” *IEEE Journal of Solid-State Circuits*, vol. 53, no. 2, pp. 642–655, 2018.
- [39] K. Ohhata, “A 2.3-mW, 1-GHz, 8-Bit Fully Time-Based Two-Step ADC Using a High-Linearity Dynamic VTC,” *IEEE Journal of Solid-State Circuits*, vol. 54, no. 7, pp. 2038–2048, 2019.
- [40] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, “XNOR-net: Imagenet classification using binary convolutional neural networks,” *Lecture Notes in Computer Science*, vol. 9908 LNCS, pp. 525–542, 2016.
- [41] Z. Cao, S. Yan, and Y. Li, “A 32mW 1.25GS/s 6b 2b/step SAR ADC in 0.13 μ m CMOS,” in *Proc. IEEE International Solid-State Circuits Conference*, 2008, vol. 51, pp. 542–544.

- [42] B. Razavi, "TSPC Logic [A Circuit for All Seasons]," *IEEE Solid-State Circuits Magazine*, vol. 8, no. 4, IEEE, pp. 10–13, 2016.
- [43] T. E. Rahkonen, and J. T. Kostamovaara, "The Use of Stabilized CMOS Delay Lines for the Digitization of Short Time Intervals," *IEEE Journal of Solid-State Circuits*, vol. 28, no. 8, pp. 887–894, 1993.
- [44] J. F. Lin, Y. T. Hwang, C. S. Wong, and M. H. Sheu, "Single-ended structure sense-amplifier-based flip-flop for low-power systems," *Electronics Letters*, vol. 51, no. 1, pp. 20–21, 2015.
- [45] D. J. White, P. E. William, M. W. Hoffman, and S. Balkir, "Low-power analog processing for sensing applications: Low-frequency harmonic signal classification," *Sensors (Switzerland)*, vol. 13, no. 8, pp. 9604–9623, 2013.
- [46] J. Yang, Y. Kong, Z. Wang, Y. Liu, B. Wang, and S. Yin, "Sandwich-RAM: An Energy-Efficient In-Memory BWN Architecture with Pulse-Width Modulation," in *Proc. IEEE International Solid- State Circuits Conference - (ISSCC)*, 2019, pp. 394–396.